Anders Fisk

2224

Midhurst Rother College

65423

A Disease Model Incorporating SIR Principles in Python

H446

22.03.2024

Table of Contents

Analysis

Project Introduction

This project will take a set of initial parameters of a given region such as recovery rate, infection rate of the region and total population number which will be used as initial values for the population in the SIR mathematical model and for interactions between agents in the agent-based model.

These SIR model equations will then be incorporated into an agent-based modelling system to stimulate the interactions of defined 'agents' within a system which can be used to model the disease spread. The advantage of using an agent-based system is that each agent can be graphically represented as a, for example, a dot with each colour of each dot representing a different SIR category as the population changes.

As to how the agent-based model will work, defined 'interactions' between the agents which will determine the number of Susceptible (S) Infected (I) and Recovered (R) used to show how the population reacts to a disease. These defined 'agents' will be the members of the population itself with their initial characteristics including susceptibility, infectiousness and recovery time which will be used in the SIR model. The defined 'interactions' between each agent and how the disease spreads will be defined by the transmission rate of the pathogen itself as well as the characteristics of the agent for example, if they're infected the probability to pass it on to a susceptible. To simulate this disease through agent-based modelling it will integrate the SIR model which in turn will involves calculate the number of agents in each category of the model at each time step based on the recovery rate and infection rate and the number of interactions between agents.

An SQL database will be used within the model to store the various values as they change throughout the simulation. The SIR model and agent-based simulation will both interact with this database using SQL queries to modify and input data. A defined 'population' table will be used to store the values of the SIR model while a defined 'individuals' table will be used to store the characteristics of each individual / agent in the simulation creating a relational database which different parts of the program can access universally. However, the structure of this database may change depending on how the software itself is structured during development.

A GUI will be used alongside agent-based modelling to graphically show the interactions between 'agents' which in turn will show the evolution of how the population responds to the disease.

Why This Problem is Suitable for a Computational Solution

The problem of visualising and calculating the spread of a disease is a computational problem. One reason for this is the method to calculate the spread of a disease is a mathematical model called the SIR model. This model uses a series of mathematical equations to calculate different values such as number of infected, number of recovered, etc. This is suited for a computer since many mathematical operations can be performed per second within the computer's system architecture therefore these values can be calculated exponentially quicker than if a pen and paper were used to calculate the results of these equations. It also means that as the algorithms are calculated, the calculated values per second don't need to be recorded manually.

Another reason this problem is suitable for a computational solution is how the results of these calculations need to be relayed to the user. Programming can be used to output the data calculated in the form of a graph within a digital graphical user interface. This allows the user to easily see the

trends and relationships between data as well as output it quickly and easily. This computational method is much easier than having to either draw the graphs by hand or explain the results of the data with raw data and no visual aid. It also means that as the values change, the graphs can change simultaneously instead of a new graph needing to be manually drawn with a pen each time parameters at the start or during the simulation are changed.

Finally, multiple simulations can be run successively one after another simply by changing parameters within the program, with the computer repeating all the calculations with these new values inserted instead of the previous ones. If this were to be done manually, all the calculations would need to be re-calculated by hand each time the user wanted a new simulation to be run which would be heavily time-consuming and not efficient.

Stakeholders

The stakeholders for this project will be users of a computer as well as being involved in the subject of biology. These users can be split into two groups. The first group will consist of mainly casual users who will only use the program as a visual aid for understanding the spread of a disease. The second group will consist of more specialised users, who will use the data that the program produces mainly for research.

The first group will consist of secondary school students who take biology as a subject, whether this is at GCSE or A-level. Students are a suitable stakeholder for this project since diseases and immunity are part of the biology curriculum, the project itself will visually display how a disease spreads in a population, meaning teachers and students can use this as a visual aid to help with their understanding. This is needed since many students often have different approaches in how they understand different concepts. This program will provide a visual alternative for students who may struggle with the idea of how a disease interacts with a population through only information given by the teacher. This can help engrain the concepts taught in the A-level curriculum such as immunity / primary immune responses by contextualising and visually demonstrating these concepts in a real life scenario which students can relate to, in this case an epidemic. It can also be used collaboratively with multiple students in a classroom environment to show the rates of different diseases simultaneously to demonstrate the difference between the transmission rates of different diseases. Furthermore, the program will have the ability to change certain parameters surrounding the disease such as transmission rate, chance of infection etc. This can be used in the classroom to demonstrate how different characteristics about the disease change how the disease behaves in a population, which could lead to a lesser or more severe epidemic.

The second group will consist of people that use the data produced by the programme. This can include researchers, epidemiologists or anyone who needs specific data on how a disease spreads. Involved in the model are numerous values that give information about the disease such as number of infected, number of recovered, and the transmission rate of the disease itself. These values are calculated and change constantly, based on some predetermined factors such as the size of the population and the chance of transmission, within the model and can be used as data to give an insight on how severe or infectious a disease is. For example, the software could be used in the context of an epidemic to see how severe the outcome will be in a subset of the population such as a county. The specific characteristics of the county can be inputted such as number of interactions between agents, transmission chance, and the size of their population to give a prediction on how severe an impact the disease will have. This means the countermeasures can found and taken by

epidemiologists to reduce the impact the disease will have on any given population depending on what the program simulates the outcome to be.

I will tailor the project further to the needs of the first group rather than the first, This is because the visual aspect of the program is the fundamental aim of this program since the goal is to relay the spread of a disease through a population and the group that will find that the most useful are the student group since it can help with understanding the topic through visual learning. It won't be as useful towards group two since they might be expecting a more scientific output in the form of statistical values about the disease such as the mortality rate and although those can be added post-development, it is not the focus of the program. Therefore, the interviews from the stakeholders in group one will be of a more valuable input than those of group two.

Interviews

My focus of these interviews will be to see how the different groups of stakeholders would interact with the programme as well as the features they'd like to see implemented for each of them to make the most use out of the program respectively.

Group 1 – Casual Users

I've selected three A-level biology students to represent the casual group for this program. They will be briefed about the program and these questions will be focused more on the visual / GUI aspect of the program and how they would interact with it in a classroom environment, as well as any useful features they'd like to see implemented within the program:

1. Do you usually use a computer during lessons?
2. How much knowledge do you have about how diseases spread?
3. Do you like to understand the reasoning behind the method to solve a problem or just memorising the method to solve it?
4. How would you imagine the spread of a disease to look like graphically?
5. What is your favourite studying technique?
6. What do you find is your best approach to understanding a concept?
7. Do you have any suggestions for what you would want a program that simulates the spread of a disease to do?

The first two questions are designed to see how much experience the stakeholder has with technology and the topic this program is about.

Questions three and four are to get ideas on how detailed or abstracted the interface should be to a typical biology student.

Questions five and six are there to gather ideas on how to most effectively visually display the spread of a disease, so it can be easily understood by a student.

Question seven is there if the stakeholder wants to give any extra feedback or has any suggestions for how the program should operate.

Group 2 – Specialised Users

I've selected my biology tutor to represent the specialised user group for this program. This set of questions will be more heavily focused towards the data-side of the program with less emphasis on the interface and more emphasis on what data would be useful towards research or epidemiology:

1. How much experience do you have with using a computer for research?
2. How much experience do you have with the spread of diseases?

3. How familiar are you with the SIR model?
4. Would you prefer a simpler interface with less control of how the program operates?
5. What functional features in a program are important for researching the spread of diseases?
6. What data would you like to see from this program?
7. Would the data be generated from this program be useful for real world applications?
8. Do you have any other suggestions for this program

Questions one, two and three are there to gauge how much experience the stakeholder has with the topic of the program.

Questions four and five are then there to see how the stakeholder would prefer to operate the program as well as any features that may be useful.

The next two questions are to do with the data and how it can be implemented and used.

Question eight is there if the stakeholder wants to give any extra feedback or has any suggestions for how the program should operate.

Interviews With Group One:

Taylor – An A-level Biology Student

1. **Do you usually use a computer during lessons?**
   "Yes"
2. **How much knowledge do you have about how diseases spread?**
   "Yes"
3. **Do you like to understand the reasoning behind the method to solve a problem or just memorising the method to solve it?**
   "I like to understand the reasoning behind the problem rather than just being able to solve it"
4. **How would you imagine the spread of a disease to look like graphically?**
   "Depending on the location, it likely would be a spike with then declining curve as the disease spreads to people. Immune system responses would then take over and reduce the curve further."
5. **What is your favourite studying technique?**
   "Looking through the textbook and picking out key information to remember, then going back over it"
6. **What do you find is your best approach to understanding a concept?**
   "Researching it"
7. **Do you have any suggestions for what you would want a program that simulates the spread of a disease to do?**
   "Nope"

Katie – An A-level Biology Student

1. **Do you usually use a computer during lessons?**
   "yes-well an iPad"
2. **How much knowledge do you have about how diseases spread?**
   "*Lots"*
3. **Do you like to understand the reasoning behind the method to solve a problem or just memorising the method to solve it?**
   "I like to understand the reasoning behind a problem"

4. **How would you imagine the spread of a disease to look like graphically?**
   "Exponential growth unless there's an intervention"
5. **What is your favourite studying technique?**
   "I'm definitely a visual-learner"
6. **What do you find is your best approach to understanding a concept?**
   "*Videos, questions and pictures/diagrams"*
7. **Do you have any suggestions for what you would want a program that simulates the spread of a disease to do?**
   "Base it on population in countries and their health care system"

The insights I got from these interviews is that the program will be useful in an educational setting for students that want a better understanding of the topic through visual information but may not be applicable to those who remember information by reading from the textbook. Interestingly how she imagined the spread of a disease is quite different to how one behaves. Suggesting that maybe guidance should be included in the program for how the display of the disease should be interpreted. Finally, the request to base disease modelling off countries and health care systems is a good idea, however it's inefficient to make pre-sets of data for every country and health system around the world. For this reason, a pre-set system based on user inputted values will be included in the program, meaning users can search up the specific disease / healthcare details of different countries and input the data into the program themselves to see how the disease would spread in that situation. They will also be able to name the pre-sets, meaning country / healthcare pre-sets can be created by the user.

Interviews With Group Two:

Mr Ovett – A biology A-level teacher

1. **How much experience do you have with using a computer for research?**
   "I have excessive experience in using a computer for research"
2. **How much experience do you have with the spread of diseases?**
   "I have a moderate to good experience with epidemiology"
3. **How familiar are you with the SIR model?**
   "Not familiar whatsoever"
4. **Would you prefer a simpler interface with less control of how the program operates?**
   "A simpler interface with fewer options because I feel as if there is only so much functionality that can be added to interacting with a disease model"
5. **What functional features in a program are important for researching the spread of diseases?**
   "Population statistics, mortality rates, recovery rates, transmission rates"
6. **What data would you like to see from this program?**
   "I would definitely like to see sufficient scientific data produced from this program, e.g the resultant fatalities and stats about the disease once it has moved through the population"
7. **Would the data be generated from this program be useful for real world applications?**
   "I think in principle this program will be useful however it will have to have sufficient complexity to be useful in real-world applications"
8. **Do you have any other suggestions for this program**
   "No I don't"

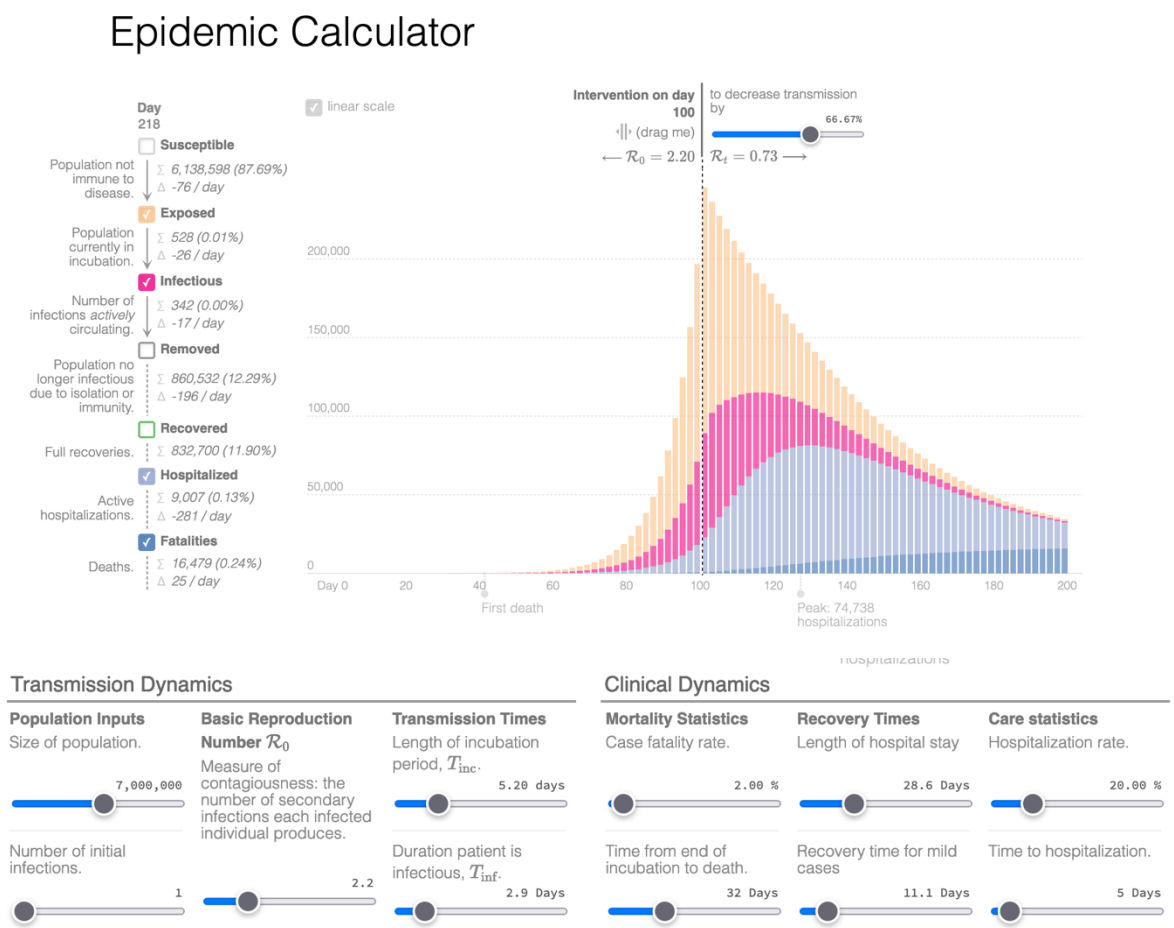
I didn't really get much insight from this interview on how the program could be applied in a scientific setting since Mr Ovett hadn't worked in the field of epidemiology for quite several years and wasn't familiar with the fundamental model that the program used. However, eI did gain some valuable insights which was that a simpler interface would be preferred even in a scientific setting and it was reassuring to hear that the features he wanted the program to include would all be part of the SIR model so will definitely be present in the final developmental build of the program. Therefore, it's useful to know that although the program is geared towards students it may still have some use in fields of research.

Research

Existing Similar Solutions

Epidemic Calculator

Overview:



This program is called 'Epidemic Calcuator' and is based on a similar mathematical model that will be used in my program. In this case they have used a slightly more complex mode called the 'SEIR' model which adds the 'Exposed' and 'Removed' instead of 'Recovered' cateogry for members of the population to be sorted into. The way this program works is that differntial equations are used in set time intervals to calcuate and categorise the amount of people in each category at any given time.

The aim of this program is to model the trasmission of the disease through the populaiton as well as the burden the disease will have on hospitals. The paramaters used in this simulation can be

changed and customsied depending on the user's needs. These starting paramaters are split into two categories. 'Transmission Dynamics', which are paramaters that affect the transmission rate of the disease. Examples of this are initial population size, incubation period and the number of initial infected. Altering these paramters will change the transmission pattern of the disease. The second cateogry is 'Clinical Dynamics'. This category includes paramaters such as recovery times, fatality rate and hospitilisation rate. Altering these will change the amount of total fatalities as well as the amount of time that each member in the population is hospitalised.

All this data is then outputted in the form of values on a graph. The graph itself is colour-coded in respect to each category making it intuitive and easy to understand. The format of the graph is as a bar chart which is useful since it shows the different values of each category as different heights, meaning it once again makes the graph easy to understand since you can see the largest and smallest values easily.

Parts that I can apply to my Solution:

Most of the program can be applied to my solution. I like the idea of being able to specify the different parameters for the simulation and seeing that change in the graph real-time since it allows greater interactivity with the program by the user and means multiple scenarios can be tested in quick succession by the user to see how different values affect the spread of the disease, therefore I can apply their approach to changing data parameters in my solution.

Some parts of the UI can be applied to my solution. My program will not be using a graph to represent data. This is because I want to take more of a visual approach by emphasizing how the disease 'spreads. The best way I feel to do this is to use a 2D grid of squares with each square representing a member of the population. However I can apply their approach of colour-coding each category of the population since it makes the simulation much easier to understand since the user can see the difference between the Susceptible, Infected and Recovered members of the population at a glance. It also emphasises the movement of the disease since the squares will change colour as the disease interacts with them creating a 'wave' effect as the squares change colour. Therefore, in my program I will assign each square / member of the population within the 2D grid a distinct colour based on which category they are currently in, this can change as the program runs.

Outbreak

Overview:

This program is called 'Outbreak' and is the most similar solution to my program that I've found. It uses the SIR model as it's basis for calculations. The aim of the program is to simulate a disease outbreak based on the COVID-19 epidemic, while being able to tweak parameters to affect how the simulation unfolds. Due to the program using COVID-19 as it's basis for a disease model it involves additional parameters such as self-quarantine and incubation periods, meant to reference the COVID-19 pandemic.

Its user interface is simple with-it presenting data using a 2D grid with each square representing a member of the population as well as each square being colour-coded into a certain category depending on its current state. Below the grid are sliders for the starting parameters of the disease which the user can alter by adjusting the respective sliders. It also includes the standard way of presenting data at the very bottom presenting graphs for each individual simulation next to each other for easy comparison by the user.

The functions for the simulation are also simply designed with the three controls for the simulation being straight underneath the grid in grey boxes with the recognisable 'play' button being light blue to stand out

Parts that I can apply to my Solution:

The presentation of data in a 2D grid in this program is near identical for how I want to present data in my own program. The reason for this is because the user can see in real time how a disease spreads since the way the squares change colour, depending on their respective category, is a great

way to intuitively understand how a disease moves through a population just from looking at it. However, the sliders in this program are too simplified for my liking. There's no explanation for which each slider does, meaning the user of the program may not understand exactly the effect adjusting each slider has on the simulation. I like the adjustable values, however since my program is aimed at students, I will add a brief explanation for what adjusting each value does within a tutorial section.

At the bottom of the page the program presents traditional graphs side by side. I really like this UI feature since it allows easy comparisons between each simulation that is run by the user and allows the user to easily see the changes when parameters are adjusted in distinct simulation iterations. This may be too complex to implement in my program, however my program will user pre-sets so users can run multiple sets of parameters consecutively to see how changing each parameter of the simulation will change the disease spread through the population.

Features of the Proposed Solution

The Initial Concept of my Solution

My solution to this problem will be a program which will allow the user to run simulations of different diseases moving through a population. The parameters of the population and disease will be entirely customisable so the user can see how changing the characteristics of a disease changes how it behaves in a population. The user will also be able to create and use pre-sets for different parameter values so they can run different iterations of the simulation quickly while loading different pre-sets consecutively to look for comparisons between different simulations.

The main section of the program will consist of a menu which will contain different options for different parts of the program. There will be a button to the main section of the programme titled 'Full Simulation'. This will take up most of the page. Next to this button will be a button titled 'Pre-sets'. This will lead to a page that can be used to create and alter named pre-sets for different diseases / simulation parameters. Finally, there will be a button underneath the 'Pre-sets' button titled 'Tutorial'. This will lead to a page which will take the user through how the program works, the different controls and what each parameter means as well as the effect adjusting it has on the simulation.

The 'Full Simulation' section of the program will be a 2D grid with adjustable parameters underneath. To the side of this grid will be a list of pre-sets which can be clicked on and loaded into the simulation. These can be used for easy comparisons by the user since different diseases can be instantaneously loaded and ran so the user can see the differences and similarities between them. Underneath the 2D grid will be a menu consisting of four buttons. To the very left will be the play button which will play out the simulation at the standard speed. Next to that on the right will be a pause button which can be used to pause the simulation if the user wants to study the current state of the simulation. The last two buttons will be a forward and backward button, these can be used to increment the simulation frame by frame so the user can study the simulation in more detail. These can be used by the user to increment the simulation frame by frame if they want to study the simulation step by step to see how the disease spreads every single time-step.

The 'Pre-sets' section of the program will allow the user to create or alter different pre-sets of parameter values for the simulation. A list of pre-sets will be displayed with the option to alter them when one is clicked on. Below the list will be two buttons which can be used to either create a new pre-set or delete one from the list above. These pre-sets will be stored in an external database which can be accessed by the program, meaning that the pre-sets are not restricted to one section of the program and can be loaded by the 'Full Simulation' and used. Once a pre-set is clicked on it will take

the user to a separate page where all the pre-set's values can be changed. There will be a 'Discard' and a 'Save' button on this page. Once the user is satisfied with the changes they have made to the pre-set, they can press the 'Save' button and the changes will be saved to the external SQL database. If the users are unhappy with the changes they've made, they can press the 'Discard' button which will take the users back to the 'Pre-sets' page and no changes will be made to the external SQL database. The 'Add' button will lead to a similar page, however the value boxes will be blank and left for the user to fill in.

The tutorial section of the program will explain the different functions and parameters of the programs using a combination of paragraph and associated diagrams to explain to the user how the program works. The section itself will consist of multiple pages each explaining a different part of the program etc. one page will explain the 2D grid imagery and what the different colours represent while another will explain the different parameters. This is so the user can get an idea of how the program works and will be able to use it effectively without prior knowledge.

Limitations of my Solution

The main limitation of my program are the exclusion of external factors. The SIR model that my program uses does not account for external factors within the population such as government policies etc. social distancing or lockdowns. It also does not consider the role that healthcare infrastructure takes in managing a pandemic. For example, it overlooks the number of hospitals which can treat infected patients, how many health supplies such as medicine are available for treating the disease and the effectiveness of hospital treatments. Not including these factors leads to a simulation that does produce a simplified version of how a disease moves through a population which limits how accurate the result is. The reason I'm not implementing this is because my program focuses more on the 'Transmission Dynamics' of the disease which the SIR model focuses on therefore external factors such as government policies and healthcare are not needed.

Another limitation of my solution is the behaviour of the agents within the population. My program assumes that the characteristics about the agents remain constant throughout the simulation for example the transmission rate and number of interactions between the agents are set at the start of the simulation and remain constant throughout. However, in a real-life scenario these behaviours would change as the disease moves through the population. For example, in a real-life scenario people would reduce their interactions with other people if they knew a disease was spreading through their area, whereas in my program it's assumed that the number of interactions stays the same. This could lead to an inaccurate simulation which may produce a completely different output than what would happen in real life. The reason I'm not implementing this is it would require more computing power to run the simulation from the user's computer and since my program is going to be used by students, they may not have access to hardware advanced enough to cope with the program load.

Requirements

Software

Windows, Mac or Linux OS – An operating system that supports SQLite and python with its subsequent modules

Python Interpreter – This program will be written in python; therefore, an interpreter is needed to interpret and execute the code.

SQLite – To create a database and perform numerous operations to store, retrieve and modify data.

Modules:

- Tkinter – *To create the interactable GUI of the program, allowing the user to interact with the different pages as well as the model itself*
- Mesa – *To create the agent-based model of how the population and disease interact*

Hardware

A Standard Computer able to Execute the Program – The software will need a computer able to execute it with a processor capable of interpreting and executing python code quickly. Most modern laptops should be suitable for this.

A Keyboard and Mouse – These are used to navigate the graphical user interface.

Success Criteria

Success Criteria	Reason	Evidence
Main Menu	To show the different sections to the user	Screenshot of the Main Menu
Buttons: Full Simulation Tutorial Pre-sets	To allow the user to navigate and use the different functionalities of the program	Screenshots of the Buttons
Quit Button	To allows the user to exit the program	Screenshot of the 'Quit' button
Full Simulation Section	Allow the user to run a simulation of how a disease moves through a population	Screenshots of the section running, screenshots demonstrating the functionality of the section
2D Grid Graphic	Presents the data through a grid which visually demonstrates how the disease moves through the population using squares to represent the population	Screenshot of the grid during a simulation
Titled List of Pre-sets Templates	Allows the user to select and load pre-set values into the simulation	Screenshot of the list, screenshots of how the parameter values change after pre-set is loaded in
Play Button	Allows the user to start the simulation in real-time	Screenshot of Button
Pause Button	Allows the user to pause the simulation	Screenshot of Button
Forward Arrow Button	Allows the user to iterate the simulation by one time-step forward	Screenshot of Button
Backward Arrow Button	Allows the user to iterate the simulation by one time-step backward	Screenshot of Button
Parameter Titles	Allows the user to see all of the parameters that can be changed for the simulation	Screenshot of the Parameter Titles

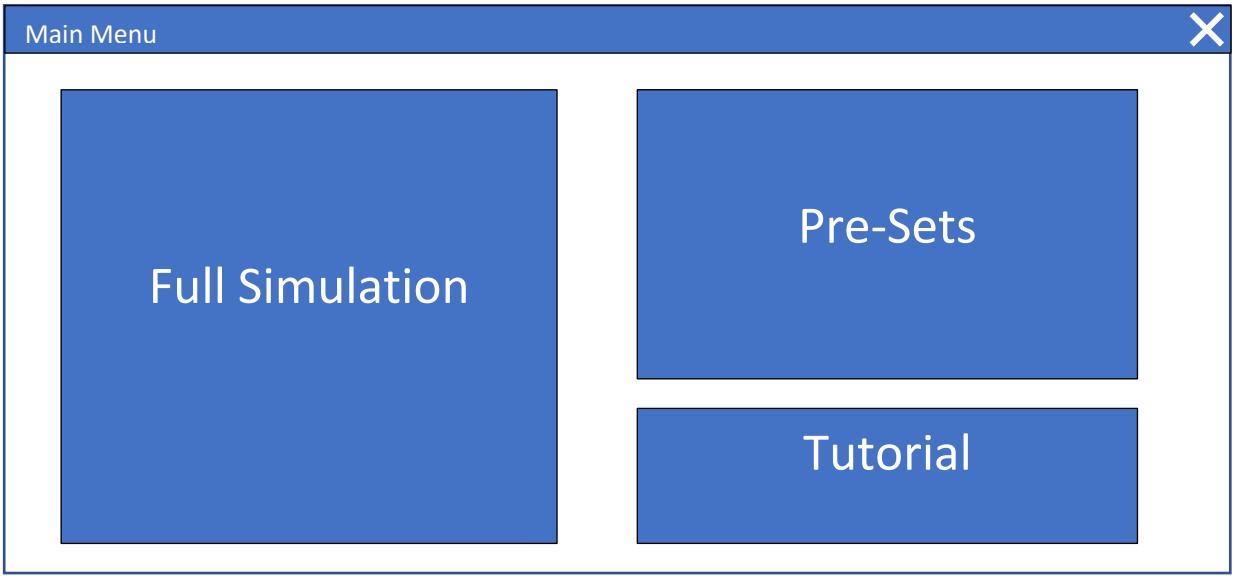
Parameter Input Boxes	Allows the user to change the values of different parameters in the simulation	Screenshot of Parameter Boxes
Exit Button	Allows the user to exit the section and return to the main menu	Screenshot of the Exit Button
Tutorial Section	Allows the user to see how the simulation functions and the information about each parameter	Screenshots of the Tutorial Section
A paragraph explaining the function of the program	Allows the user to learn what the function of the program is	Screenshot of the Paragraph
Diagram of 'Full Simulation' Section with each button labelled	Allows the user to see how the 'Full Simulation' section works as well as the function of each button	Screenshot of the Diagram and labels
Parameter Titles with accompanying paragraphs	Allows the user to learn the information on each parameter and the relevance each one has to the simulation	Screenshot of the titles and paragraphs
Diagram of 'Pre-sets' section with each button labelled	Allows the user to see information about the 'Pre-sets' section as well as the function of each button	Screenshot of the Diagram and labels
Next Page Button	Will move to the next page in the tutorial section	Screenshot of the Forward Button
Previous Page Button	Will move to the previous page in the tutorial section	Screenshot of the Backward Button
Exit Button	Allows the user to return to the main menu	Screenshot of the Exit button
Pre-sets Section	Allows the user to manage, add and remove parameter pre-sets	Screenshots of the section, screenshots of the functionality of the program
List of Pre-sets stored in a database	Allows the user to click on different pre-sets and see available ones in the database	Screenshots of the list, screenshots of named pre-sets in the list, screenshot of the database
Editable Pre-sets	Allows the user to click on different pre-sets and edit their properties	Screenshots of the edit function, screenshots of values before and after
Add Button	Allows the user to add a pre-set of values to the database	Screenshot of the Button, screenshots of the button's functionality, screenshots of database before and after a pre-set is added
Remove Button	Allows the user to remove a pre-set of values from the database	Screenshot of the Button, screenshots of the button's functionality, screenshots of database before and after a pre-set is removed

Exit Button	Allows the user to return to the main menu	Screenshot of the Exit button

Design

User Interface Design

Main Menu

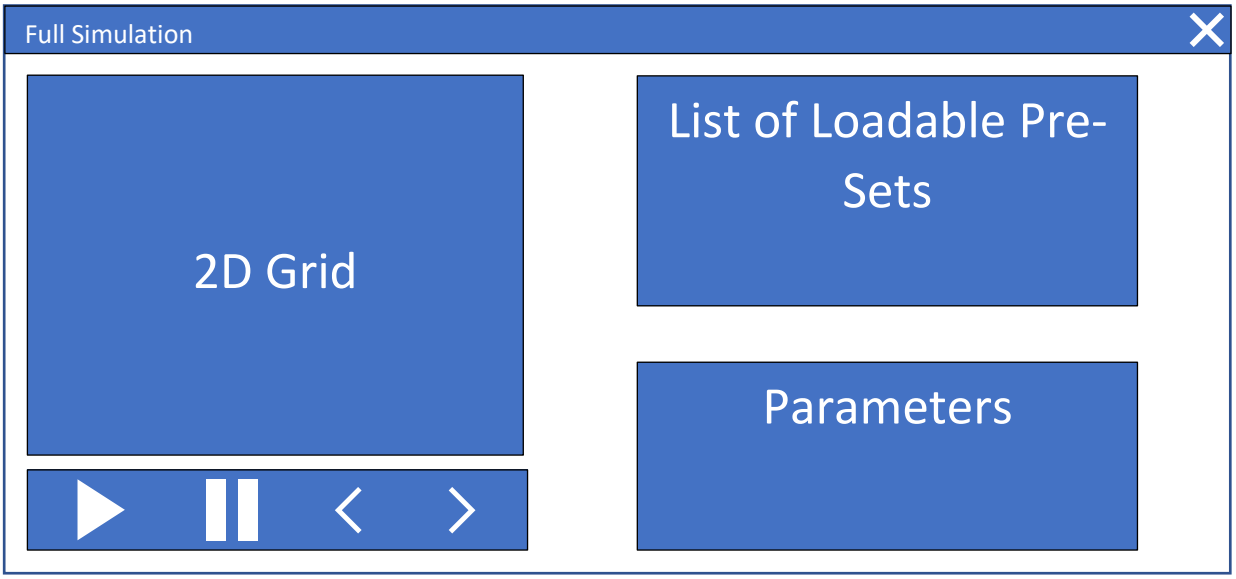


This is the main menu of the program and includes button that link to the 'Full Simulation' page, 'Pre-Sets' page and 'Tutorial' page. It's a simple design and has large buttons for ease of use by the user.

Links to Success Criteria:

- Buttons: Full Simulation, Tutorial, Pre-sets
- Quit Button

Full Simulation Page



This is the main page of the program where the simulation will be run and adjusted by the user.

Links to Success Criteria

- Exit Button

2D Grid

This 2D grid box will contain the real-time simulation presenting the data of how the disease spreads through a population as a grid of squares, changing colour respective to what category they are in the SIR model. This will take up most of the page since it's the main function of the program as well as causing the user to focus the majority of their attention onto it.

Links to Success Criteria:

- 2D Grid Graphic
- SIR Model

Control Buttons

This box contains four control buttons for the simulation. They are placed right below the 2D grid since they directly control the graphical output of the simulation by starting and stopping it etc., this placement helps the user make the association that they are closely related and linked to the function of the 2D grid. For the start and stop buttons I've used recognisable symbols that are near universally known, this means the user knows their function just by looking at them. I've chosen to use two arrow keys for the forward and backward time-step functions, since like the stop and start icons, they are recognisable and demonstrate their function the user easily

Links to Success Criteria

- Play Button
- Pause Button
- Forward Arrow Button
- Backward Arrow Button

List of Loadable Pre-sets

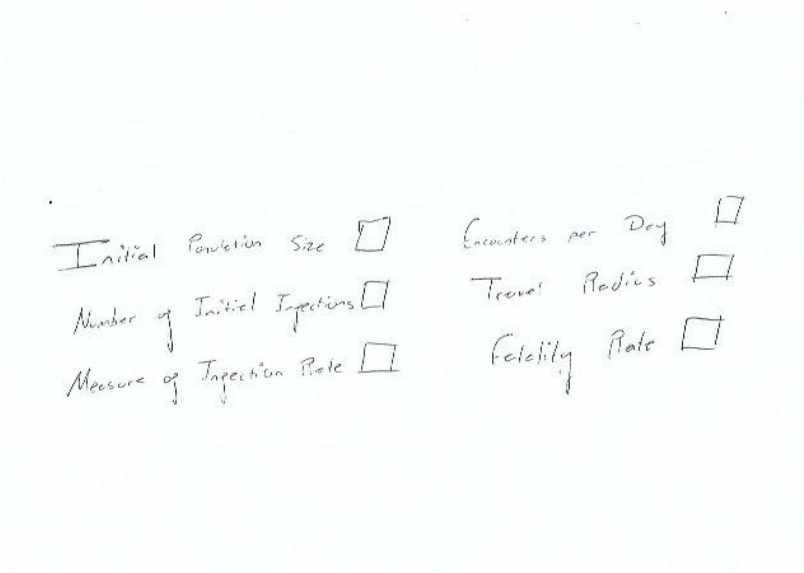
This box will contain a list of named pre-sets that can be loaded into the simulation by the user. The box itself will contain a scroll bar, allowing the user to scroll through all the pre-set values they've created. It's just above the parameters for the program, so that when the user loads a pre-set in they can see how the values of the parameters change to the values specified in the pre-set.

Links to Success Criteria

- Titled List of Pre-set Templates

Parameters

This box will contain all the adjustable parameters for the simulation. Next to each parameter title will be an input box that the user can enter a value into. The interface for this box will look something like this:

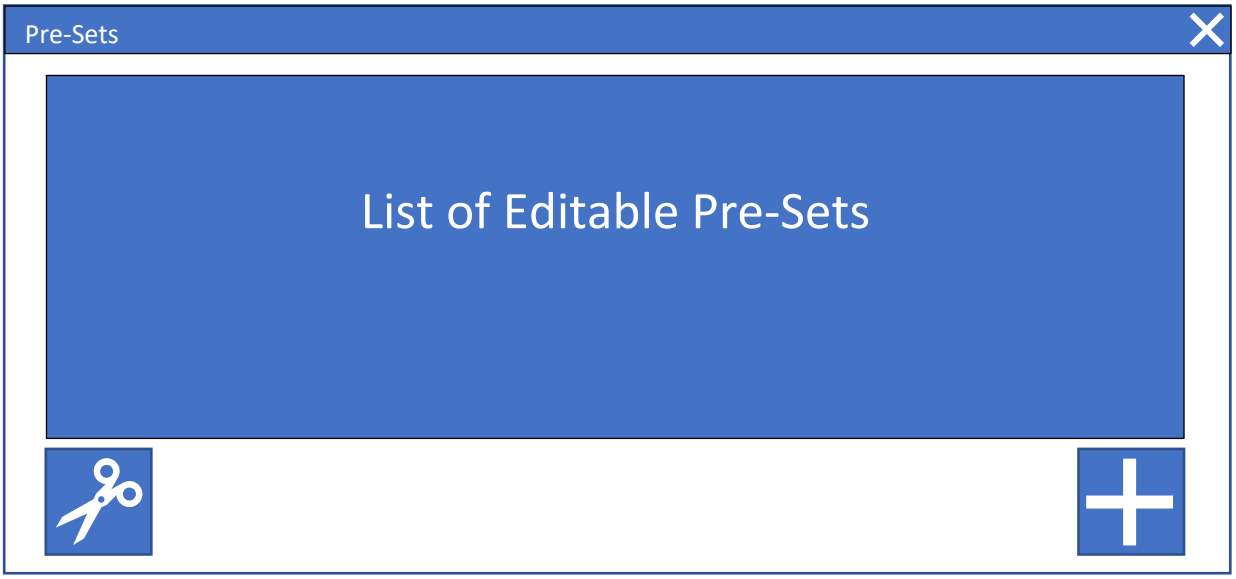


These parameter titles may change as the program develops but the alignment of having the title of the parameter then an input box for the number next to it in some way will be how the parameters are laid out in the page. This means that the user can instantly recognise which label relates to which parameter box and so knows exactly where to input each number to change the parameter they want.

Links to Success Criteria:

- Parameter Titles
- Parameter Input Boxes

List of Pre-sets Page

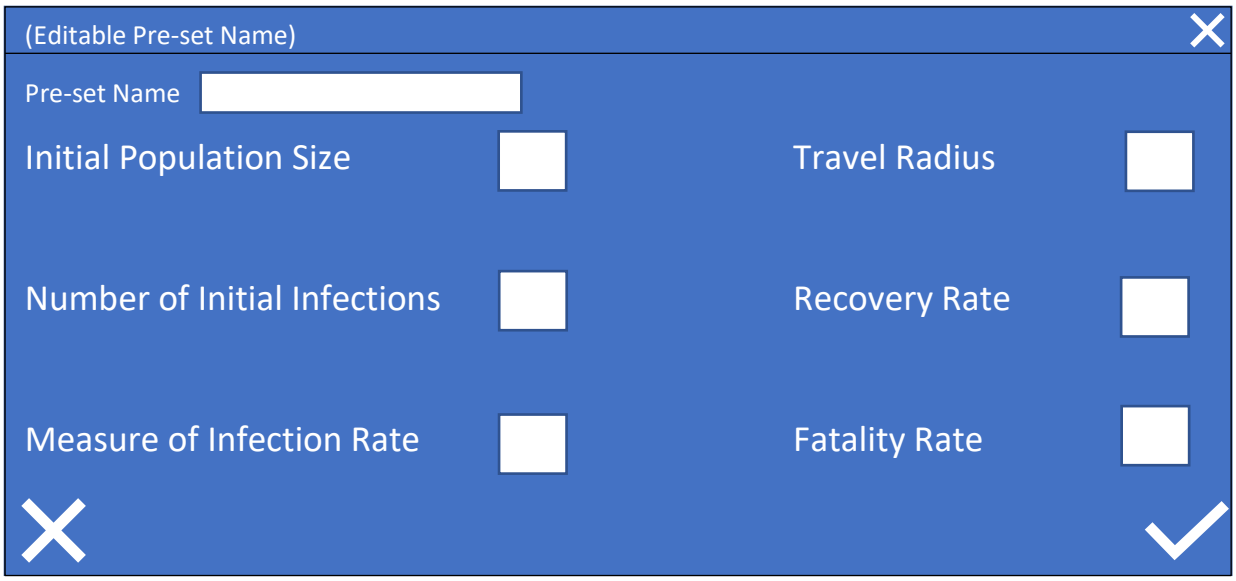


The focus of this page is the list of pre-sets which will take up most of the page. The user will be able to scroll through the different pre-sets and click on the ones they want to edit. At the bottom of this list are the 'Remove' and 'Add' buttons. I chose a scissor as an icon for the remove button since the function of scissors is to 'cut' so the user knows that the button is used to delete pre-sets from the list/database. When the cut-button is pressed, the user will be able to click on a pre-set within the list to remove it from the database. When the add button is pressed the user will be taken to a separate page, to enter parameters for a new pre-set as well as name. This pre-set will then be added to the database and list of editable pre-sets.

Links to Success Criteria:

- List of Pre-Sets stored in a database
- Add Button
- Remove Button
- Exit Button

Editable Pre-set Page

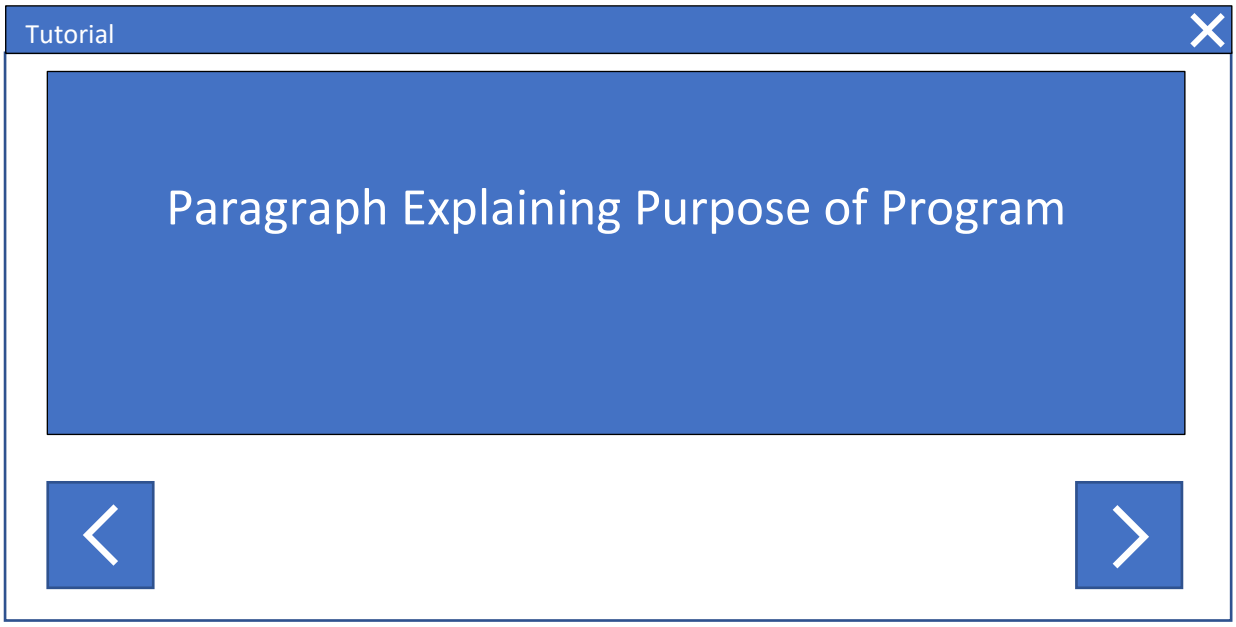


This is the page that the user will be taken to when they want to edit a pre-set stored in the database. The different parameters are stated with an input box next to each one. These input boxes will be filled with the stored values and the user will have the option to click on the box and replace them with new values. There are two buttons on the bottom of the page. The cross button will close the page and return the user to the 'Pre-sets' page without saving the changes the user has made. The check button will save the changes has made and return the user to the 'Pre-sets' page. I chose the cross and check symbols for these two buttons since they demonstrate the functionality of the button easily to the user. This page will also be used if the user wants to create a new pre-set by using the 'Add Pre-set' button. It allows them to enter the parameters they want saved to the pre-set. The title of the pre-set at the top of the page will also be editable whether they want to add or edit a pre-set.

Links to Success Criteria

- Editable Pre-sets

Tutorial Page



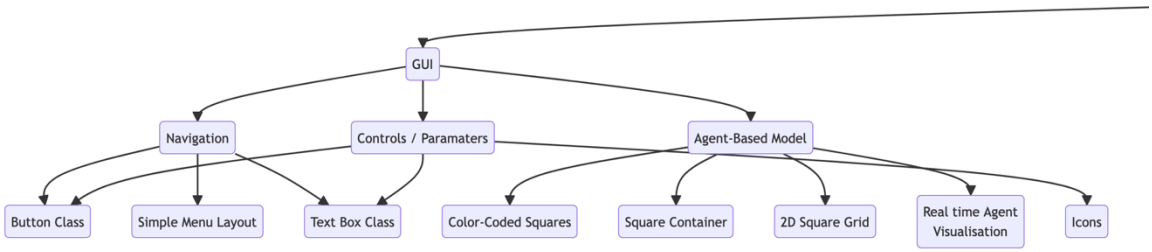
This page will be used if the user needs any advice or help on how to operate the program. It's mainly targeted for Group 1 who may be struggling with this topic in the curriculum. This page will have a main paragraph explaining the purpose of the program as well as two buttons at the bottom which can navigate to other tutorial pages which will contain images of diagrams that show the functions of buttons in the 'Simulation' page as well as the 'Pre-sets' page.

Decomposition

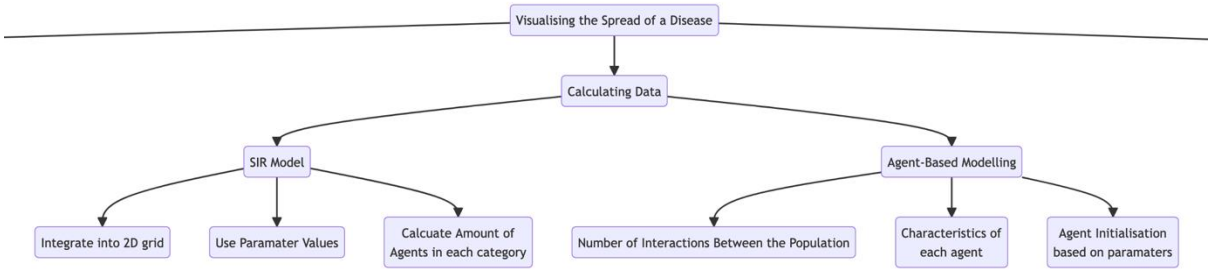
I've decomposed this problem of visualising the spread of a disease into three different problems. GUI, Calculating Data and the SQL Database. These problems are further decomposed into separate problems that when each solved separately will make up the program.

The first problem that I came across was creating the GUI for the program and how I would implement the different components in the program in a graphical way that the user can intuitively understand. To do this I split the problem into three components of what the user interacts with within the GUI. For the navigation of the program using the template pages from the user interface design, much of the navigation consists of buttons as well as text in the form of titles and paragraphs relaying information to the user. To implement this effectively, I've decided to create button and text classes, so these can be repeated across the GUI without having to repeatedly write them.

For the controls / parameters component of the GUI it is also able to use the text class for naming parameters, the button class for the control functions of the program such as exit, save a pre-set, etc. However, with the addition of an icons class. These are needed to tell the user the function of each control function and since, for example, the exit function is repeated in nearly every page of the program it's beneficial to create a class so it can be easily replicated across the program.



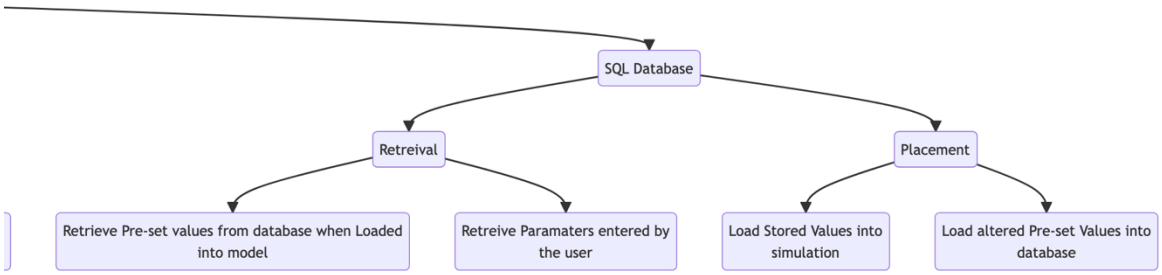
Finally, the final part of the GUI is representing the Agent-Based model itself. I've broken this problem down into separate components that will effectively represent the data structure. Firstly, I needed a way to represent the population to the user. I've decided to do this in a 2D grid meaning the GUI consists of a square container with a grid of squares within to with each square representing a member of the population. I also needed to represent the different categories of the model such as infected, recovered etc in a way so the different categories can be distinguished by the user. This can be done through different colours thus the color-coded squares.



The next problem I needed to solve was a way to effectively calculate the data needed to run the simulation in the program. All of the data calculated in the program can be split into two categories, the SIR model which is used to calculate the way the disease spreads through the population when members of the population interact with each other. The second category is the agent-based model which is used to calculate the amount of interactions that the members of the population have with each other.

I split the problem of implementing the SIR model into three components. The first is implementing the function of the SIR model which is calculating the number of agents in each SIR category within the program. The second is integrating the ability to use different values for different parts of the SIR model based on what the user has inputted. An easy way to do this would be to implement classes within the model, meaning different values can be used without having to re-write code. The final problem is to integrate the model into the 2D grid so it can be used simultaneously with the agent-based model. This is probably the most difficult problem to solve and will be the most time consuming.

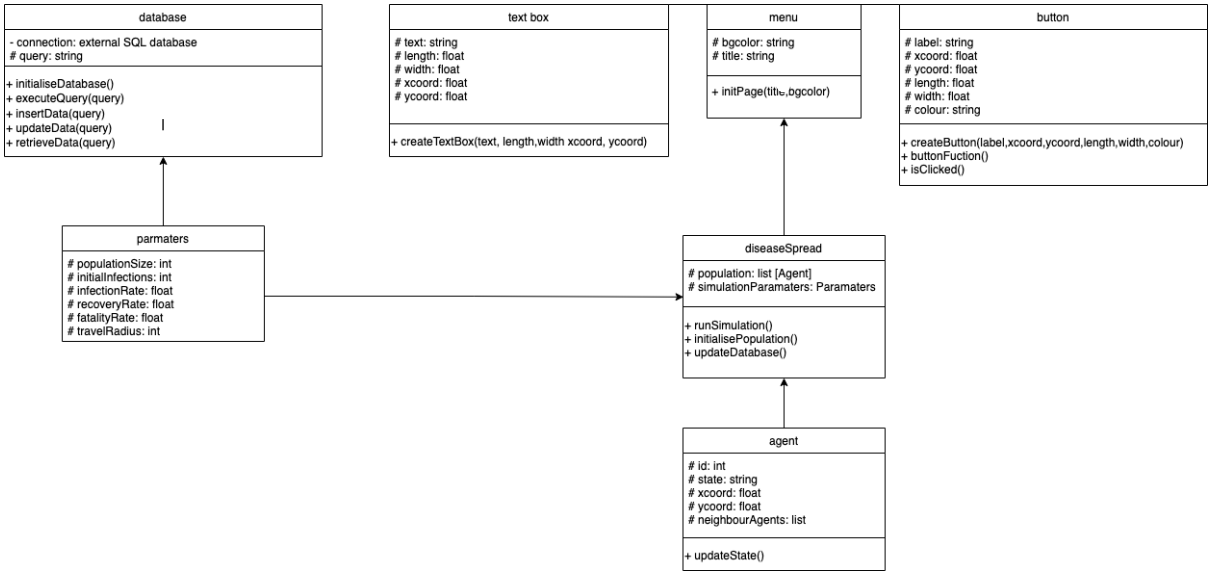
Moving on to implementing the Agent-Based modelling problem, I've once again split this problem into three components. The first is finding a way to calculate the number of interactions between the population, this can be done maybe through a random variable, seeing what the chance is that members interact with each other. Secondly, I need to find a way to define the characteristics of each agent. Once again classes can be used since I would be able to input different characteristics while each agent would still have the same template. This can also be used for agent initialisation, since the parameters defined by the user can be inputted into the class, initialising different agents based on the pre-defined parameters.



Finally the last problem to solve is how to implement the SQL database. I broke down this problem into two main components, retrieving data and placing data from and to the database. These functions of the database can be coded into my program using the sqlite3 module for python.

Object Orientated Design

Here is a class diagram for the main classes that will be used in my program:



The menu class will be called each time I need to create a new page for every section I need to implement within the program; it only has two attributes which are the background colour and title, which will be used to show the user the page that they're on. It includes a method for creating each page in the program. I also need button and text box classes since on every page there will be text labels explaining to the user where each button navigates and what the button does, there will also be buttons on every page since they make up the foundation of the navigation and functionality of the program since they are what the user interacts with to interact with the program itself. Thus, instead of having to re-write code every time a new page is created with buttons and text, I can use the menu and button classes respectively.

The disease spread class will be the class that is involved with calculating the disease spread and running the simulation. The class itself inherits data from two different classes: parameters and agent. This is because the data needed to run the simulation comes from these two classes. The

population attribute uses a list of agents to act as the population for the simulation in which the disease spreads through. These agents will each have a state, susceptible, infected, or recovered assigned to them which can be altered as the program runs. For the simulationParamaters attribute the parameters class is used since it contains all the individual parameters that alter the outcome of the simulation.

These parameters will be stored in the database in the form of pre-sets. For the database class this class will be used to manage the external SQL database in which data from the program is stored. This is shown by the connection to the external SQL database specified in the diagram. The methods in the class are used to edit the data within the database with the only attribute being the SQL query that needs to be carried out. The data stored in this database will be the multiple instances of the class parameters (which will be used to store and edit pre-sets of the simulation). Therefore, it has an association with the parameters class however not an inheritance.

Database Design

The external relational SQL database will contain two tables, the 'parameters' table which will store the set of parameters that the user saves. It will also contain the 'pre-sets' table which will be used to store the sets of parameters while associating them to specific named pre-sets. The reason I have created two tables in the database is it improves the readability of the database since pre-sets are named and normalises the database since it means there are no repeating attributes, partial dependencies or non-key dependencies. This means the database is normalised, making it easier to query as well as massively improving the readability of the database.

Parameters Table

This table stores sets of values of the parameters. Each set will have a unique id that acts as a primary key in the 'Parameters' table. Each attribute in this table is a parameter that will be used in the simulation. I've included a few in the diagram in the diagram below as an example, however the actual table will have all the parameters such as 'interactions between agents' and 'fatality rate'.

Paramater_id	**infection_rate**	**recovery_rate**	**initial_population**	**etc..**
1	0.87	1.00	50	
2	0.55	0.4	60	
3	0.32	0.37	50	

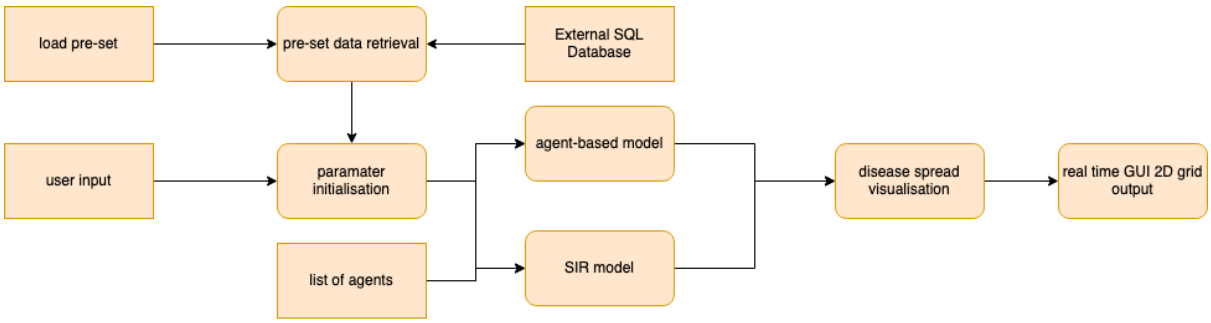
Pre-sets Table

This table will store the numerous pre-sets that the user creates, assigning a unique id to each one. This table will also be linked to the 'parameters' table through the 'paramater_id' field which acts as a foreign key linking the two tables together. This table will also take the name specified by the user and store it in the 'pre-sets name' field as a secondary key to be used as an index for locating a specific pre-set.

Pre-sets_id	**pre-sets_name**	***Parameter_id****
1	Simulation1	4
2	100% Fatality	1
3	Small Population	9

Data Flow Diagram

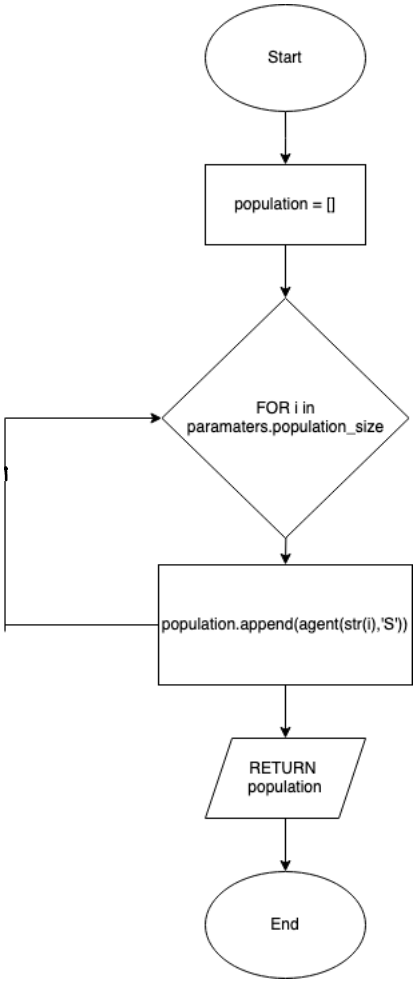
This is a diagram of how the data will move within the real-time disease visualisation model:



The user can either input the parameters or choose to load a pre-set, which will then be retrieved from the database. These parameters will then be initialised into both the SIR model, which calculates the how many members of the population are Susceptible, Infected or Recovered and the agent-based model, which is the model which calculates how the agents / population members interact with each other. Both models will be able to access the list of agents to change to be able to change their respective disease category and their interactions with each other. These models can then use their respective calculations to produce the disease spread visualisation which is then output by the GUI.

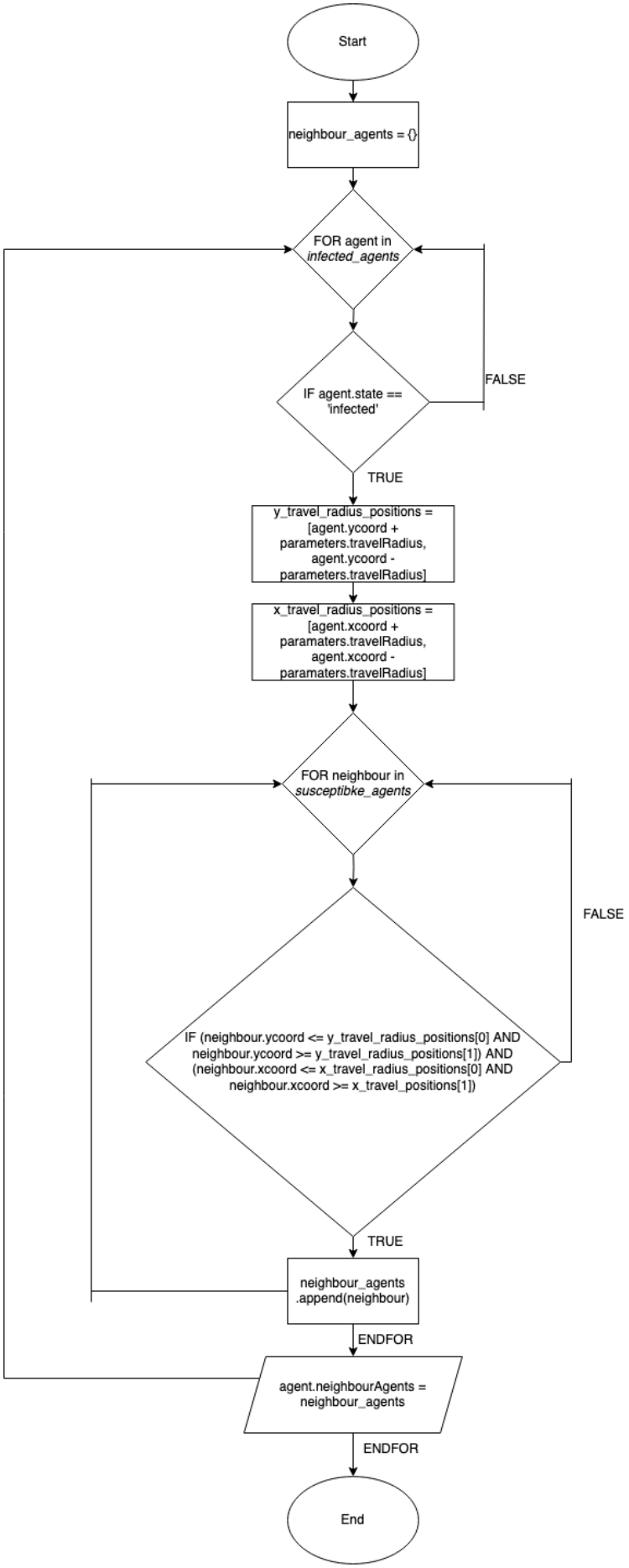
Algorithms

Agent Initialization Algorithm



This algorithm will be used to generate the list of agents that act as the population that the simulation will use. It uses the populationSize attribute of the Parameters class as the defined number of agents that will be added to the list. It then loops that specified number, adding an agent every time. Each agent starts off in the 'Susceptible' state since they haven't been infected as the simulation hasn't run and this is just the initialisation. The algorithm then returns the list. The purpose of this algorithm is to create the population of people that the disease will infect and spread throughout.

Neighbouring Agents within Travel Radius Algorithm


Start
neighbour_agents = {}
FOR agent in infected_agents
IF agent.state == 'infected'
FALSE
TRUE
y_travel_radius_positions = [agent.ycoord + parameters.travelRadius, agent.ycoord - parameters.travelRadius]
x_travel_radius_positions = [agent.xcoord + paramaters.travelRadius, agent.xcoord - paramaters.travelRadius]
FOR neighbour in susceptibke_agents
IF (neighbour.ycoord <= y_travel_radius_positions[0] AND neighbour.ycoord >= y_travel_radius_positions[1]) AND (neighbour.xcoord <= x_travel_radius_positions[0] AND neighbour.xcoord >= x_travel_positions[1])
FALSE
TRUE
neighbour_agents .append(neighbour)
ENDFOR
agent.neighbourAgents = neighbour_agents
ENDFOR
End

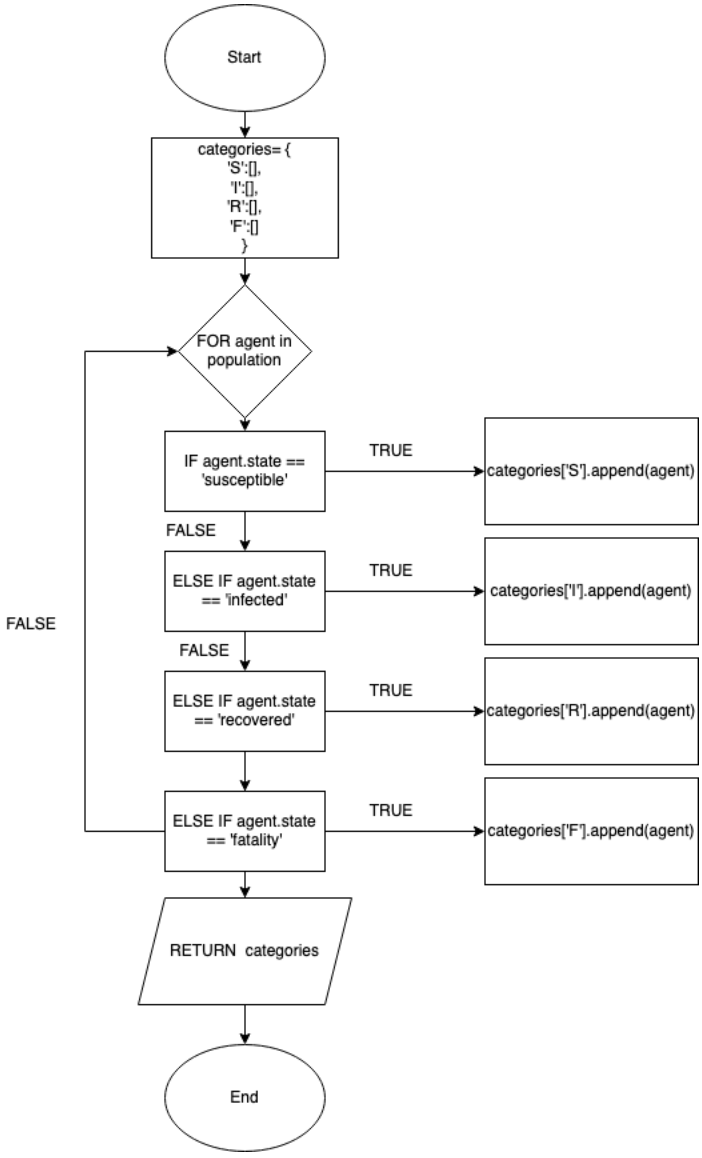
This will be the algorithm used to gather and return the neighbouring agents within the disease travel radius of all infected agents in the population. Firstly, it initialises an empty dictionary in which each infected agent will be placed with their corresponding neighbouring agents. Then it initialises the empty list that these neighbouring agents will be stored in it then iterates through the population list to check for infected agents in the total agent population.

If an infected agent is found it then it uses the pre-defined parameter ‘travel radius' as the radius around that agent for which other agents can be considered, ‘neighbouring agents’. It then creates two lists, one with the maximum and minimum y coordinates based on the travel radius and then another that’s identical however with x coordinates. Another for loop is then used to iterate through the population to check for neighbouring agents. If the current agent selected from the population has coordinates that are higher or equal to the minimum x and y coordinates as well as lower than or equal to the maximum x and y coordinates, they can be considered a neighbouring agent within the travel radius and are added to the neighbouring agents list. Once all neighbours are added to the neighbouring agents list, the currently selected infected agent’s id is used as a key in the ‘total_neighbours’ dictionary, with the corresponding neighbour list being added to the dictionary.

This is a basic representation of the algorithm and there are some issues such as nested for loops which take a lot of processing power and is not efficient. However, in the final solution a list of infected agents per time-step will be created which this algorithm can iterate through instead of the entire population. This eliminates the need for nested for loops, increasing the efficiency of the program.

The two parameters that this function uses, ‘categories[‘I’]’ and ‘categories[‘S’]’ have been named *infected agents* and *susceptible agents* within the class which relates to their purpose within this class.

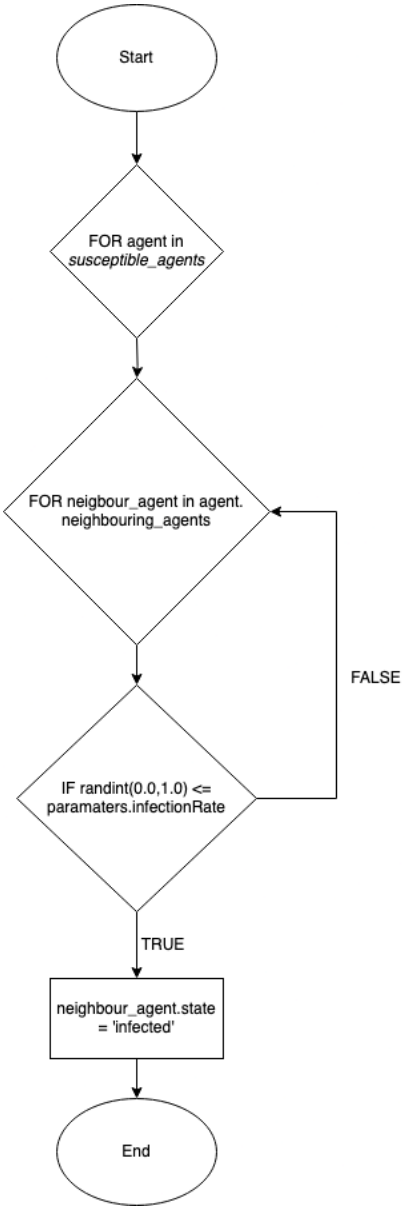
SIR Categorisation Algorithm



This algorithm categories each agent into a different category in the SIR model, at each time-step in the simulation based on their state attribute. It creates a dictionary called ‘categories’ with an index of three letters which represent each SIR category. Each category has a list where multiple agent objects can be stored in. This means the simulation can access the category to track each agent in each category, as well as keeping track of the amount in each category. This is useful since it eliminates the need for iterating throughout the entire population when an agent from a particular subset category such as ‘infected’ needs to be found. It reduces the processing power needed leading to a more efficient program.

This function accepts one parameter which is the ‘agentInitalisation()’ function. Within the class this is called ‘population’ and is used by the function to iterate through every possible agent used in the simulation.

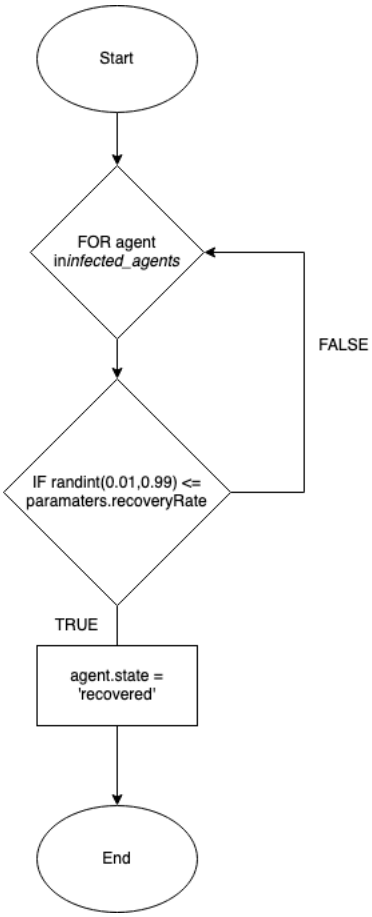
SIR Calculation Algorithm



This algorithm is used to calcuate the chance of a neighbouring agent being infected by the disease. It uses the neighbouring agents lists assigned to each infected agent as the subset of the population that can be infected in the new time-step. It uses a random integer to determine the chacne that an agent will be infected as in real-life the chance that a disease infects someone is also down to chance. I've made it so if the random integer is smaller than the infection rate then it is a successful infection and the neighbouring agent becomes infected. This also means that the higher the infection rate the higher the chance that the random integer will be smaller, therefore the higher chance that the neighbouring agent will become infected. However, to begin with the algorithm needs to iterate through the entire population to determine which agents are infected, this is inefficent since looping through the entire population is resource-intensive. A more efficent solution will be implemented during development, and this serves as a good basis. This more efficent solution uses 'SIR Categorisation' and iteraters through the subset of that population, howver this has not been included in this algorithm since it was discovered later in the development process.

The parameter that this function uses, categories['S']' has been named '*susceptible agents'* within the algorithm.

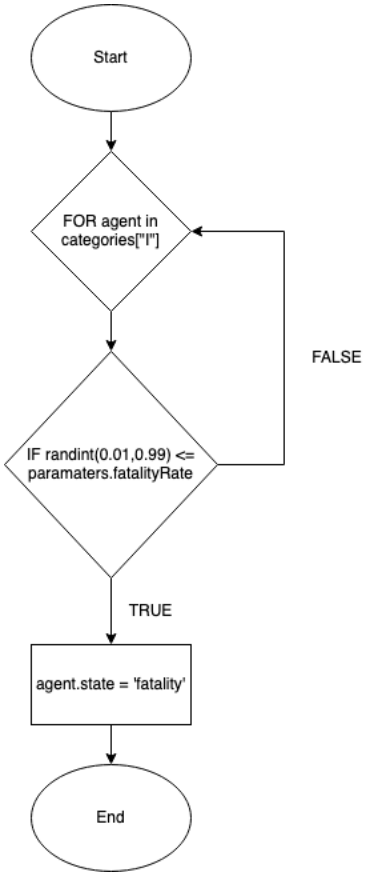
Chance of Recovery Algorithm



This algorithm will iterate through the infected population at the end of each time-step. It generates a random number and compares whether it is smaller than the recovery rate. This means the higher the recover rate the the user hass inputted, the more chance that the infected members of the population change to a recovered state rather than staying as infected. It is also more effective than the 'SIR Calcuation' algorithm, since it makes use of the 'SIR Categorisaiton' algorithm, so instead of iterating through the entire population at the start of the algorithm; it only iterates through the infected subset of the population. This new idead has not been implemented into the 'SIR' calcuation aglirhtm flowchart, however in the actual development of the program it will not loop through the entire population, only the subset declated by the 'SIR Categorisation' algorithm.

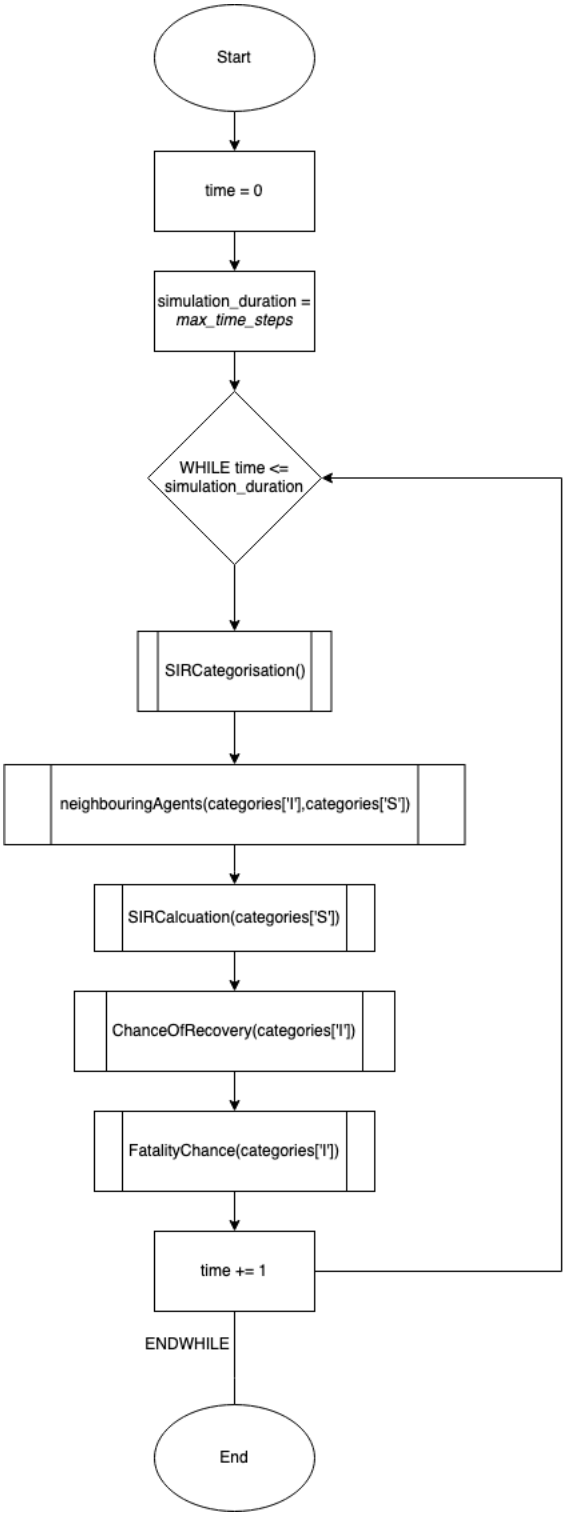
The parameter that this function uses, categories['I']' has been named 'infected agents' within the algorithm.

Fatality Chance Algorithm



This algorithm is identical to the previous, however it determines whether a member of the population that is infected will die. If they die this means they will not be part of the population and unable to be included in the simulation.

Simulation Loop Algorithm


Start
time = 0
simulation_duration = max_time_steps
WHILE time <= simulation_duration
SIRCategorisation()
neighbouringAgents(categories['I'],categories['S'])
SIRCalcuation(categories['S'])
ChanceOfRecovery(categories['I'])
FatalityChance(categories['I'])
time += 1
ENDWHILE
End

This is the algorithm that will be ran each time the simulation increments by one time-step. Firstly, it initialises the list of the simulation population using the agent initialisation algorithm, it then selects one random agent from that respective list to become infected, making the agent 'patient zero' and starting the spread of the disease. It then uses the 'neighbouringAgents' algorithm to calculate the neighbouring agents of every infected agent within the simulation. After that, it calls the 'sirCalcuation' algorithm which will calculate and store the agents in each disease category within a dictionary called 'sirCategories'. It then increments the time by one. Once the time meets or is lower than the 'maximum_time_steps' (which will be inputted by the user), the algorithm ends and the simulation will stop running

Key Inputs, Outputs, Data Structures and Variables

Inputs and Outputs:

Inputs	Process	Intended Output	Actual Output
User clicks on Full Simulation Button	Initialises the 'menu' class which creates the 'Full Simulation' page	'Full Simulation' page is displayed with text boxes and icons	
User clicks on 'Pre-sets' button	Initialises the 'menu' class which creates the 'Pre-sets' page	'Pre-sets' page will be displayed with list of loadable pre-sets and icons	
User clicks on 'Tutorial' button	Initialises the 'menu' class which creates the 'Tutorial' page	'Tutorial' page will be displayed with a central text box	
Parameters are inputted by user	Parameters class will be called, with attributes becoming values that the user has inputted	Parameters class will be used by the simulation loop to define simulation characteristics	
User clicks on a Loadable Pre-Set	User will click a pre-set title in a list of them to load a set of saved parameters into the simulation	Pre-set values will be loaded from the database. Attributes of the class 'Parameters' will be changed to the pre-set values	
User clicks on Play / Pause Button	Will pause at the current time-step of the simulation, will resume from the current time-step of the simulation	Simulation will resume from its current time-step, or it will pause at its current time-step	
User clicks on Forward or Back Buttons	Will increment the simulation forward by one time-step, or will move the simulation back one time-step	Simulation will move forward by one time-step or move backwards by one time-step	
User clicks on Cut Pre-set Button, then clicks on a Pre-set	Will remove the selected pre-set from the database	The selected pre-set will be removed from the 'Editable pre-sets list' and database	

User clicks on Add Pre-set button	Initialises the 'menu' class which will creates the 'Editable Pre-set' page	'Editable Pre-set Page' will be displayed with parameter input boxes	
User clicks on Exit Button in Editable Pre-set Page	Will initialise the 'menu' class to return to the 'Pre-sets' page	'Editable Pre-set Page' will close, and 'Pre-sets' page will be displayed	
User clicks on Save Button in Editable Pre-set Page	Will initialise the 'menu' class to return to the 'Pre-sets' page, adds new pre-set to database	'Editable Pre-set Page' will close, and 'Pre-sets' page will be displayed, entered pre-set values will be added to the database	

Data Structures

Data Structure	Purpose	Reasoning
List	To store all agent objects in the population as the simulation runs, to store neighbouring agents around an infected agent	It means other functions can iterate through the population, tracking their state and attributes
Dictionary	To store lists of agents, corresponding with their state (SIR)	It means that functions in the simulation can easily access the agents in each SIR category, without iterating through the whole population

Key Variables

Variable Name	Purpose	Reasoning
time	Used in a for loop to keep track of the time-steps that have been executed	While this variable is lower than the max-duration of the simulation, the simulation loop will run. Once it's equal to the max-duration of the simulation the loop will end.

User Requirements

Here are some features that will make the program more accessible to users:

Feature	Justification
Labelled Buttons	Allows user to easily navigate the program / pages
Labelled Boxes	Allows user to easily know how each input box affects the outcome of the program
Main Menu	Allows user to navigate the whole program from a single page, simplifies layout
Large Buttons	Makes the GUI more intuitive, user can easily see different functions / pages of program

Tutorial Page	Explains different functions of program to the user, increases understanding of the program
SIR Colours (within grid)	Allows the user to see the state of a square at a glance with only three different colours
SIR Colour Key	This means that the user will be able to know which colour corresponds to which state that an agent is in
Squares (within grid)	Simplifies the population of the simulation to graphical shapes which can be understood by the user
Pre-sets	Allows the user to load parameters into the simulation without having to manually input them every time the simulation is run

Validation

To make sure my program will function properly, this will outline all possible types of inputs that the user can input as well as how they will be validated.

Input Boxes

These will be the boxes used to input parameters for the program. These will be constrained by the type of data that can be inputted as well as the range of values that can be inputted. In this case, different input boxes will accept different data types, for example the starting population parameter will have to be an integer, since there can't be a fraction of an agent. However the infection rate could definitely be a float since it allows for a greater variety of simulations to be run with a greater variety of infection rates. In all cases though, the data type cannot be a string. The validation method I will use to confirm this will be the in-built python function type(), if it returns the data type expected to be in that box then the simulation can run.

The range of values that can be inputted into each input box will also be based on each input box individually. For example, the starting population input box will have to have a maximum allowed integer, otherwise the simulation will use too much processing power or take too long. The starting population can also not be allowed to be zero, since otherwise the simulation cannot run. The validation method to confirm this will be selection, where the condition specified will be to make sure the input isn't larger or smaller than the specified bounds of values.

In total, the number inputted into the input boxes will be checked that it's within the specified range and right data type. If one of these conditions is incorrect, a pop-up will appear stating that these values are invalid and will ask the user to input these values again.

Buttons

There is no need for validation methods since the only two actions that the user can use to interact with the buttons is to click or not click and neither are invalid inputs from the user.

Test Plan

Iterative Testing (White-Box)

Iteration 1: Menu, Boxes and Buttons:

This iterative test will test the functionality of aspects of the GUI, including menus, boxes and buttons. I need to check boundary data for the range of values that can be inputted, erroneous data should also be checked to make sure the input is the right data type. Invalid data should also be checked since it may bass the boundary and erroneous data checks but may be unreasonable, for example a starting population too large will use too many computing resources. A message should also be displayed to the user, asking them to enter valid data if the entered data is erroneous / illogical. The test plan is outlined below:

Buttons

1. Note what the button's intended function is
2. Confirm the button does this function when it is clicked

Below is a table containing each button within the program as well as it's respective functionality test:

Button Name	Intended Functionality	Functionality when User Clicks
Exit	Will close program when clicked	
Full Simulation	Will take the user to the 'Full Simulation' page when clicked	
Tutorial	Will take the user to the 'Tutorial' page when clicked	
Pre-sets	Will take the user to the 'pre-sets' page when clicked	
Play Button	Will start the simulation when clicked	
Pause Button	Will pause the simulation when clicked	
Add Button	Will add pre-set values to database when clicked	
Remove Button	Will remove selected pre-set values when clicked	
Next Time-step Button	Will move to the next time-step when clicked	
Previous Time-step Button	Will move to the previous time-step when clicked	

Input Boxes

1. Confirm that the box is registering user inputs
2. Confirm that the inputs are valid
3. Confirm that the input box allows for errors and asks the user for a valid input
4. Confirm that the button produces the intended output

There are six input-boxes in my program, each with their respective, valid data type and range of values that can be accepted. For all of these I have created an individua test plan for each input box.

Each individual test plan contains a list of input values, as well as stating the validation check that it's testing for. It also records the output message if the data entered by the user does not pass the respective validation. Once testing has been carried out the actual output of each input box will be recorded:

Maximum Number of Time-Steps

Input	Validation Type	Intended Output	Actual Output
3,200	Normal	Simulation will run for 3,200 time-steps	
0	Invalid	User will be asked to enter a valid value within specified range of 1 – 10,000	
10,001	Invalid	User will be asked to enter a valid value within specified range of 1 – 10,000	
1	Boundary	Simulation will run for one time-step	
10,000	Boundary	Simulation will run for ten thousand time-steps	
'1'	Erroneous	Program will not allow user to input quotation marks	
1.5	Erroneous	Program will not allow the user to input a decimal point	
''	Erroneous	User will be asked to enter a value	

Number of Initial Infections

Input	Validation Type	Intended Output	Actual Output
3,200	Normal	Simulation will start with 18,720 infected agents	
0	Invalid	User will be asked to enter a valid value within specified range of 1 – 18,270	
18,721	Invalid	User will be asked to enter a valid value within specified range of 1 – 18,720	
1	Boundary	Simulation will start with one infected agent	
18,720	Boundary	Simulation will start with 18,720 infected agents	

‘1’	Erroneous	Program will not allow user to input quotation marks	
1.5	Erroneous	Program will not allow the user to input a decimal point	
‘’	Erroneous	User will be asked to enter a value	

Measure of Infection Rate

Input	Validation Type	Intended Output	Actual Output
0.4	Normal	Simulation will start with an infection rate of 0.4	
0.0	Invalid	User will be asked to enter a valid value within specified range of 0.01 - 0.99.	
1.0	Invalid	User will be asked to enter a valid value within specified range of 0.01 - 0.99.	
0.01	Boundary	Simulation will start with an infection rate of 0.01	
0.99	Boundary	Simulation will start with an infection rate of 0.99	
‘0.4’	Erroneous	Program will not allow user to input quotation marks	
‘’	Erroneous	User will be asked to enter a value	

Travel Radius

Input	Validation Type	Intended Output	Actual Output
2	Normal	Adds travel radius amount to the current x and y coordinates to find the	
-1	Invalid	User will be asked to enter a valid value within the specified range of 1 - 10	
11	Invalid	User will be asked to enter a valid value within the specified range	

1	Boundary	Simulation will start with a travel radius of 1	
10	Boundary	Simulation will start with a travel radius of 10	
‘1’	Erroneous	Program will not allow user to input quotation marks	
1.5	Erroneous	Program will not allow the user to input a decimal point	
‘’	Erroneous	User will be asked to enter a value	

Recovery Rate

Input	Validation Type	Intended Output	Actual Output
0.4	Normal	Simulation will start with an infection rate of 0.4	
0.0	Invalid	User will be asked to enter a value in valid range	
1.0	Invalid	User will be asked to enter a value in valid range	
0.01	Boundary	Simulation will start with an infection rate of 0.01	
0.99	Boundary	Simulation will start with an infection rate of 0.99	
‘0.4’	Erroneous	Program will not allow user to input quotation marks	
‘’	Erroneous	User will be asked to enter a value	

Fatality Rate

Input	Validation Type	Intended Output	Actual Output
0.4	Normal	Simulation will start with an infection rate of 0.4	
0.0	Invalid	User will be asked to enter a value in valid range	
1.0	Invalid	User will be asked to enter a value in valid range	

0.01	Boundary	Simulation will start with an infection rate of 0.01	
0.99	Boundary	Simulation will start with an infection rate of 0.99	
'0.4'	Erroneous	Program will not allow user to input quotation marks	
''	Erroneous	User will be asked to enter a value	

Iteration 2: Simulation Initialization

For this iteration I need to create the population for the disease model. Firstly I will need toe make sure that each agent within the population has the correct attributes and position within the square grid. The next step will be to implement the neighbouring agent's algorithm which will iterate through the population and create a list of neighbouring agents for each agent as an attribute. The button inputs on this page do not need to be tested since they have been validated by iteration 1.

Since all of the parameter data has been previously validated by iteration 1, there is no need test whether the use of the input data within the algorithms will lead to errors since the input data and conditional statements are all valid, however I need to test the logical pathways of each algorithm to make sure it produces the correct output. The respective algorithm test plans are outlined below:

Neighbouring Agents within Travel Radius Algorithm

1. Confirm that the only agents that the algorithm uses are directly adjacent to the selected agent
2. Confirm that each conditional statement produces the correct output

Conditional Statement	Intended Output	Actual Output
IF neighbour within maximum x and y coordinates	Will append agents within the maximum x and y coordinate radius to the neighbour_agents list	

Simulation Loop Algorithm

There is no need to test this function since it is a culmination of all the previous functions that already have a test plan laid out. It also does not have any conditional statements, therefore there is no need for testing the logical pathways through the program.

Iteration 3: Simulation Display

This iteration will link the previous two iterations together, with the simulation model being linked to the graphical grid of squares in the tkinter GUI. The product of this simulation will be that each square in the grid will be linked to an agent in the model, with the squares changing colour depending on what category the agent is in.

Simulation Display

1. Assign each category an intended colour

2. Each agent's square outputs the correct colour depending on their respective category at each time-step

SIR Category	Intended Colour Output	Actual Colour Output
Susceptible	White	
Infected	Pink	
Recovered	Green	
Fatality	Red	

Iteration 4: Infection Algorithm, Recovery Algorithm, Fatality Algorithm and Travel Radius

The goal of this iteration will be to implement the infection algorithm itself, which will emulate the spread of the disease itself through the previously defined population model using each agent's 'neighbouring agent' list attribute as a list of potential people to be infected. The algorithm will be based on chance and use the random module provided by python.

The goal of this algorithm is that when the model's step() method is called then neighbouring agents around every infected agent will have a chance to be come infected as well, excluding agents that are either recovered or a fatality.

Infection Algorithm

1. Will take the user's input as the infectionRate variable
2. Confirm that the algorithm only infects an agent due to a random chance
3. Confirm that the algorithm only uses agents directly neighbouring an infected agent

Conditional Statement	Intended Output	Actual Output
IF randominteger <= infectionRate	Will change the agent's state to 'infected'	
IF agent NOT IN neighbouring_agents	Will not consider agent in the algorithm	

Recovery Algorithm

1. Will take the user's input as the recoveryRate variable
2. Confirm that the algorithm only recovers an agent due to a random chance
3. Confirm that the algorithm only uses infected agents

Conditional Statement	Intended Output	Actual Output
IF randominteger <= recoveryRate	Will change the agent's state to 'recovered'	
IF agent.state NOT 'infected'	Will not consider agent in the algorithm	

Fatality Algorithm

1. Will take the user's input as the fatalityRate variable
2. Confirm that the algorithm only considers an agent as a fatality due to random chance
3. Confirm that the algorithm only uses infected agents

Conditional Statement	Intended Output	Actual Output

IF randominteger <= recoveryRate	Will change the agent's state to 'fatality'	
IF agent.state NOT 'infected'	Will not consider agent in the algorithm	

Iteration 5: Pre-sets

For this page I will need to test the functionality of the SQL database e.g., adding and removing pre-sets and making sure that pre-set values created and edited will be updated in the SQL database. I can test this by using SQLite to check that the database behaves accordingly to the user's respective actions in the program.

This test plan will test the functionality of the buttons in the 'Full Simulation', 'Pre-sets' page and 'Editable Pre-set Page' since they are directly linked to each other. It will test that pre-sets can be loaded into the simulation, as well as being added or cut on the pre-sets page as well as testing that the when the values are edited by the user on the 'Editable Pre-sets' page that they can then be saved to database. The buttons and input boxes already have previous test plans however, this test plan goes into more detail about what will happen in the SQL database itself when pre-sets are added, removed, or edited when the user presses the respective buttons on each page. It is outlined below:

Full Simulation Page Pre-set List

1. Confirm that pre-set values can be loaded as parameters to be used in the current simulation
2. Confirm that the specific values of the pre-set that the user clicks will be used in the simulation

Input	Intended Output	Actual Output
User Clicks on Pre-set Name	Associated pre-set values are loaded from the database and used in the current simulation	

List of Pre-set Page

1. Confirm that the cut button removes the selected pre-set from the SQL database

Input	Intended Output	Actual Output
User clicks on Cut button then clicks on a pre-set	Corresponding named pre-set entity will be removed from the SQL database	

Editable Pre-set Page

1. Confirm that the save button saves the current pre-set values as a named entity in the database

Input	Intended Output	Actual Output

User clicks on Save Button	The pre-set values will be added to the data base as a named entity	

Post-Development (Black-Box)

This section will look at a few different scenarios that the user is likely to engage in. It will test numerous sections across the program to make sure it's easily accessible and user friendly while making sure all the necessary functionality for the program to work is implemented. There will be three stages to my user testing, making sure that each section of the program works:

1 – the user wants to run a simulation, moving backwards and forwards

- selects the full simulation page from the main menu
- enters the parameters they want for their simulation and hits the play button
- presses the backwards arrow button a number of times
- presses the forwards arrow button a number of times
- pauses the simulation and repeats the two earlier steps
- continues the simulation using the play button
- exits simulation

2 – the user wants to run a simulation, changing the parameters while the simulation is in progress

- selects the full simulation page from the main menu
- enters the parameters they want for their simulation and hits the play button
- changes a parameter in one of the parameter boxes while the simulation is in progress
- exits simulation

3 – the user wants to load pre-sets into the simulation

- selects the full simulation page from the main menu
- selects a named pre-set from the list adjacent to the simulation box
- continues the simulation using the play button
- exits simulation

4 – the user wants to add and remove a pre-set

- selects the pre-sets page from the main menu
- presses the remove button
- subsequently presses a pre-set to remove
- clicks on the add button
- adds the desired values on the editable pre-set page
- saves the values
- exits the pre-set page

5 – the user wants to edit a pre-set

- selects the pre-sets page from the main menu
- presses a pre-set

- alters the value in the editable pre-set page
- presses the save button
- exits the pre-set page

6 – the user wants to view the tutorial pages

- selects the tutorial page from the main menu
- presses on the forwards and backwards buttons to move through tutorial pages
- exits the tutorial page

Alpha Testing

This stage will be dedicated to the practical testing of the program with a few of my stakeholders testing the program like how it would operate when being used in real-life applications.

To do this I decided to give each of my stakeholders a list of tasks that they can complete and fill out a feedback sheet afterwards to see what aspects of the program can be improved.

Here is the task list:

- run 4 different simulations with different parameters
- open the pre-sets page and add a pre-set
- open and read the tutorial page
- interact with anything else in the program you find interesting
- close the program

And here's the feedback sheet:

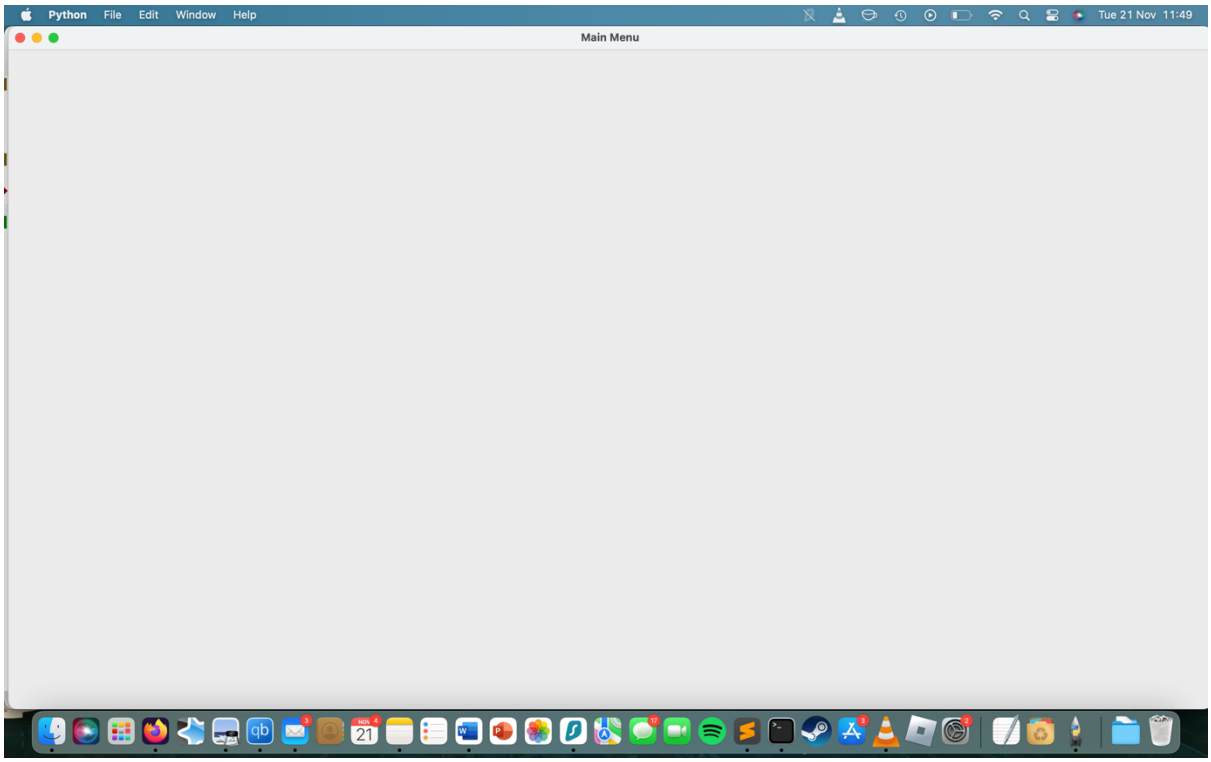
Question	**Answer**
Do you find the program easy to navigate?	
Do you find that the simulation presents enough options for running diseases?	
Do you find the display for the simulation intuitive to understand what's going on within the simplified population?	
Were there any tasks that didn't work as intended?	
Are there any features that you would like to see added to the program?	
Could you rate the ease of use of the system on a scale from 1-10?	

Development

Iteration 1: Menus, Boxes and Buttons

The navigation system of my program will solely be composed of the GUI, and for this reason I want to develop this iteration first, since it sets the basis for the more advanced parts of the program such as agent-based modelling. The module that I will be using for this iteration is tkinter. I am choosing this since it integrates well with mesa (the agent-based modelling module I will be using for this program)/

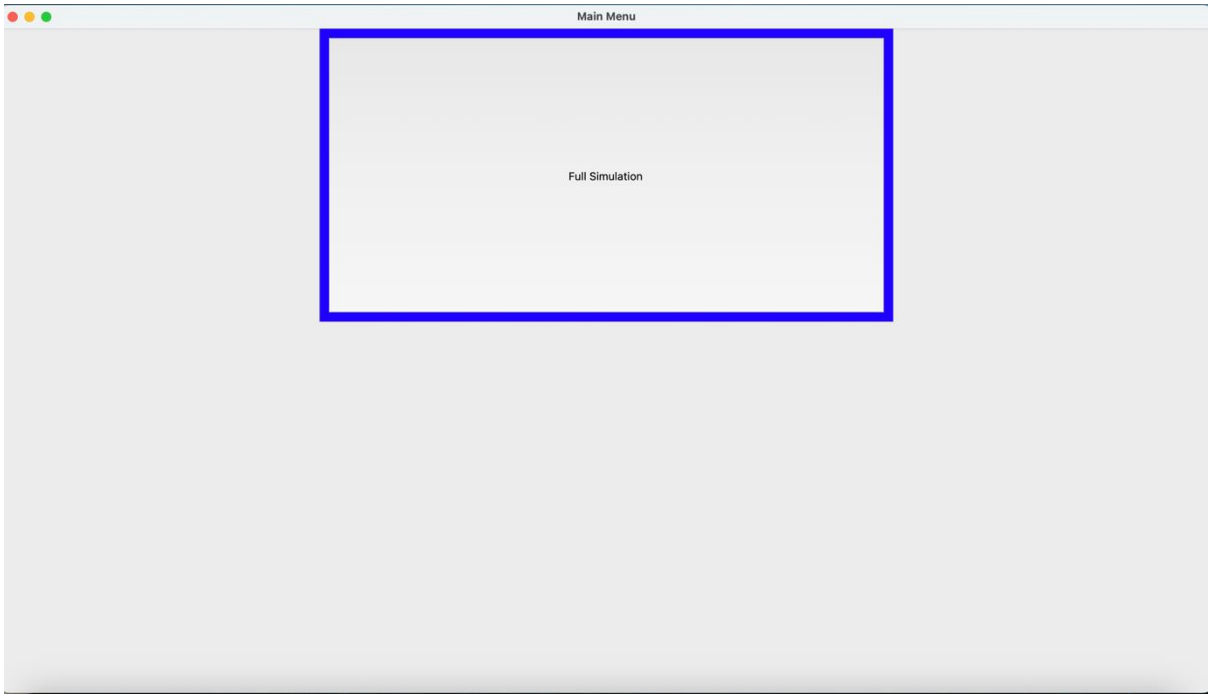
```
from tkinter import *

#function to initialise main window
def initialise():
    global root

    root = Tk()
    root.title("Main Menu")
    root.geometry("1440x900")
    root.mainloop()


#initates main window
initialise()
```



Firstly, I've created a function to initialise the window which is set as the resolution of the screen. This will be the basis for all the buttons leading to other parts of the program, acting as the main menu.

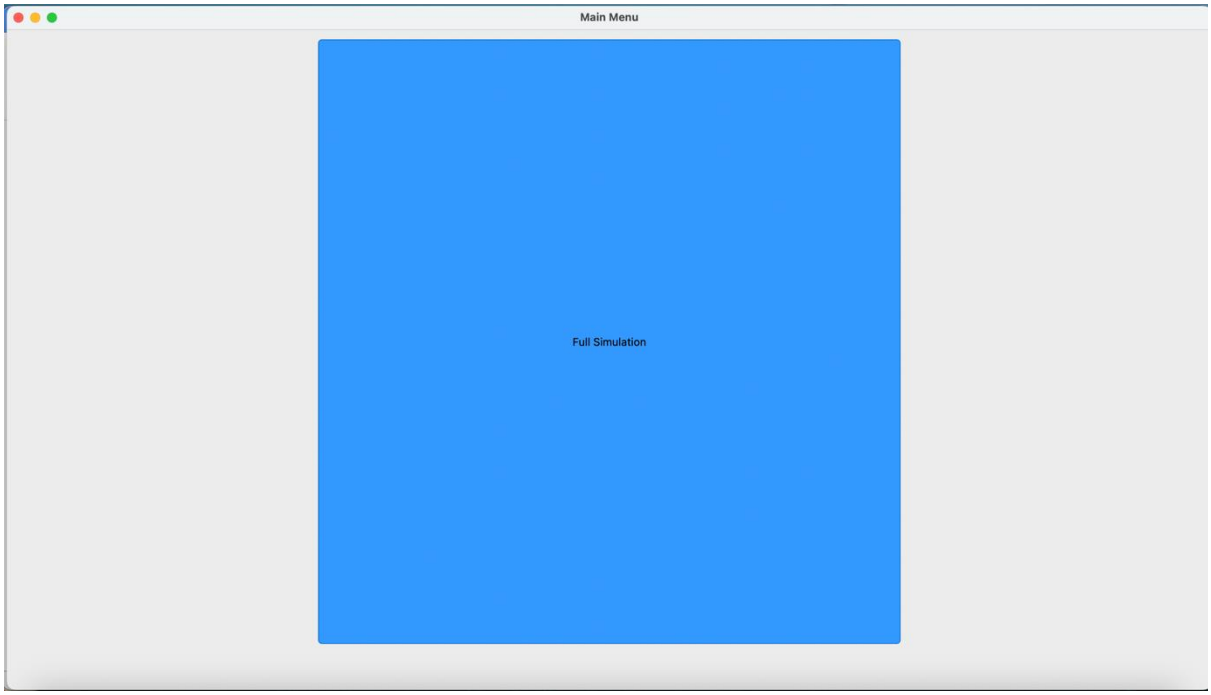
The next step of my program was to create a class for buttons, as outlined in my design. However, the module tkinter already has a built in class titled 'Button' with identical attributes to those I outlined in my class diagram. For this reason, I have not defined a button class, instead using tkinter's built in class. However in trying to use this class I ran into my first problem:

```
bfullSimulation = button1 = tk.Button(root,borderwidth = 10,text = "Full Simulation", height = 20, width = 70, highlightbackground='blue')
bfullSimulation.pack()
```



Instead of the background of the button going blue, it only made a specific border around the button blue. After some research it seems that the tkinter module has an error on mac where the background colour of a button is not displayed properly. To remedy this, I installed an alternative module called 'tkmacosx' which solves this issue, and called the button class from that which fixed the issue:

```
import tkmacosx as tkmacosx
import tkinter as tk

#function to initialise main window
def initialise():
    global root

    root = tk.Tk()
    root.title("Main Menu")
    root.geometry("1440x900")
initialise()

bfullSimulation = tkmacosx.Button(root,borderless = 1,text = "Full Simulation", height = 725, width = 700, bg='#3399ff')
bfullSimulation.pack(pady=10)

root.mainloop()
```

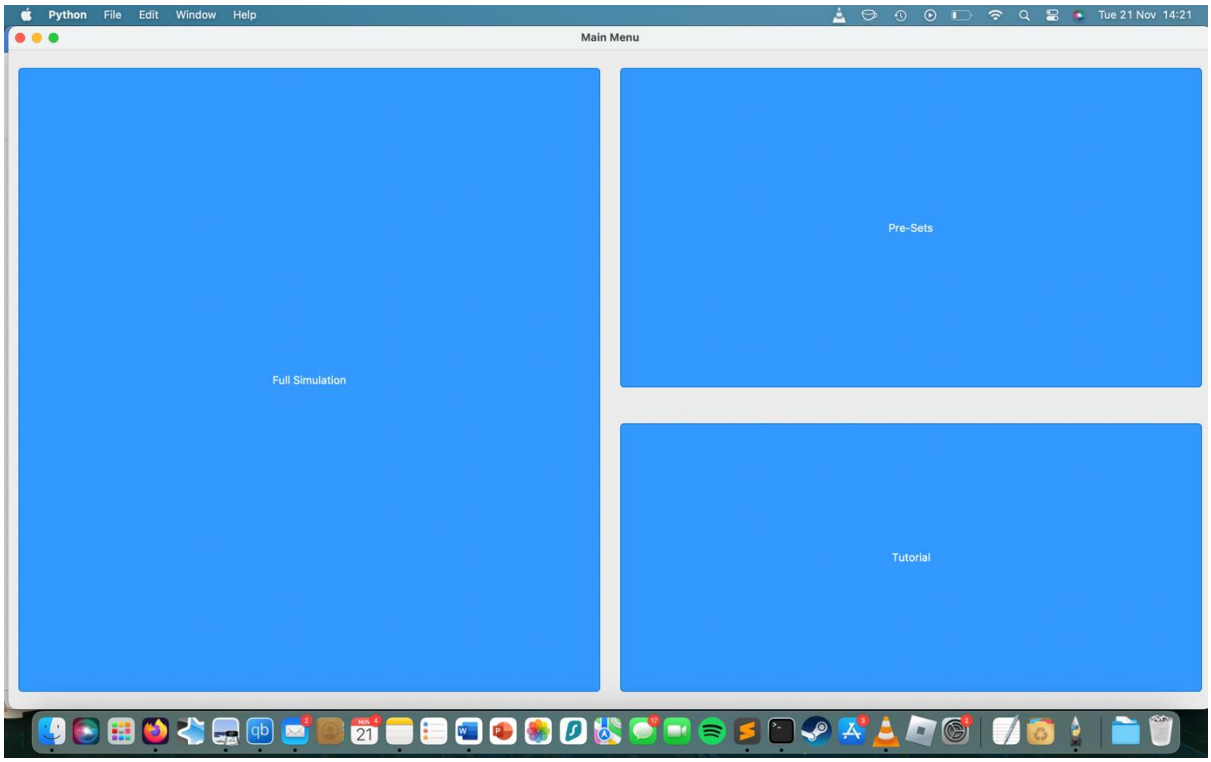


I then got to work on creating the other buttons for the main menu page. Firstly, I defined a set method for the creation of a button. This means that I can repeatedly call that method with different parameters for the button, meaning I don't need to repeatedly input the same parameters. There's no need to make this a class since it does not require attributes and is only created and then nothing else is done to it, contrary to the class diagram outlined in the design phase. I then positioned these buttons using tkinter's pack() feature. I used this alternatively to tkinters place() function, since this method automatically resizes the buttons as well as being simpler to implement since I don't need to define the individual x and y coordinates for each button, only the padding around the button and the direction that the button is anchored to, e.g. the 'pre-sets' button is anchored to the north while the 'tutorials' button is anchored to the south, allowing them to be positioned on top of each other.

Through my defined set method and the pack() method I implemented three different buttons within the main menu page:

```
import tkmacosx as tkmacosx
import tkinter as tk

#function to initialise main window
def initialise():
    global root

    root = tk.Tk()
    root.title("Main Menu")
    root.geometry("1440x900")

initialise()

#method to create buttons
def createButton(ptext,pheight, pwidth):
    #assigns constant and paramater values to a new button
    return tkmacosx.Button(root,borderless=1,font=("Helvetica", 30),text=ptext,height=pheight,width=pwidth,bg='#3399ff',fg='#FFFFFF')

#assigns a new button to a variable
bfullSimulation = createButton("Full Simulation",750,700)
#positions the button within the page
bfullSimulation.pack(anchor='center',side='left',pady=20,padx=10)

bpreSets = createButton("Pre-Sets",385,700)
bpreSets.pack(anchor='n',pady=20,padx=10)

btutorial = createButton("Tutorial",385,700)
btutorial.pack(anchor='s',pady=20,padx=10)

#loops the page so it's interactable
root.mainloop()
```

Leading to the creation of the main menu page for the program:



The next step was to create individual pages for each button, in which when clicked the user would be transferred to that specific page. To do this I firstly needed to re-organise the main menu page into a frame. This would allow it to be linked by other pages as well as having the ability to place links to multiple pages in a one to many relationship. The first thing I did was to define the different classes of the pages as well as the main class so that different components of the different pages would be separated into individual modules. I began by re-organising the main menu page into a class which looks like this:

```
class MainMenu :
    #setter method that nitialises buttons as attributes
    def __init__(self,b1,b2,b3) :
        self.b1 = b1
        self.b2 = b2
        self.b3 = b3

    #method used to pack buttons into page / initialise buttons
    def setPage(self):
        self.b1.pack(anchor='center',side='left',pady=20,padx=10)
        self.b2.pack(anchor='n',pady=20,padx=10)
        self.b3.pack(anchor='s',pady=20,padx=10)
```

I then initiated this class and used the setter method to initiate the main menu page:

```
#creates an instance of the mainmenu class with createButton methods as paramaters for buttons in the page
p1 = MainMenu(createButton(window,"Full Simulation",750,700),createButton(window,"Pre-Sets",385,700),createButton(window,"Tutorial",385,700))
#initalises the setter method to pack buttons into page
p1.setPage()
```

However, after tinkering around with the class, I realised this was an incorrect structure for the program, since I would need to input the buttons as parameters every time MainMenu was initiated. Another reason is that when a button is defined in tkinter you need to define the command along with it, in this case the command to go to a different page, which would be a method of the class MainMenu. I can't reference the class within itself, so I changed the structure of the class, so instead of the buttons being parameters inputted by the users, they are constant attributes that are already defined in the initialisation method:

```
class MainMenu :
    #setter method that nitialises buttons as attributes
    def __init__(self) :
        b1 = createButton(window,"Full Simulation",750,700)
        b2 = createButton(window,"Pre-Sets",385,700)
        b3 = createButton(window,"Tutorial",385,700)

        b1.pack(anchor='center',side='left',pady=20,padx=10)
        b2.pack(anchor='n',pady=20,padx=10)
        b3.pack(anchor='s',pady=20,padx=10)
```

This new structure calls the createButton functions (defiend earlier in the program) to create the buttons and then assings it as a named attribute to MainMenu. It then uses the tkinter method pack() to display it on the page. Howver after running the program the buttons did not display on the page. This turned out to be because I did not define the atrrbitures in referene to the class with the self paramtaer. Here is the correct version:

```
class MainMenu :
    #initialises the buttons on the page
    def __init__(self) :
        self.b1 = createButton(window,"Full Simulation",750,700)
        self.b2 = createButton(window,"Pre-Sets",385,700)
        self.b3 = createButton(window,"Tutorial",385,700)

        self.b1.pack(anchor='center',side='left',pady=20,padx=10)
        self.b2.pack(anchor='n',pady=20,padx=10)
        self.b3.pack(anchor='s',pady=20,padx=10)
```

After this I began working on a way to create multiple pages in tkinter, however after some research I found out that tkinter does not support loading and discarding individual pages. So I found a workaround. I needed to create two individual methods, one to clear a selected page's contents and one to load a selected page's content. These two methods would give the user the illusion that the program was switching pages when in fact it would only be wiping and different configurations of objects all on the same page. Therefore, I needed to create a list of the objects on each page which could be passed to these methods so they would receive all the contents of the page as one parameter. I did this for the MainMenu class by placing all the buttons inside a list called objects:

```
self.objects = [self.b1,self.b2,self.b3]
```

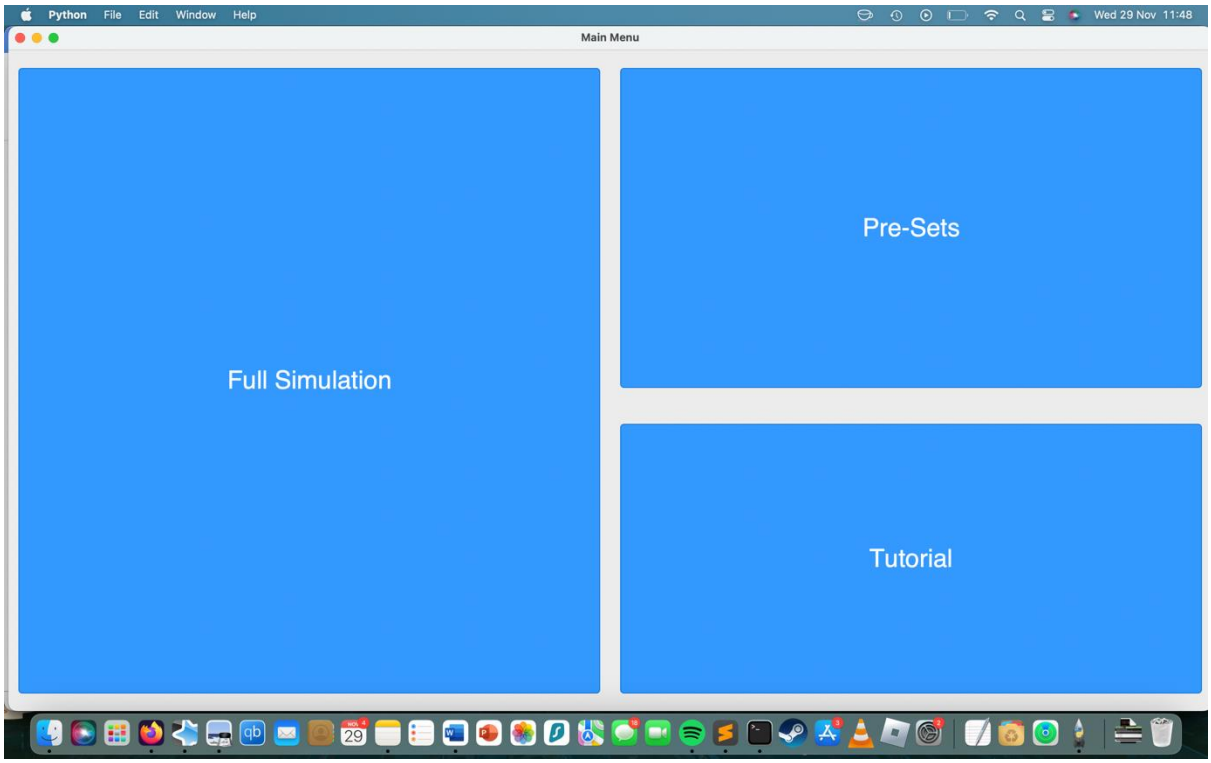
I then defined the clear method outside of the scope of the class, so it can be used with multiple pages in the future:

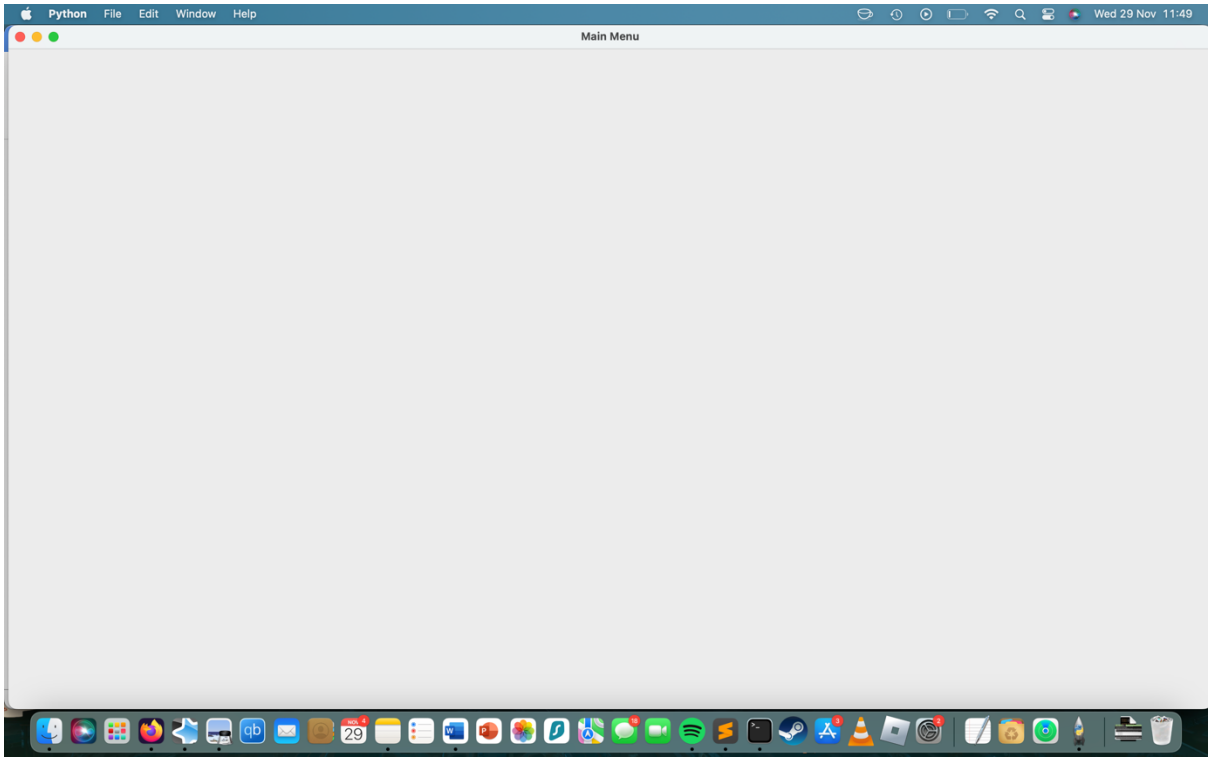
```
def clear(items):
    [items[i].pack_forget() for i in range(len(items))]
```

This method iterates through the list and uses the tkinter pack_forget() for each item in the list, removing it from the page. I then assigned this method to every button in the MainMenu class using tkinter's configure() method with the objects list as a parameter:

```
self.b1.configure(command=lambda: clear(self.objects))
self.b2.configure(command=lambda: clear(self.objects))
self.b3.configure(command=lambda: clear(self.objects))
```

I had to use the lambda keyword otherwise the clear() method was called when it was assigned to a button, wiping the contents of the page before the user had even clicked on the button.. So when a button was clicked the contents of the page were wiped like so:





The next step was to initialise a new page, once the contents of the page were wiped. Firstly I created a new class called Page1 with only a button that was titled 'This is a new page' as a test to make sure the contents aren't just being wiped and the page is being initialised:

```
class Page1 :
    def __init__(self) :
        self.b1 = createButton(window,"This is a new page",750,700)
        self.b1.pack(anchor='center',side='left',pady=20,padx=10)
```

I then re-defined the clearPage() method into getPage(). I changed it by adding a paramater and a process that would load the assosiated page class, based on the page-number paramater. This means that the method would clear the page and then initialise the new page's class:

```
def getPage(items,pageNum):
    #iterates through items list and gets rid of each item from page
    [items[i].pack_forget() for i in range(len(items))]
    #loads the specified page based on number paramater
    if pageNum == 1 :
        Page1()
```

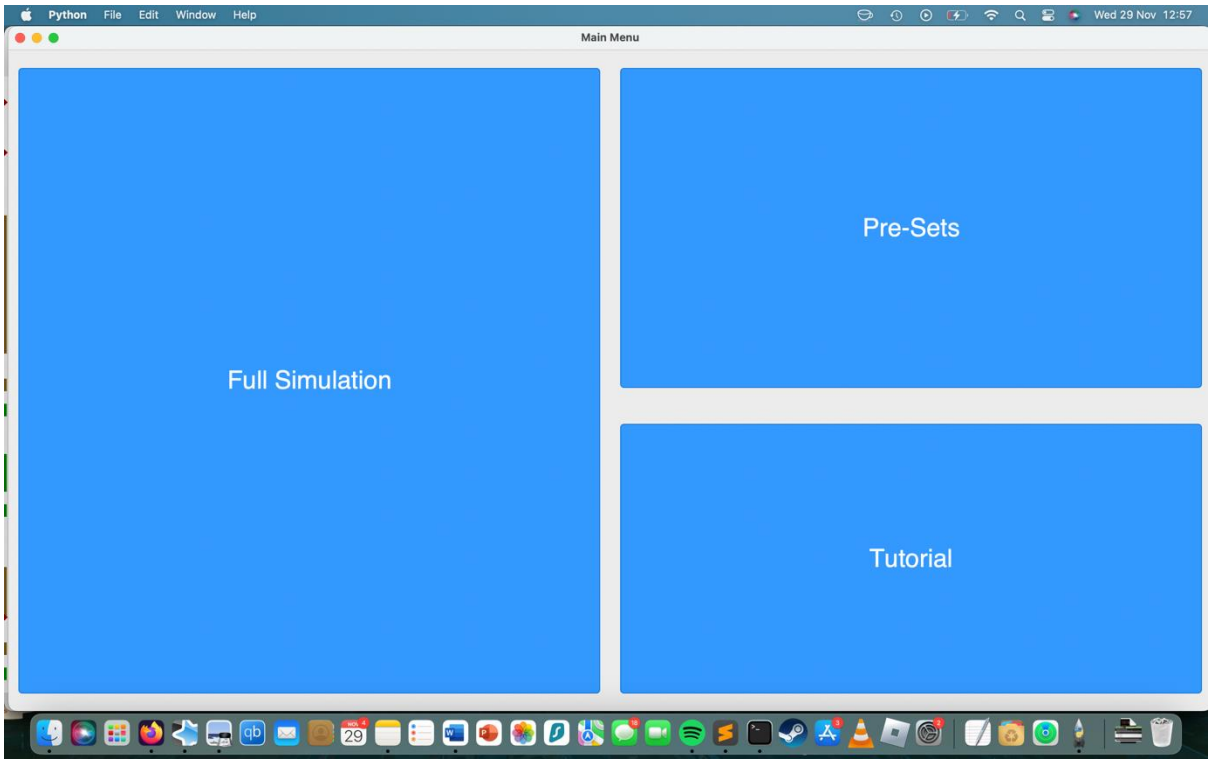
I then edited the command for the 'Full Simulation' button in the MainMenu() class so that when it was clicked it would call the getPage() method with the list of items and a pageNum of 1:

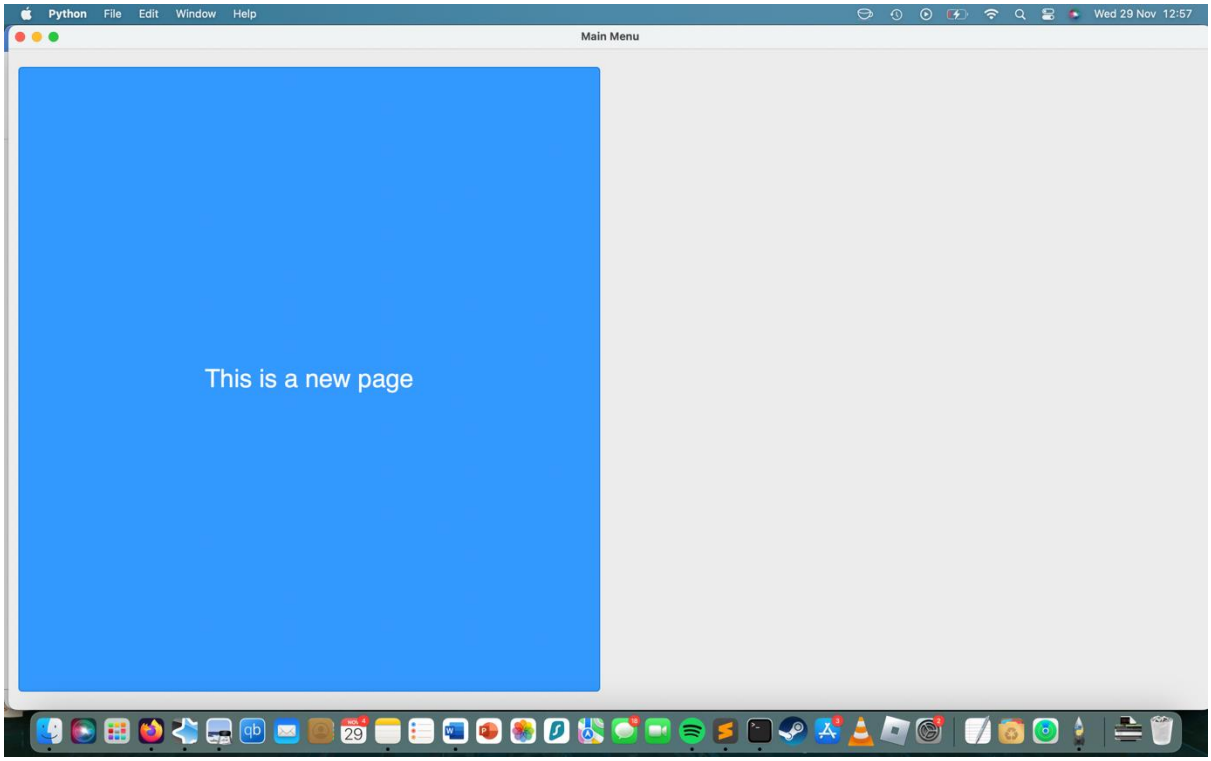
```
#runs getPage() method to run with objects list when button is clicked
self.b1.configure(command=lambda: getPage(self.objects,1))
self.b2.configure(command=lambda: getPage(self.objects,1))
self.b3.configure(command=lambda: getPage(self.objects,1))
```

This is what the source code looks like after implementing those changes:

```
import tkmacosx as tkmacosx
import tkinter as tk

window = tk.Tk()
window.title("Main Menu")
window.geometry("1440x900")

def createButton(ppage,ptext,pheight, pwidth):
    #assigns constant and paramater values to a new button
    return tkmacosx.Button(ppage,borderless=1,font=("Helvetica", 30),text=ptext,height=pheight,width=pwidth,bg='#3399ff',fg='#FFFFFF')

def getPage(items,pageNum):
    #iterates through items list and gets rid of each item from page
    [items[i].pack_forget() for i in range(len(items))]
    #loads the specified page based on number paramater
    if pageNum == 1 :
        Page1()

class Page1 :
    def __init__(self) :
        self.b1 = createButton(window,"This is a new page",750,700)
        self.b1.pack(anchor='center',side='left',pady=20,padx=10)

class MainMenu :
    #initialises the objects on the page
    def __init__(self) :
        #creates each button for the main menu page
        self.b1 = createButton(window,"Full Simulation",750,700)
        self.b2 = createButton(window,"Pre-Sets",385,700)
        self.b3 = createButton(window,"Tutorial",385,700)

        #adds each button to a list
        self.objects = [self.b1,self.b2,self.b3]

        #packs each button into page
        self.b1.pack(anchor='center',side='left',pady=20,padx=10)
        self.b2.pack(anchor='n',pady=20,padx=10)
        self.b3.pack(anchor='s',pady=20,padx=10)

        #runs getPage() method to run with objects list when button is clicked
        self.b1.configure(command=lambda: getPage(self.objects,1))
        self.b2.configure(command=lambda: getPage(self.objects,1))
        self.b3.configure(command=lambda: getPage(self.objects,1))

main = MainMenu()
#loops the page so it's ineractable
window.mainloop()
```

So when the ‘Full Simulation’ button was clicked this was the result:





Therefore my program can now initiate new pages. Now that that function has been implemented, I created separate classes for each page and then associated each class with their respective button in the MainMenu(). Here are the separate classes for each page that MainMenu() links to:

```
class FullSimulation :
    def __init__(self) :
        self.b1 = createButton(window,"This is the full simulation page",750,700)
        self.b1.pack(anchor='center',side='left',pady=20,padx=10)

class PreSets() :
    def __init__(self) :
        self.b1 = createButton(window,"This is the pre-sets page",750,700)
        self.b1.pack(anchor='center',side='left',pady=20,padx=10)

class Tutorial():
    def __init__(self) :
        self.b1 = createButton(window,"This is the tutorial page",750,700)
        self.b1.pack(anchor='center',side='left',pady=20,padx=10)
```

And then I assosiated each page with a number in the getPage() method:

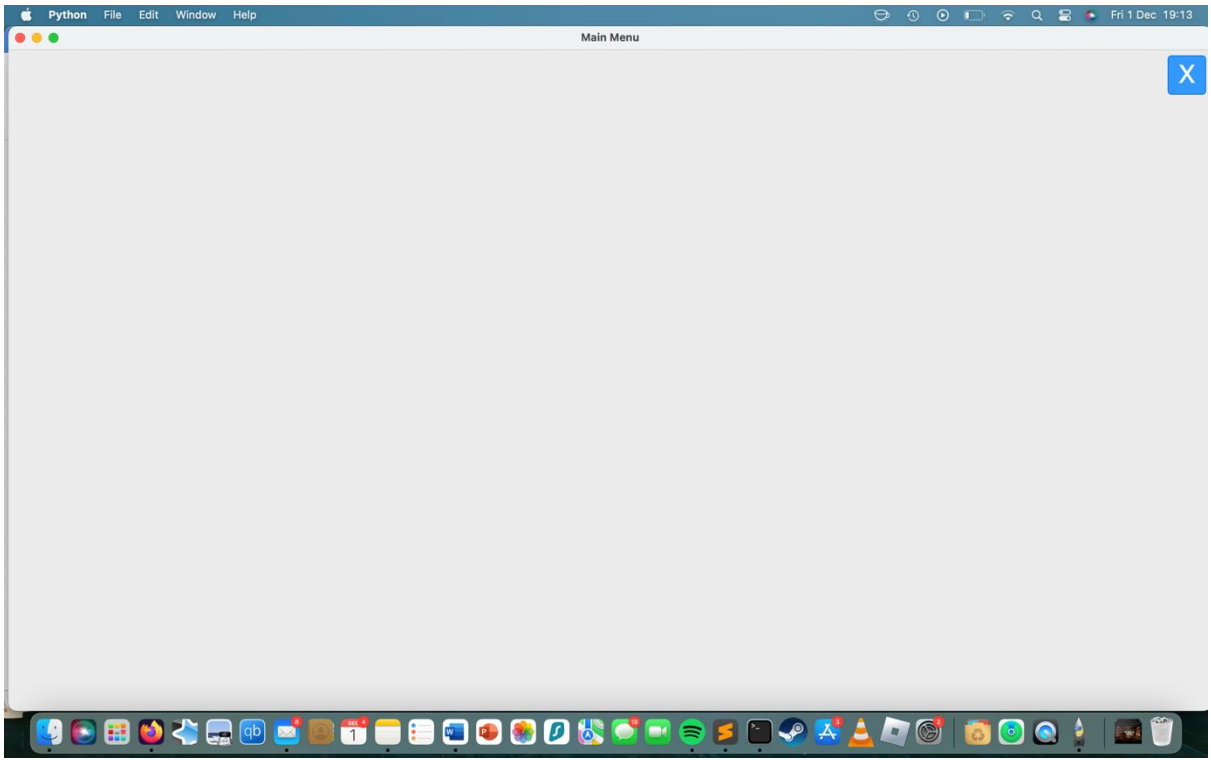
```
def getPage(items,pageNum):
    #iterates through items list and gets rid of each item from page
    [items[i].pack_forget() for i in range(len(items))]
    #loads the specified page based on number paramater
    if pageNum == 1 :
        FullSimulation()
    elif pageNum == 2:
        PreSets()
    elif pageNum == 3:
        Tutorial()
```

And then associated each button with each page number in the getPage() method:

```
#runs getPage() method to run with objects list when button is clicked
self.b1.configure(command=lambda: getPage(self.objects,1))
self.b2.configure(command=lambda: getPage(self.objects,2))
self.b3.configure(command=lambda: getPage(self.objects,3))
```

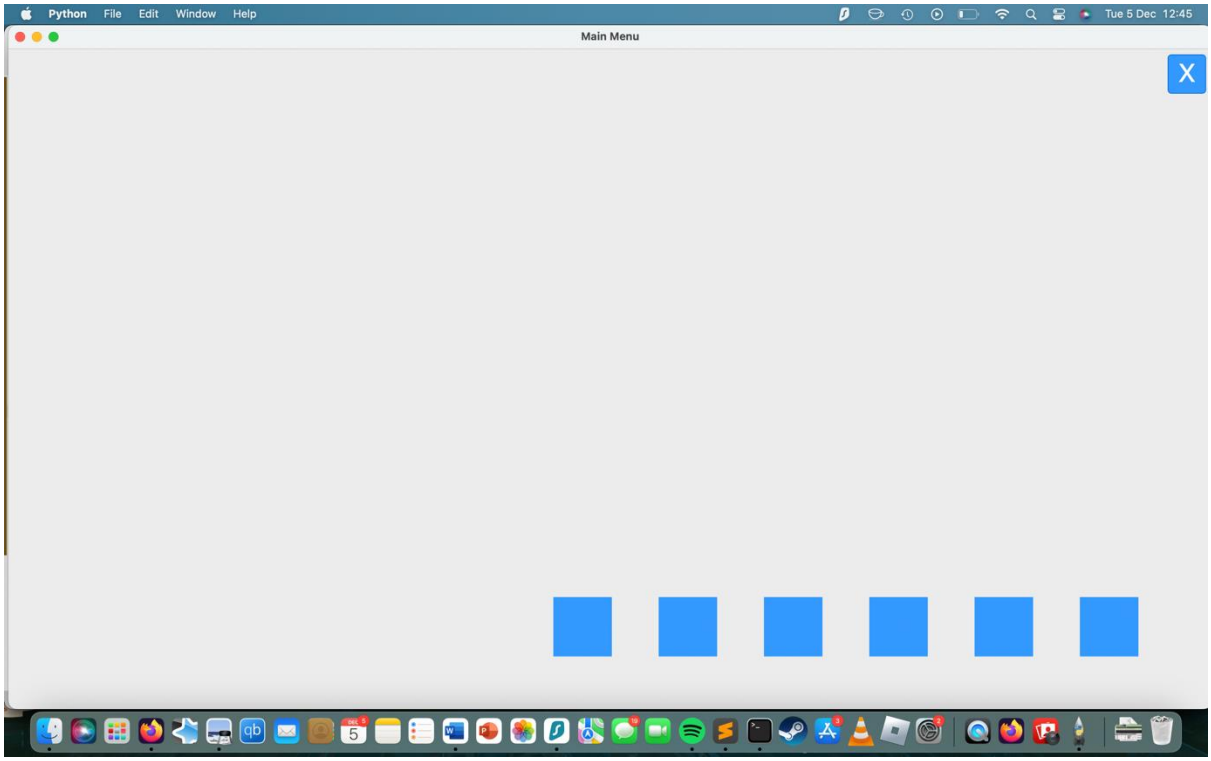
I then created a back button on each individual page and assigned it to call the MainMenu() class when it was clicked:

```
def __init__(self) :
    #back button
    self.b1 = createButton(window,"X",50,50)
    self.b1.pack(anchor='n',side='right',pady=5,padx=5)
    self.objects = [self.b1]

    #calls main menu class
    self.b1.configure(command=lambda: getPage(self.objects,0))
```



I then decided to add input boxes to the Full Simulation page, to record the user's inputted values, as outlined in the design section. To do this I used tkinter's built in Text() method which create a box in which text can be inputted and recorded, I created six of these on the 'Full Simulation' page, each to represent an imputable parameter in the future:

```
#defines input boxes
self.i1 = tk.Text(window,width=2,height=1,bg='#3399ff',fg='#FFFFFF',font=("Helvetica",60))
self.i2 = tk.Text(window,width=2,height=1,bg='#3399ff',fg='#FFFFFF',font=("Helvetica",60))
self.i3 = tk.Text(window,width=2,height=1,bg='#3399ff',fg='#FFFFFF',font=("Helvetica",60))
self.i4 = tk.Text(window,width=2,height=1,bg='#3399ff',fg='#FFFFFF',font=("Helvetica",60))
self.i5 = tk.Text(window,width=2,height=1,bg='#3399ff',fg='#FFFFFF',font=("Helvetica",60))
self.i6 = tk.Text(window,width=2,height=1,bg='#3399ff',fg='#FFFFFF',font=("Helvetica",60))

self.b1.pack(anchor='n',side='right',pady=5,padx=5)

#packs input boxes
self.i1.pack(anchor='s',side='right',pady=60,padx=25)
self.i2.pack(anchor='s',side='right',pady=60,padx=25)
self.i3.pack(anchor='s',side='right',pady=60,padx=25)
self.i4.pack(anchor='s',side='right',pady=60,padx=25)
self.i5.pack(anchor='s',side='right',pady=60,padx=25)
self.i6.pack(anchor='s',side='right',pady=60,padx=25)

self.objects = [self.b1,self.i1,self.i2,self.i3,self.i4,self.i5,self.i6]
```



I then needed a way to record the values entered by the user. To do this I used the get() method provided by tkinter to retrive the text that the user has entered into the text box. Since I would be repeating this throughout the program with different input boxes as well as the data produced needing to be passed to other processes in the program in the future. Therefore I created a function that would perform this and return the text entered by the user:

```
def retrieve_input(inputText):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    print(inputText.get("1.0",tk.END))
```

This used get() to return the text that would be entered by the user. I then needed to assign this to each input box in the 'Full Simulation' class as well as assigning this to a keystroke. To do this I used tkinter's bind() command to bind the function retrieve_input() to each input box, so that whenever

the return key was pressed by the user the function would run and hopefully return the inputted text, in this case printing it to terminal. To do this I create a small anonymous function using lambda, that would call the retrieve_input function whenever the return key was pressed:

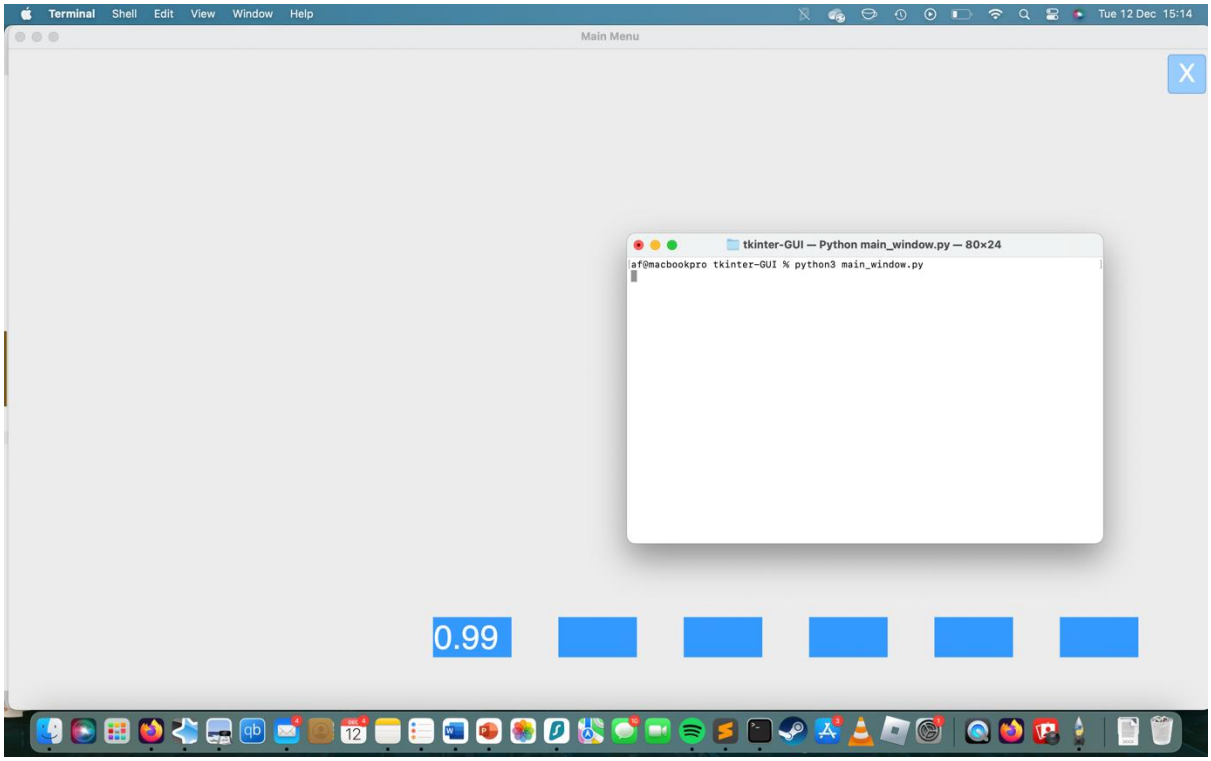
```
#binds retrieve input function to input box whenever enter key is pressed
self.i1.bind("<Return>",lambda: retrieve_input(self.objects[1]))
self.i2.bind("<Return>",lambda: retrieve_input(self.objects[2]))
self.i3.bind("<Return>",lambda: retrieve_input(self.objects[3]))
self.i4.bind("<Return>",lambda: retrieve_input(self.objects[4]))
self.i5.bind("<Return>",lambda: retrieve_input(self.objects[5]))
self.i6.bind("<Return>",lambda: retrieve_input(self.objects[6]))
```

However, when I tried to run this code I encountered an error:

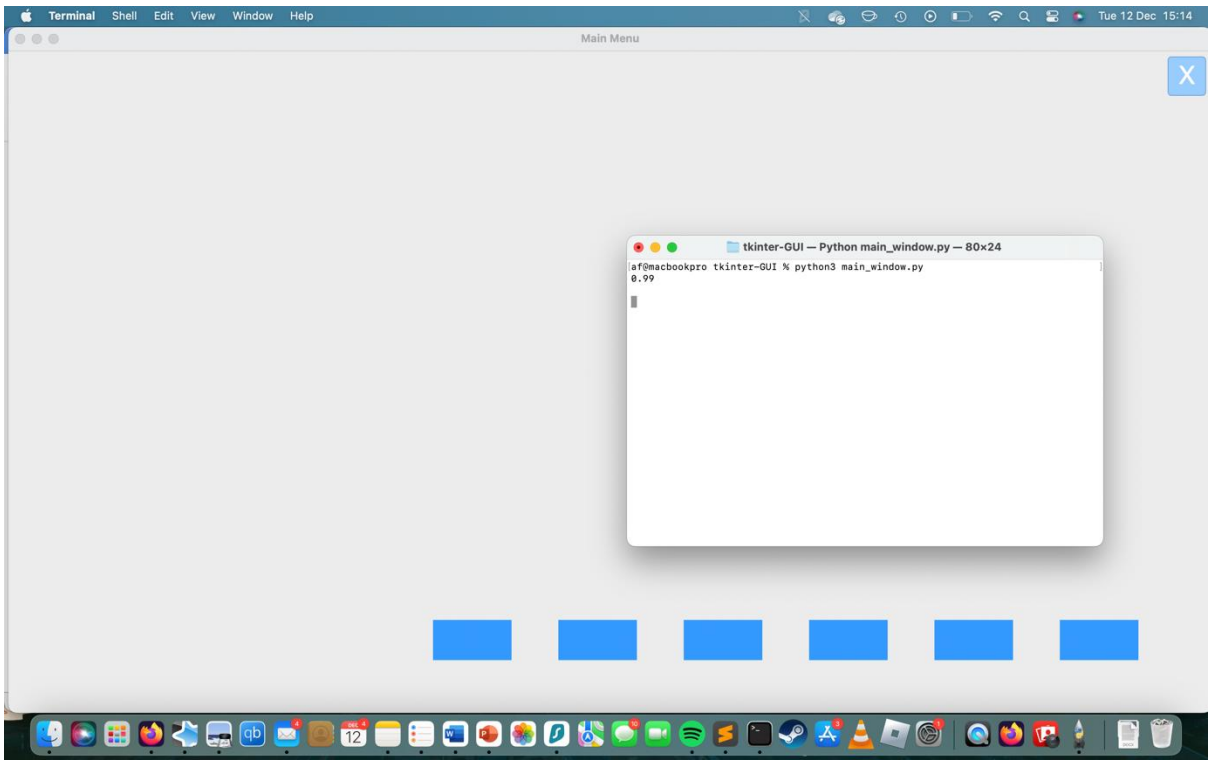
```
TypeError: FullSimulation.__init__.<locals>.<lambda>() takes 0 positional argume
nts but 1 was given
```

This took a while to figure out, but after researching the lamba function it turns out that the problem was occuring since pressing the Return key was considered an event, so this event object was being passed to the created lambda function alongside the retrieve_input function, meanining there were two arguments being passed instead of one. The way to solve this, was to define an ‘event’ paramater within the lambda function which would handle the event, and when it occurred would run the retrieve_input function:

```
#binds retrieve input function to input box whenever enter key is pressed
self.i1.bind("<Return>",lambda event: retrieve_input(self.objects[1]))
self.i2.bind("<Return>",lambda event: retrieve_input(self.objects[2]))
self.i3.bind("<Return>",lambda event: retrieve_input(self.objects[3]))
self.i4.bind("<Return>",lambda event: retrieve_input(self.objects[4]))
self.i5.bind("<Return>",lambda event: retrieve_input(self.objects[5]))
self.i6.bind("<Return>",lambda event: retrieve_input(self.objects[6]))
```



Now whenever text was entered and the return key was pressed, the number would be outputted to the terminal via the retrieve_input function:



Once I had implemented this the next step was to implement the validation as outlined in the test plan for this iteration. To do this I altered the retrieve input function to add input validation based on what input box the user had entered data in. I did this using python's case feature, where each case depended on what button was being pressed. I did this by adding a parameter to the function so that I could add the number for the parameter box that was being validated, so each box would

have a different validation case:

```
def retrieve_input(inputText,inputBox):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    paramaterValue = inputText.get("1.0",tk.END)
    #validates whether the value is an integer or float
    try:
        paramaterValue = int(paramaterValue)
    except ValueError:
        paramaterValue = float(paramaterValue)

    #input validation as outlined via test plan
    match inputBox :
        case 1 :
            if paramaterValue > 0 and paramaterValue < 5001 and type(paramaterValue) == int:
                print(f'{paramaterValue} is valid')
            else :
                print(f'{paramaterValue} is not valid')
        case 2 :
            if paramaterValue > 0 and paramaterValue < 5000 and type(paramaterValue) == int:
                print(f'{paramaterValue} is valid')
            else :
                print(f'{paramaterValue} is not valid')
        case 3 :
            if paramaterValue > 0 and paramaterValue < 11 and type(paramaterValue) == int:
                print(f'{paramaterValue} is valid')
            else :
                print(f'{paramaterValue} is not valid')
        case 4 | 5 | 6 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                print(f'{paramaterValue} is valid')
            else :
                print(f'{paramaterValue} is not valid')
```

And then added a value for this new inputBox parameter to each input box in the ‘Full Simulation’ class:, so that each input box would correspond to it’s responding case in the retrieve_input function.

```
#binds retrieve input function to input box whenever enter key is pressed
self.i1.bind("<Return>",lambda event: retrieve_input(self.objects[5],1))
self.i2.bind("<Return>",lambda event: retrieve_input(self.objects[6],2))
self.i3.bind("<Return>",lambda event: retrieve_input(self.objects[7],3))
self.i4.bind("<Return>",lambda event: retrieve_input(self.objects[8],4))
self.i5.bind("<Return>",lambda event: retrieve_input(self.objects[9],5))
self.i6.bind("<Return>",lambda event: retrieve_input(self.objects[10],6))
```

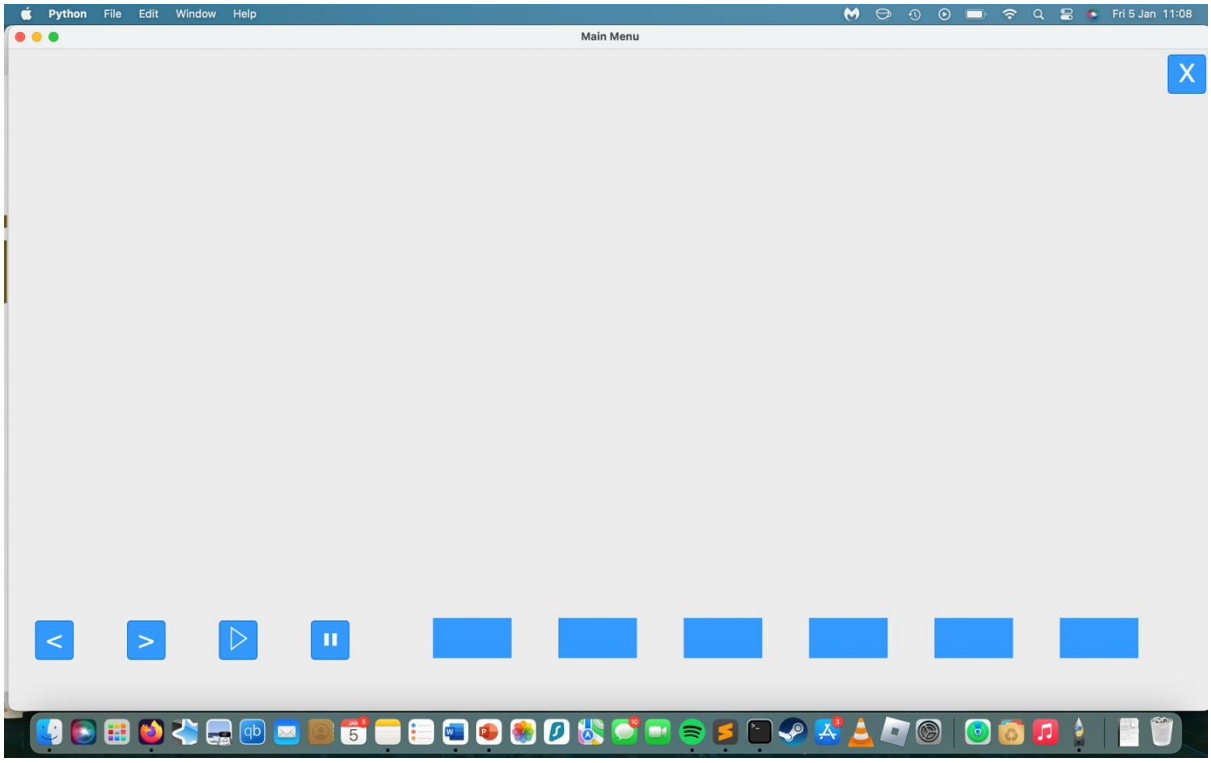
However, when I tried to enter two values in the same input box, one after another, an error appeared:

```
ValueError: could not convert string to float: '99\n99\n'
```

What was interesting about this error was that the output contained new line spaces. I deduced after some trial and error that the previous value entered the input box wasn’t cleared when a new one was put in. This meant that when two values were entered, one after another, the previous one became part of the new value. The way to fix this error was to wipe the input box once the value had been assigned to the paramaterValue variable so it wouldn’t carry over to the next value inputted by the user. Adding this into the retrieve_input function solved the error:

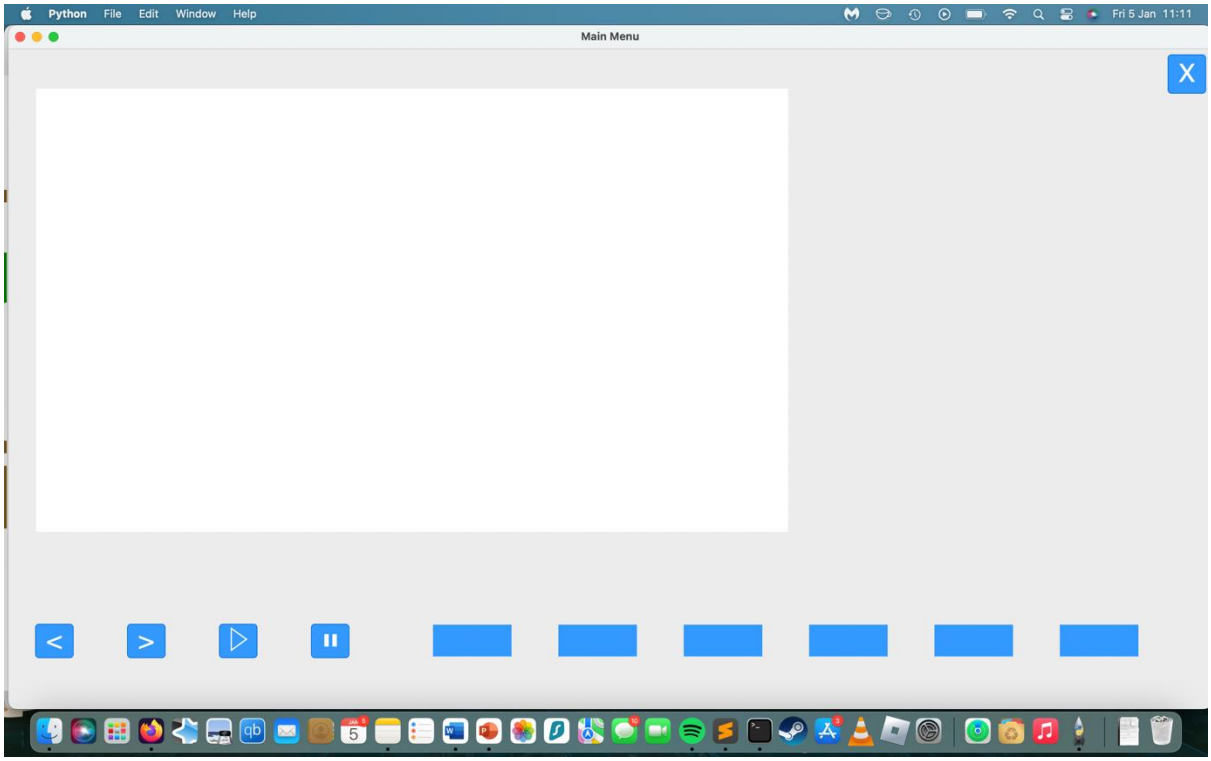
```
def retrieve_input(inputText,inputBox):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    paramaterValue = inputText.get("1.0",tk.END)
    #deletes text from input box so isnt added on next time function is run
    inputText.delete("1.0",tk.END)
```

I then needed to create the two buttons to move the simulation forwards one time step, as well as backwards one time-step so I just created two more buttons each with a symbol on them. I also added play and pause buttons that would start and stop the simulation when pressed:



I then needed an area within the main page to add my square grid for the simulation. After some research I realised that to implement graphics into a tkinter program the canvas function is used. I therefore implemented a canvas in the FullSimulation class:

```
#canvas
self.squareGrid = tk.Canvas(window,bg="#FFFFFF", height=530, width=900)
self.squareGrid.pack(side='top',anchor='w',pady=45,padx=30)
```



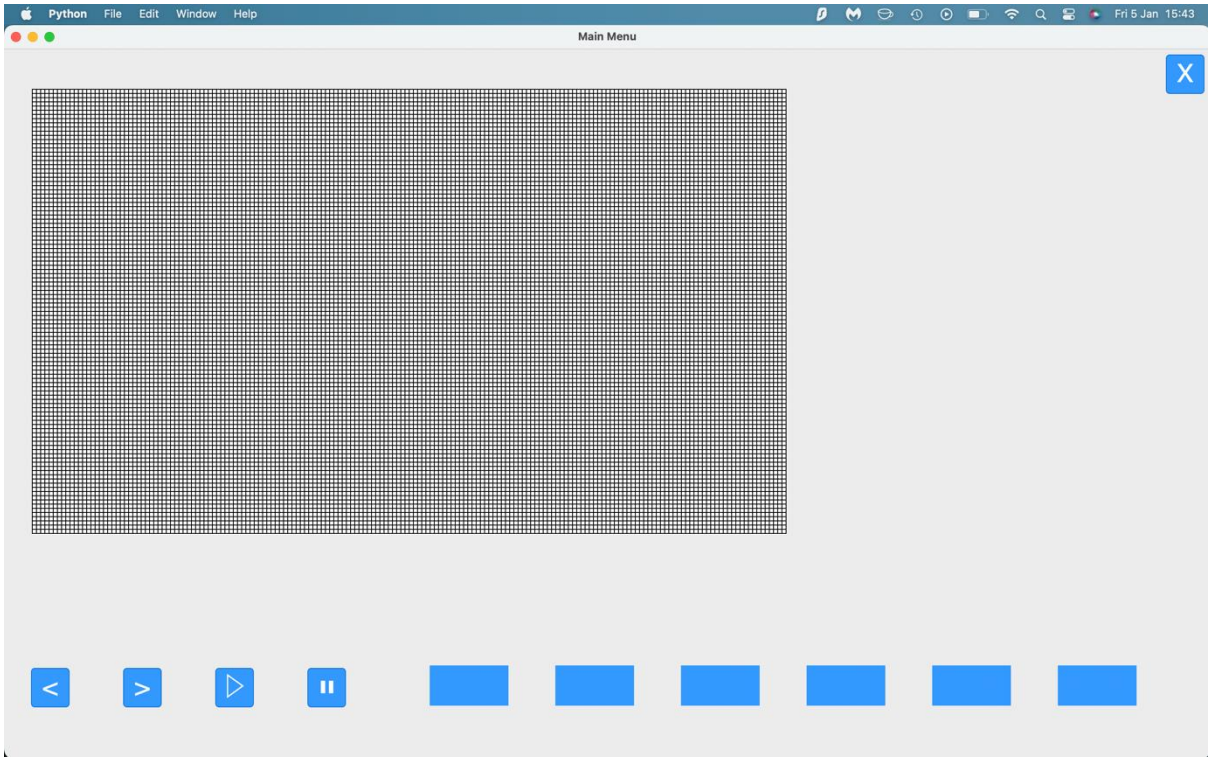
I then needed to create a grid of squares within the canvas that I could link to the simulation itself so that each square in the canvas would represent an agent in the simulation and could change colour according to the agent's state. To do this I used tkinter's create_rectangle method to create the squares with a width and height of 5 pixels, then I used nested for loops, with the ranges of the canvas' height and width to fill the canvas with blank squares. I then put all of this inside a function:

```
def draw_canvas(canvas) :
    #nested for loop for coordinates
    for x in range(3,900,5) :
        for y in range(3,530,5):
            #creates and packs squares
            canvas.create_rectangle(x, y, x+5, y+5, fill="white")
            canvas.pack()
```

I then told it to run at the end of the initialisation of the FullSimulation class so that when the page was initialised:

```
#binds retrieve input function to input box whenever enter key is pressed
self.i1.bind("<Return>",lambda event: retrieve_input(self.objects[5],1))
self.i2.bind("<Return>",lambda event: retrieve_input(self.objects[6],2))
self.i3.bind("<Return>",lambda event: retrieve_input(self.objects[7],3))
self.i4.bind("<Return>",lambda event: retrieve_input(self,self.objects[8],4))
self.i5.bind("<Return>",lambda event: retrieve_input(self.objects[9],5))
self.i6.bind("<Return>",lambda event: retrieve_input(self.objects[10],6))

#creates reference square grid
self.squareGrid = draw_canvas(self.squareCanvas)
```

Leading to this result:



However,, after this step I could not begin work on other aspects since the square grid required an agent-based model for the squares to be linked to. The input boxes at the bottom of the page also needed the simulation to be developed so they could be linked to it for user inputs. I also required a functioning SQL database to begin work on the pre-sets page. And finally, I required the program to be near completed to begin work on the tutorial section which would detail all the different functionalities of the program. Therefore, I decided to move onto my second iteration of the program, which would be developing the actual simulation itself using agent-based modelling and then implementing it into the 'Full Simulation Page'.

Changes in Design Made in this Iteration

Firstly, there cannot be separate page titles for each subsection of the program e.g. the tutorial page cannot have tutorial as the title on the top of the page since tkinter does not have the functionality to accommodate that.

I have also made major changes to the class structure. I've structured the GUI so that each page is its own class with the elements of that page being the attributes which are then packed into the page. There's also no need for a text box class since tkinter allows text boxes to be within buttons automatically without the user having to define them.

The parameters are not their own class but instead are attributes of the MainMenu class. Buttons are also not their own class but the method planned to be included in the button class is in the program with it being repeatable function that creates buttons and is referenced in multiple page classes. Although the menu is a class called MainMenu, it has much more parameters and the method of initiating a new page is its own function since it's referenced by multiple page classes not just MainMenu.

How has this filled Success Criteria and User Requirements?

This iteration has fulfilled the following success criteria:

- Buttons:

- Full Simulation
- Tutorial
- Pre-sets
- Quit Button
- 2D Grid Graphic
- Play Button
- Pause Buttons
- Forward Arrow Button
- Backward Arrow Button
- Parameter Input Boxes
- Exit Button

However, parameter titles have not yet been implemented but they will at a later date.

It has filled the user requirements by creating large, labelled buttons and a main menu page that functions as a central hub for the other pages from which they can be accessed. Squares within the grid have also been implemented however not the colours within the grid haven't been implemented yet but will in a later iteration. However, text boxes have not been implemented yet due to the packing system tkinter uses for elements, however this will be implemented later in the program.

Iteration 2: Simulation Initialisation

The first step is to create the simulation independent of tkinter, and then once all the functionalities of the simulation have been developed, I can integrate the simulation into the GUI itself to allow the simulation to be interacted with in the 'Full Simulation' page in the next iteration. Therefore, I began by developing the grid that the represented the disease population. I decided to develop this iteration in a file called 'population.py', completely independent of the main program so I could focus solely on the population of the simulation without worrying about affecting the already-existing GUI.

Firstly, I needed to create the population, to do this I used the mesa module to create a class with parameters that would define the model. It also inherits all methods from mesa's model class. I then created the grid using mesa's SingleGrid class to create a grid where each cell can at most hold only one agent, since in this model, agents will only get infected by their proximity and not whether they're in the same cell in the grid. Then I defined the scheduler to be used for the model, this scheduler would define how agents are activated at each time step. I've chosen to use the SimultaneousActivation scheduler so that all agents within the grid are activated at the same time during each time step. Finally, I defined a step method which would call the scheduler at each time step, which in turn will call the step method of each agent which has not yet been defined:

```
import mesa
import seaborn as sns
import numpy as np

class PoopulationModel(mesa.Model):
    def __init__(self, num_agents, width, height):
        self.num_agents = N
        #agents can't be within the same cell in the grid
        self.grid = mesa.space.SingleGrid(width, height, True)
        #defines scheduler that will activate all agents simultaenously at each time step
        self.schedule = mesa.time.SimultaneousActivation(self)
```

The next step was to define the agent class that will be used within the disease model. To do this I created another class called PopulationAgent that inheritaded all the methods from mesa's agent class. I then defined the paramaters as outlined in my class diagram as well as one paramter detailing the model that it will be used with, they then were also passed to mesa's agent class via the super() method. Since every person in a population starts off as 'susceptible', the agent's initial state was set to 'S' to represent them being suceptible:

```
class PopulationAgent(mesa.Agent) :
    def __init__(self,unique_id,model):
        super().__init__(unique_id,model)
        self.state = 'S'
        self.neighbour_agents = []
```

I then added in a step method for the model that when called would initialise every agent's step method simultaneously due to the SimulatneousActivation method referenced earlier when creating the grid attribute :

```
class PopulationModel(mesa.Model):
    def __init__(self, num_agents, width, height):
        self.num_agents = N
        #agents can't be within the same cell in the grid
        self.grid = mesa.space.SingleGrid(width, height, True)
        #defines scheduler that will activate all agents simultaenously at each time step
        self.schedule = mesa.time.SimultaneousActivation(self)

    def step(self):
        self.schedule.step()
```

The next step was to add all the agents to the grid defined in the PopulationModel class. The reason for this is that every cell in the grid needed to have an agent in it since, as outlined in my algorithms section, the disease would spread to directly neighbouring agents that are in contact, meaning there can't be empty spaces in the grid since the disease would not be able to spread between agents. To achieve this I created a reference 2-indices coordinate array of coordinates using nested for loops via list comprehension and the height and width parameters since python has no in-built array() method. This meant it had the identical coordinates and number of spaces to the grid since the width and height parameters could be set to those identical to the grid. This meant the coordinate positions of the agents could be used to reference coordinate positions found in the grid. I then used a for loop to iterate through this array, each time creating an agent and adding it to the scheduler as well as adding it to the model grid using the respective coordinate found in the refCoordinates array.

```
class PopulationModel(mesa.Model):
    def __init__(self, num_agents, width, height):
        self.width = width
        self.height = height

        self.num_agents = num_agents
        #agents can't be within the same cell in the grid
        self.grid = mesa.space.SingleGrid(self.width, self.height, True)
        #defines scheduler that will activate all agents simultaenously at each time step
        self.schedule = mesa.time.SimultaneousActivation(self)

        #creates reference array of coordinates for agents to be added to
        refCoordinates = [[x,y] for x in range(self.height) for y in range(self.width)]

        for i in refCoordinates:
            a = PopulationAgent(refCoordinates.index(i), self)
            #adds the agent to the scheduler
            self.schedule.add(a)
            #places agent within the grid
            self.grid.place_agent(a, (i[0],i[1]))
```

Finally, I needed to add a step method to each agent that the scheduler could call when it moves forward one time-step. I decided to add a method to print the agent's id and current state as a test to see whether the model was functioning:

```
class PopulationAgent(mesa.Agent) :
    def __init__(self,unique_id,state,model):
        super().__init__(unique_id,state,model)
        self.state = 'S'

    def step(self):
        #prints agent id and state
        print(f'I am agent{self.unique_id} and my state is {self.state}')
```

I then defined and ran the model for a single time-step to see if it worked:

```
testModel = PopulationModel(9,3,3)
testModel.step()
```

Therefore I had created a blank grid and added agents to each cell of the grid to represent a population, each initially starting in the category of suceptible. The next step was to create a list of each agent's direct neighbours. To do this I've used the algorithm outlined in the design section called the 'Neighbouring Agents within Travel Radius Algorithm'. I then implemented the 'Neighbouring Agents within Travel Radius Algorithm' with a travel radius of 1 in python code within a temporary file which looked like this:

```
#creates a grid, representing populationModel grid
array = [[x,y] for x in range(10) for y in range(10)]
#radius of what is considered a neighbour agent
radius = 1

#current agent position
agent_position = [3,3]

#max and min x,y neighbour positions
x_travel_radius_positions = [agent_position[1]+radius,agent_position[1]-radius]
y_travel_radius_positions = [agent_position[0]+radius,agent_position[0]-radius]

#creates a list of all possible positions with range of the max and min x,y neighbour positions
total_radius_positions = [[x,y] for x in range(x_travel_radius_positions[1],x_travel_radius_positions[0]+1)
                                for y in range(y_travel_radius_positions[1],y_travel_radius_positions[0]+1)]

#removes the position of the agent it's finding the neighbours of
total_radius_positions.remove([agent_position[0],agent_position[1]])
```

I then added this algorithm within the code with the travelRadius being an attribute of the class instead of set at 1. Since the position of the agents do not change while the model runs, I decided to add this within the populationModel's initialisation so that every agent would have a list of neighbouring agents assigned to them when the model starts:

```
for i in self.grid:
    print(f'Finding neighbouring agents for agent {i.unique_id} at {i.pos[0]},{i.pos[1]}')
    #max and min x,y neighbour positions
    x_travel_radius_positions = [i.pos[0]-self.travelRadius,(i.pos[0]+self.travelRadius)+1]
    y_travel_radius_positions = [i.pos[1]-self.travelRadius,(i.pos[1]+self.travelRadius)+1]

    #creates a list of all possible positions with range of the max and min x,y neighbour positions
    total_radius_positions = [[x,y] for x in range(x_travel_radius_positions[0],x_travel_radius_positions[1])
                                    for y in range(y_travel_radius_positions[0],y_travel_radius_positions[1])]

    #removes the position of the agent it's finding the neighbours of
    total_radius_positions.remove([i.pos[0],i.pos[1]])
    print(f'Here are the neighbouring agents: {total_radius_positions}')
```

```
af@macbookpro full-simulation % python3 main.py
Finding neighbouring agents for agent 0 at 0,0
Here are the neighbouring agents: [[-1, -1], [-1, 0], [-1, 1], [0, -1], [0, 1], [1, -1], [1, 0], [1, 1]]
Finding neighbouring agents for agent 1 at 0,1
Here are the neighbouring agents: [[-1, 0], [-1, 1], [-1, 2], [0, 0], [0, 2], [1, 0], [1, 1], [1, 2]]
Finding neighbouring agents for agent 2 at 0,2
Here are the neighbouring agents: [[-1, 1], [-1, 2], [-1, 3], [0, 1], [0, 3], [1, 1], [1, 2], [1, 3]]
Finding neighbouring agents for agent 3 at 1,0
Here are the neighbouring agents: [[0, -1], [0, 0], [0, 1], [1, -1], [1, 1], [2, -1], [2, 0], [2, 1]]
Finding neighbouring agents for agent 4 at 1,1
Here are the neighbouring agents: [[0, 0], [0, 1], [0, 2], [1, 0], [1, 2], [2, 0], [2, 1], [2, 2]]
Finding neighbouring agents for agent 5 at 1,2
Here are the neighbouring agents: [[0, 1], [0, 2], [0, 3], [1, 1], [1, 3], [2, 1], [2, 2], [2, 3]]
Finding neighbouring agents for agent 6 at 2,0
Here are the neighbouring agents: [[1, -1], [1, 0], [1, 1], [2, -1], [2, 1], [3, -1], [3, 0], [3, 1]]
Finding neighbouring agents for agent 7 at 2,1
Here are the neighbouring agents: [[1, 0], [1, 1], [1, 2], [2, 0], [2, 2], [3, 0], [3, 1], [3, 2]]
Finding neighbouring agents for agent 8 at 2,2
Here are the neighbouring agents: [[1, 1], [1, 2], [1, 3], [2, 1], [2, 3], [3, 1], [3, 2], [3, 3]]
```

However I came across a problem, the algorithm was including coordinate positions that were not in the grid. To fix this I made a series of if statement that summarised that if the coordinate was out of range, it would be replaced by the maximum or minimum limit of what that coordinate can be. This solved the problem. While doing this I also assigned a list of neighbouring agents to each agent as the loop iterated through them:

```
for i in self.grid:
    print(f'Finding the neighbouring agents for agent {i.unique_id} at {i.pos[0]},{i.pos{1}}')
    #max and min x,y neighbour positions
    minxPos, maxxPos = a.pos[0]-self.travelRadius, a.pos[0]+self.travelRadius
    minyPos, maxyPos = a.pos[1]-self.travelRadius, a.pos[1]+self.travelRadius

    #coordinate validation
    if minxPos < 0 : minxPos = 0
    if maxxPos > self. width: maxxPos = self.width
    if minyPos < 0 : minyPos = 0
    if maxyPos > self.width: maxyPos = self.height

    #creates a list of all possible positions with range of the max and min x,y neighbour positions
    total_radius_positions = [[x,y] for x in range(minxPos,maxxPos+1)
                                    for y in range(minyPos,maxyPos+1)]

    #removes the position of the agent it's finding the neighbours of
    total_radius_positions.remove([a.pos[0],a.pos[1]])
    #assigns the list as an attribute
    a.neighbour_agents = total_radius_positions
    print(f'Here are the neighbouring agents: {total_radius_positions}')
```

```
af@macbookpro full-simulation % python3 main.py
Finding neighbouring agents for agent 0 at 0,0
Here are the neighbouring agents: [[0, 1], [1, 0], [1, 1]]
Finding neighbouring agents for agent 1 at 0,1
Here are the neighbouring agents: [[0, 0], [0, 2], [1, 0], [1, 1], [1, 2]]
Finding neighbouring agents for agent 2 at 0,2
Here are the neighbouring agents: [[0, 1], [0, 3], [1, 1], [1, 2], [1, 3]]
Finding neighbouring agents for agent 3 at 1,0
Here are the neighbouring agents: [[0, 0], [0, 1], [1, 1], [2, 0], [2, 1]]
Finding neighbouring agents for agent 4 at 1,1
Here are the neighbouring agents: [[0, 0], [0, 1], [0, 2], [1, 0], [1, 2], [2, 0], [2, 1], [2, 2]]
Finding neighbouring agents for agent 5 at 1,2
Here are the neighbouring agents: [[0, 1], [0, 2], [0, 3], [1, 1], [1, 3], [2, 1], [2, 2], [2, 3]]
Finding neighbouring agents for agent 6 at 2,0
Here are the neighbouring agents: [[1, 0], [1, 1], [2, 1], [3, 0], [3, 1]]
Finding neighbouring agents for agent 7 at 2,1
Here are the neighbouring agents: [[1, 0], [1, 1], [1, 2], [2, 0], [2, 2], [3, 0], [3, 1], [3, 2]]
Finding neighbouring agents for agent 8 at 2,2
Here are the neighbouring agents: [[1, 1], [1, 2], [1, 3], [2, 1], [2, 3], [3, 1], [3, 2], [3, 3]]
```

Once I had done this, the next step was to link each agent created in this simulation to a respective square on the square grid within the 'Full Simulation' page thus creating the graphical aspect of the program. I decided to do this before coding the algorithm that would be used to model disease spread, since it would be easier to troubleshoot the simulation when each square is visible and the squares will be able to change colour based on their respective category. Therefore I decided to add a new iteration to the program which would focus on incorporating the simulation and the graphical square grid together.

Changes in Design Made in this Iteration

Firstly, although this algorithm does use the algorithms for this iteration outlined in the test plan, they are not identical to those outlined since although the concepts are identical, the implementation of these algorithm have changed to fit the mesa module. For example, with the agent initialisation algorithm, it does create a population however instead of a list it is a grid and instead of using the append() method it uses an in-built module method titled add() and then place_agent() to add the agent to the population grid.

For the SIR categorisation algorithm outlined in the test plan, instead of having a dictionary to store each agent in each category, I implemented a more effective method which was for each agent to have a separate attribute which would define their respective state. This meant I saved on storage

space and was a more efficient solution since it eliminated the need for another loop to add agents to a dictionary.

Finally, the simulation loop algorithm wasn't needed since this looping function was handled within the mesa module using it's time scheduler so it could automatically loop without the need to explicitly code it into the program, calling the step() method of each agent each time it looped.

Regarding the class diagram for the agent, it largely remains unchanged in the implementation, however it doesn't require the ycoord and xcoord attributes since they gain coordinates automatically when added to the grid and therefore when coordinates are referenced in the grid, they will return the respective agent. Another thing is that I did not implement was an updateState() method since I didn't see any need for it since in the future, I could alter the agent's 'state' attribute by referencing it directly without the need for a method. Although this doesn't make use of the encapsulation principle of object-orientated programming, since it's quite a compact program with the user not being able to access the source code, I decided that implementing encapsulation would be unnecessary.

How has this filled Success Criteria and User Requirements?

- Population Model

Iteration 3: Simulation Display

This iteration will solely focus on integrating the population model with the square grid in the tkinter GUI. The goal of this iteration is to assign each square in the grid it's own respective agent, as well as coding functionality for each square changing colour based on it's respective agent's state, e.g. red if their state is 'infected'.

The first thing I needed to do was to alter the reference grid within the 'population.py' file to match the square grid in the GUI, so that the coordinates of each square and agent would line up:

```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.width = 180
        self.height = 104

        #agents can't be within the same cell in the grid
        self.grid = mesa.space.SingleGrid(self.width, self.height, True)
        #defines scheduler that will activate all agents simultaenously at each time step
        self.schedule = mesa.time.SimultaneousActivation(self)

        #creates reference array of coordinates for agents to be added to
        refCoordinates = [[x,y] for y in range(0,self.height) for x in range(0,self.width)]
```

Then I needed to call my population model within the GUI itself. To do this I altered the 'FullSimulation', 'PreSets and 'Tutorial' pages to include an attribute that would account for the population model. After doing this I altered the get_page function so that whenever a page was called by the user, it would specify the population as an attribute to be passed into the page:

```
def get_page(items,pageNum,model):
    #iterates through items list and gets rid of each item from page
    [items[i].pack_forget() for i in range(len(items))]
    #loads the specified page based on number paramater
    if pageNum == 0:
        MainMenu()
    elif pageNum == 1 :
        FullSimulation(model)
    elif pageNum == 2:
        PreSets(model)
    elif pageNum == 3:
        Tutorial(model)
```

Once this was done, I focused on assigning each square in the grid an agent from the population model. Firstly, I needed to alter the population model so it would create the same number of agents as there are squares in the grid. As the method to draw the canvas was pixel specific, so had pixel intervals of 5 as well as starting from three pixels so that the squares aligned properly with the border, it was not a reliable way to create a reference grid. So, I firstly needed to find out how many squares there were in the grid as well as see the possible coordinate positions without the intervals of five as well as making sure to start the coordinate count from zero instead of three. To do this I created two separate variables which would count the number of x and y loops to return the total number of coordinate positions:

```
def draw_canvas(canvas) :
    ix = -1
    iy = -1
    #nested for loop for coordinates
    for x in range(3,900,5) :
        #accurate coordinates
        ix += 1
        iy = -1
        for y in range(3,520,5) :
            iy += 1
            #creates and packs squares
            canvas.create_rectangle(x, y, x+5, y+5, fill="white")
            #prints accurate coordinates
            print(f'coords are [{ix},{iy}]')
```

program — Python main_window.py — 80×24

```
coords are [179,82]
coords are [179,83]
coords are [179,84]
coords are [179,85]
coords are [179,86]
coords are [179,87]
coords are [179,88]
coords are [179,89]
coords are [179,90]
coords are [179,91]
coords are [179,92]
coords are [179,93]
coords are [179,94]
coords are [179,95]
coords are [179,96]
coords are [179,97]
coords are [179,98]
coords are [179,99]
coords are [179,100]
coords are [179,101]
coords are [179,102]
coords are [179,103]
d
```

Since this included the [0,0] coordinate position the total amount of coordinate pairs amounted to 18,720 which was the amount of squares. Therefore I had created an equal number of coordinate positions to squares in the grid.

The next step was to implement this in the population model in the 'population.py' file so that the grid within the population model would have identical coordinate positions to the squares in the GUI. As I looked over the model, I realised that the ability to change the agent population is no longer possible due to the grid resizing the elements (such as buttons) of the simulation page if the number of squares fluctuated and the grid size changed. Therefore I decided to remove this paramater from the population model in 'population.py' and set the height and width of the model to match with the gird within tkinter:

```
class PopulationModel(mesa.Model):
    def __init__(self,travelRadius):
        self.width = 900
        self.height = 520
        self.travelRadius = travelRadius
```

However while trying to run 'population.py''s code I came across an error where nothing was returned. This confused me and I tried to pinpoint the erroneous section of code in my program. This turned out to an error within the for loop where the agents were assigned to their positions in the grid:

```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.width = 180
        self.height = 104

        #agents can't be within the same cell in the grid
        self.grid = mesa.space.SingleGrid(self.width, self.height, True)
        #defines scheduler that will activate all agents simultaenously at each time step
        self.schedule = mesa.time.SimultaneousActivation(self)

        #creates reference array of coordinates for agents to be added to
        refCoordinates = [[x,y] for y in range(0,self.height) for x in range(0,self.width)]

        #loop to place agents in grid and scheduler
        for i in refCoordinates:
            a = PopulationAgent(refCoordinates.index(i), self)
            #adds the agent to the scheduler
            self.schedule.add(a)
            #places agent within the grid
            self.grid.place_agent(a, (i[0],i[1]))
```

The problem was that although I was assigning each agent in the model a unique id, I was doing this by using the index of where the agent was in refCoordinates which had to run a linear search in the whole of the refCoordinates array each time an agent was created, meaning the loop was an unintetional nested loop which reduced efficency and meant that it took to long to run so nothing was returned in terminal since the loop was taking so long the run. The solution to fix this was to use a different method to assign each a unique agent_id. I chose to use enmuerate() which creates another counter that records the loop number. I then used this second counter value as each agent's unique id:

```
#loop to place agents in grid and scheduler
for d,i in enumerate(refCoordinates):
    a = PopulationAgent(d, self)
    #adds the agent to the scheduler
    self.schedule.add(a)
    #places agent within the grid
    self.grid.place_agent(a, (i[0],i[1]))
```

However once this was done and the program was running I encountered another error:

```
Traceback (most recent call last):
  File "/Users/af/Library/CloudStorage/OneDrive-Mi
m/population.py", line 62, in <module>
    t = PopulationModel(1)
        ^^^^^^^^^^^^^^^^^^
  File "/Users/af/Library/CloudStorage/OneDrive-Mi
m/population.py", line 34, in __init__
    self.grid.place_agent(a, (i[0],i[1]))
  File "/usr/local/lib/python3.11/site-packages/me
ce_agent
    if self.is_cell_empty(pos):
       ^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.11/site-packages/me
cell_empty
    return self._grid[x][y] == self.default_val()
           ~~~~~~~~~~~^^^
IndexError: list index out of range
```

To find the error I added a print statement within the reference coordinate loop to print out coordinate positions to see if any were out of range of the model's grid:

```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.width = 180
        self.height = 104

        #agents can't be within the same cell in the grid
        self.grid = mesa.space.SingleGrid(self.width, self.height, True)
        #defines scheduler that will activate all agents simultaenously at each time step
        self.schedule = mesa.time.SimultaneousActivation(self)

        #creates reference array of coordinates for agents to be added to
        refCoordinates = [[x,y] for y in range(0,self.height) for x in range(0,self.width)]
```

After doing this I realised that due to me adding 1 to the range of the heigth and width as I assumed you needed to do to index the coordinates properly, lead there to being one more coordinate position than was in the grid. So it was quite an easy fix by just removing the plus-one for the range of the height and width paramater. Once this was fixed all I set the height to 108 and the width to 180 to match the for loop in the main window page:

```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.width = 180
        self.height = 104
```

I then printed the coordinate positions of the population model to make sure they matched up with the grid:

```
#creates reference array of coordinates for agents to be added to
refCoordinates = [print([x,y]) for y in range(0,self.height) for x in range(0,self.width)]
```

```
program — -zsh — 80×24
coords: [81,179]
coords: [82,179]
coords: [83,179]
coords: [84,179]
coords: [85,179]
coords: [86,179]
coords: [87,179]
coords: [88,179]
coords: [89,179]
coords: [90,179]
coords: [91,179]
coords: [92,179]
coords: [93,179]
coords: [94,179]
coords: [95,179]
coords: [96,179]
coords: [97,179]
coords: [98,179]
coords: [99,179]
coords: [100,179]
coords: [101,179]
coords: [102,179]
coords: [103,179]
af@macbookpro program %
```

I then assigned each coordinate position to it's respsecitve agent and placed the agent in the grid as shown earlier:

```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.width = 180
        self.height = 104

        #agents can't be within the same cell in the grid
        self.grid = mesa.space.SingleGrid(self.width, self.height, True)
        #defines scheduler that will activate all agents simultaenously at each time step
        self.schedule = mesa.time.SimultaneousActivation(self)

        #creates reference array of coordinates for agents to be added to
        refCoordinates = [[x,y] for y in range(0,self.height) for x in range(0,self.width)]

        #loop to place agents in grid and scheduler
        for d,i in enumerate(refCoordinates):
            a = PopulationAgent(d, self)

            #adds the agent to the scheduler
            self.schedule.add(a)
            #places agent within the grid
            self.grid.place_agent(a, (i[0],i[1]))
```

The next step was to have a way to link the agents and the squares in the grid together. To do this I decied to create a dictionary within the 'main.py' file that would use a second pair of coordinates to create an index out of coordinates which could be referenced to access both the square at that coordinate position as well as the agent. I decided to create this array as the canvas was drawn since I could add the agents in later while initially assigning each square to a coordinate position in the dictionary:

```
def draw_canvas(canvas) :
    #reference dictionary to link agent and square coords
    grid = {}
    #nested for loop for coordinates
    ix = -1
    iy = -1
    for y in range(3,900,5) :
        ix += 1
        iy = -1
        for x in range(3,520,5) :
            iy += 1
            #creates and packs squares
            rectangle = canvas.create_rectangle(y, x, y+5, x+5, fill="white")
            grid[ix,iy] = rectangle

    return grid
```

This now meant that the coordinate positions of each agent would link one-to-one with another agent with the same coordinates. Since the square grid and agent grid have the exact same number of coordinates each agent is linked to a specific square by their coordinates since both the agent and the square would have the same coordinates. This is useful since it now means each square is effectively an agent that can be manipulated e.g in this case by changing the square's colour based on the agent's state.

I then had to implement the colour functionality so that depending on the agent's cateogry the square grid would output the associated colour. To do this I randomly set each agent to be either Susceptible, Infected, Recovered or a fatality within the population model in 'population.py' as they are initialised. This meant a random amount of agents would be every colour, allowing me to see all the different colours throughout the grid:

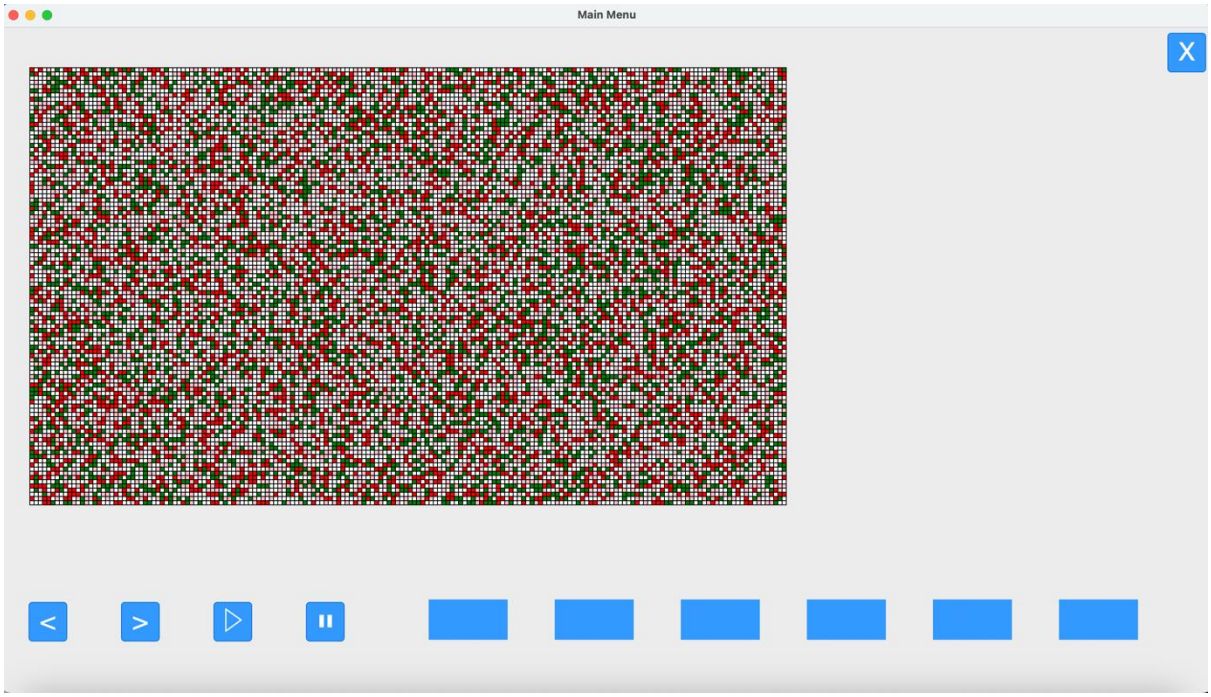
```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.width = 180
        self.height = 104

        #agents can't be within the same cell in the grid
        self.grid = mesa.space.SingleGrid(self.width, self.height, True)
        #defines scheduler that will activate all agents simultaenously at each time step
        self.schedule = mesa.time.SimultaneousActivation(self)

        #creates reference array of coordinates for agents to be added to
        refCoordinates = [[x,y] for y in range(0,self.height) for x in range(0,self.width)]

        #loop to place agents in grid and scheduler
        for d,i in enumerate(refCoordinates):
            a = PopulationAgent(d, self)
            match random.randint(5) :
                case 1 :
                    a.state = 'S'
                case 2 :
                    a.state = 'S'
                case 3 :
                    a.state = 'S'
                case 4 :
                    a.state = 'S'

            #adds the agent to the scheduler
            self.schedule.add(a)
            #places agent within the grid
            self.grid.place_agent(a, (i[0],i[1]))
```

I then created a function in 'main.py' that would categorise agents by assigning a certain colour to each square depending on what state the respective agent was in. Since the square was linked to the agent by their coordinate position, it meant that I could reference the agent's position in the squareGrid to find the associated square and change it's colour. Firstly I iterated through all the agents in the model then based on that agent's state it would reference the square with the same coordinates as the agent, using the grid of squares dictionary created earlier. Since they started off white if they were suscpetible there was no need to assign a colour since it was the default white colour that the canvas started off in:

```
def categorise_agents(model,canvas,squareGrid) :
    #iterates through agents in the modl
    for a in model.schedule.agents :
        if a.state == 'I' :
            #references agent coordiate position to find according square
            canvas.itemconfig(squareGrid[a.pos], fill='pink')
        elif a.state == 'R' :
            canvas.itemconfig(squareGrid[a.pos], fill='green')
        elif a.state == 'F' :
            canvas.itemconfig(squareGrid[a.pos], fill='red')
```

I then referenced this function to run in the FullSimulation page class leading to this result:

```
#creates reference square grid
self.squareGrid = draw_canvas(self.squareCanvas)
#draws grid
self.squareCanvas.pack()
#categorises agents
categorise_agents(self.model,self.squareCanvas,self.squareGrid)
```



This result showed that agents had been assigned categories randomly with each cateogry displaying a different colour, meaning that the agents and squares were successfully linked and each colour was distinct to a category, thus achieving the two goals of this iteration. It was time to move on to the next one.

Changes in Design Made in this Iteration

There are actually no changes to design in this iteration since it merely expands on all the already defined algorithms, merely linking it to the graphical side of the program by displaying colours and displaying the different agents states

How has this filled Success Criteria and User Requirements?

This iteration has fulfilled the following success criteria:

- 2D Grid Graphic

It has filled the user requirements by implemeting SIR colour as well as expanding on the concept of the population being represented by squares, with each square being an agent of the population that can be sorted into an SIR cateogry.

Iteration 4: Infection, Recovery and Fatality Algorithms and Travel Radius

This iteration focuses on implementing the SIR model algorithms in the step method, which will be activated at regular intervals. This algorithm will implement infection rates so that every agent that is infected in the simulation will use its list of neighbouring agents as potential candidates to be infected, then use a random chance to see if the agent in the neighbouring agent list will become an infected agent or not. It will also iterate through infected agents to determine, based on random chance, whether they will become a fatality or recovered.

The first algorithm I decided to code was the algorithm that would randomly infect susceptible agents. The algorithm is based off the ‘SIR Calculation Algorithm’ in the design plan, with the infection rate being defined by the user. Therefore, I created a function called infection_algorithm that would contain code to implement this algorithm in the model. The first step was to take the

input from the infection rate input box that the user inputs and pass it to this function. Firstly I created a new attribute in the FullSimulation class titled infectionRate and set it to 0.2:

```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.infectionRate = 0.2
```

The next step was to create a function that would actually carry out the infection algorithm. I decided to implement this algorithm in a temporary separate file, which could later be integrated into each agent's step() method, so that when the simulation runs one time-step this algorithm will be carried out with each agent.

Per my design plan for this algorithm this algorithm would iterate through each agent, then compare the infection rate with a random integer between 0.0 and 1.0. This meant I had to import python's random module:

```
from population import PopulationModel
import tkmacosx as tkmacosx
import tkinter as tk
import random
```

I then wrote the algorithm based off the 'SIR calculation' algorithm in the design plan using python's round module to get the random value to two decimal places. However I made it so that at the end of every iteration through all the agents, it would update the graphical user interface by calling the categorise_agents function, so it would update newly infected agent's respective colour to red:

```
def infection_algorithm(self,model,canvas,squareGrid) :
    #iterates through all agents in the model
    for a in model.schedule.agents :
        #if agent is suceptible
        if a.state == 'S' :
            #chance of infection
            randnum = (random.randint(0,100))
            if randnum <= self.infectionRate :
                neighbour.state = 'I'
    #updates canvas
    categorise_agents(model,canvas,squareGrid)
```

I then referenced this function in the initialisation of the 'Full Simulation' class so that it would run after everything else had already been initialised:

```
#creates reference square grid
self.squareGrid = draw_canvas(self.squareCanvas)
#draws grid
self.squareCanvas.pack()
#categorises agents
categorise_agents(self.model,self.squareCanvas,self.squareGrid)
#infects agents
infection_algorithm(self,model,self.squareCanvas,self.squareGrid)
```

However while trying to run this algorithm, I came across a problem. No matter how the infectionRate was altered, every time I ran the program the same number of agents were getting infected, meaning that the chance wasn't random. To solve this, I decided to print the random number and infectionRate to see how the algorithm was making comparisons between the two:

```
program — Python main_window.py — 80×24
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
0 is smaller than the infection rate of 0.2
```

It turned out that you cannot set random.randint to a decimal range and since it was set to be between 1 and 0, it always came out as zero, therefore every agent was becoming infected. To solve this I set the random integer range to 1-100 and then divided the result by 100 to produce a two decimal number result:

```python
randnum = (random.randint(0,100)/100)
```

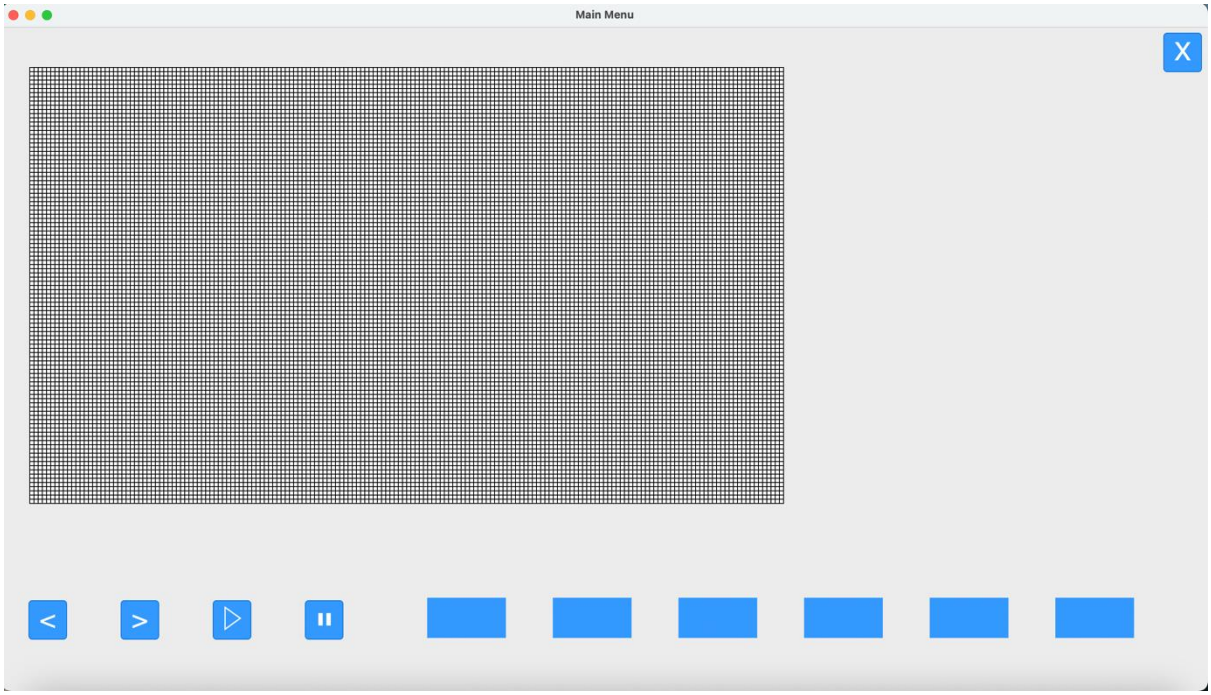
I then tested this with an infection rate of 0.8 showing that this change had fixed the error:

```
program — Python main_window.py — 80×24
0.25 is smaller than the infection rate of 0.8
0.4 is smaller than the infection rate of 0.8
0.28 is smaller than the infection rate of 0.8
0.01 is smaller than the infection rate of 0.8
0.45 is smaller than the infection rate of 0.8
0.6 is smaller than the infection rate of 0.8
0.58 is smaller than the infection rate of 0.8
0.33 is smaller than the infection rate of 0.8
0.67 is smaller than the infection rate of 0.8
0.66 is smaller than the infection rate of 0.8
0.09 is smaller than the infection rate of 0.8
0.21 is smaller than the infection rate of 0.8
0.38 is smaller than the infection rate of 0.8
0.27 is smaller than the infection rate of 0.8
0.32 is smaller than the infection rate of 0.8
0.29 is smaller than the infection rate of 0.8
0.8 is smaller than the infection rate of 0.8
0.69 is smaller than the infection rate of 0.8
0.0 is smaller than the infection rate of 0.8
0.24 is smaller than the infection rate of 0.8
0.76 is smaller than the infection rate of 0.8
0.7 is smaller than the infection rate of 0.8
0.53 is smaller than the infection rate of 0.8
```

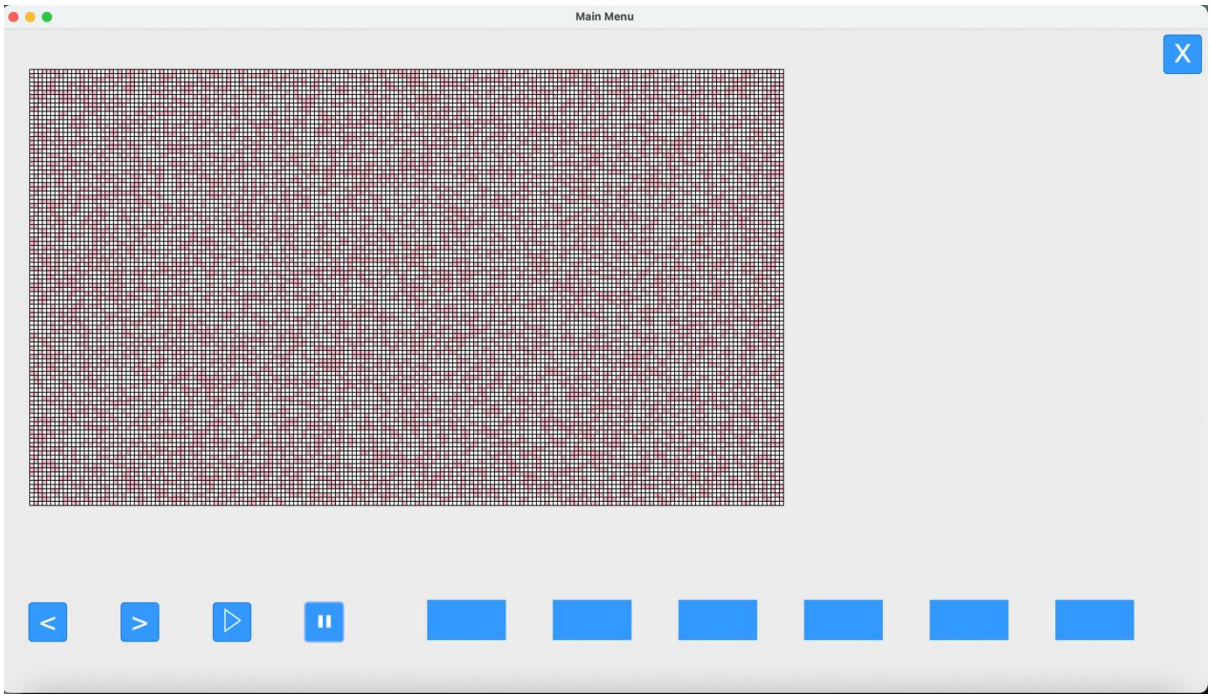
I then assigned this function to the pasue button defined within the 'FullSimulation' class as 'p2', so that when it was pressed the infection algorithm would run. This would show whether it worked graphically in the gird e.g the squares change colour to red if they become infected:

```
#temporarily assigns infection algorithm to pause button
self.p2.configure(command=lambda: infection_algorithm(self,model,self.squareCanvas,self.squareGrid))
```

This was the grid with all squares set so susceptible before the infection algorithm was run:



This was the grid after the pause button was pressed, thus running the infection algorithm (with a rate of 0.4):



This meant that the algorithm was working effectively, changing agents to infected based on random chance and then categorising them by calling the categorise_agents function to change their colour to pink.

The next step was to make it so that the algorithm only considered agents directly neighbouring an agent to be potential candidates to be infected using the neighbour_agents list attribute which contains a list of coordinates of directly neighbouring agents. The way to do this was that if the agent was infected, the algorithm would iterate through the neighbouring list, and use each coordinate in that list to retrieve the corresponding agent using mesa's get_cell_content method which references the model to get the agent at that coordinate position. This means that each neighbouring agent to an infected agent will be tested to see if they become infected:

```
def infection_algorithm(self,model,canvas,squareGrid) :
    #iterates through all agents in the model
    for a in model.schedule.agents :
        #if the agent is infected
        if a.state == 'I' :
            #iterates through agent list
            for b in a.neighbour_agents :
                #gets the agent at that coordinate position
                neighbour = model.grid.get_cell_list_contents([(b[0], b[1])])
                if neighbour.state == 'S' :
                    #chance of infection
                    randnum = (random.randint(0,100)/100)
                    if randnum <= self.infectionRate :
                        neighbour.state = 'I'
    #updates canvas
    categorise_agents(model,canvas,squareGrid)
```

However when I tried to run this an error appeared:

```
  File "/usr/local/lib/python3.11/site-packages/mesa/space.py", line 400, in <genexpr>
    if (cell := self._grid[x][y]) != self.default_val()
                ~~~~~~~~~~~~~~^^^
IndexError: list index out of range
```

This meant that the algorithm was trying to retrieve the contents of a coordinate position that does not exist. This meant there was an error when neighbouring coordinate positions were genereated in the 'population.py' file. I decided to print out the maximum and minimum coordinates that these neighbour coordinates were being generated between to see if they were causing the error:

```
for a in self.grid:
    #max and min x,y neighbour positions
    minxPos, maxxPos = a.pos[0]-self.travelRadius, a.pos[0]+self.travelRadius
    minyPos, maxyPos = a.pos[1]-self.travelRadius, a.pos[1]+self.travelRadius
    #coordinate validation
    if minxPos < 0 : minxPos = 0
    if maxxPos > self. width:
        maxxPos = self.width - 1
    if minyPos < 0 : minyPos = 0
    if maxyPos > self.height:
        maxyPos = self.height - 1

    print(f'min x coords: {minxPos},{minyPos}')
    print(f'mix y coords: {maxxPos},{maxyPos}')
```

```
mix y coords: 1,99
min x coords: 0,98
mix y coords: 1,100
min x coords: 0,99
mix y coords: 1,101
min x coords: 0,100
mix y coords: 1,102
min x coords: 0,101
mix y coords: 1,103
min x coords: 0,102
mix y coords: 1,104
Traceback (most recent call last):
  File "/Users/af/Library/CloudStorage/OneDrive-MidhurstRotherCollege/NEA/program/population.py", line 80, in
 <module>
```

It turns out that the maximum and minimum coordinate positions are out of range since the grid only goes up to 103 on the x-axis. To fix this I altered the validation so that the max x and y positions were compared with the height – 1 and width – 1; this meant that out of range coordinates were no longer added to neighbour_agents list:

```
if minxPos < 0 : minxPos = 0
if maxxPos > self. width -1:
    maxxPos = self.width - 1
if minyPos < 0 : minyPos = 0
if maxyPos > self.height -1:
    maxyPos = self.height - 1
```

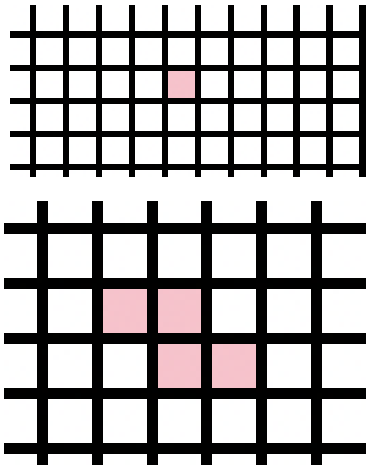
This fixed the program and it ran without error. I then ran into another error, this time with the infection algorithm function in 'main.py':

```
  File "/Users/af/Library/CloudStorage/OneDrive-MidhurstRotherCollege/NEA/progra
m/main_window.py", line 48, in infection_algorithm
    if neighbour.state == 'S' :
      ^^^^^^^^^^^^^^^
AttributeError: 'list' object has no attribute 'state'
```

For some reason it was saying that the agent retrieved from the coordinates in the neighbour_agent list didn't have a state attribute which didn't make sense since every agent was defined with a state attribute. To try and solve this I looked at how the get_cell_list_contents method worked and it turns out I was being a bit stupid since, as the name suggested, it returned a list of the agents in the cell, but since my program only had one agent per cell it was returning a list with one agent. To fix this I made it, so the neighbour was defined as the agent object in the list by using list slicing:

```
for b in a.neighbour_agents :
    #gets the agent at that coordinate position
    neighbour = model.grid.get_cell_list_contents([(b[0], b[1])])[0]
```

Once I had done this the error was fixed. I then changed the agent initialisation in 'population.py' so that one singular agent was defined as infected, to make sure that the algorithm would only run within the neighbouring agents of that singular agent (I've zoomed in on the grid a bit) as well as making the infection rate 0.1 :



This now meant that the infection algorithm was complete. The next step was to move this algorithm to the step method in 'population.py' so that when the step method of the model was called, this algorithm would run for every infected agent in the simulation simultaneously. This was pretty hassle-free with only a few modifications needing to be made to the infection algorithm:

```
class PopulationAgent(mesa.Agent) :
    def __init__(self,unique_id,model):
        super().__init__(unique_id,model)
        self.state = 'S'
        self.neighbour_agents = []

    def step(self):
        #infection algorithm
        #if the agent is infected
        if self.state == 'I' :
            #iterates through agent list
            for b in self.neighbour_agents :
                #gets the agent at that coordinate position
                neighbour = self.model.grid.get_cell_list_contents([(b[0], b[1])])[0]
                if neighbour.state == 'S' :
                    #chance of infection
                    randnum = (random.randint(0,100)/100)
                    if randnum <= self.infectionRate :
                        neighbour.state = 'I'
```

I then removed the function from 'main_window.py' and removed the association with the function from the pause button. I then called the step method at the end of the FullSimulation class as well as the categorise_agents to update the colour changes. However I ran across an error:

```
AttributeError: 'PopulationAgent' object has no attribute 'infectionRate'
```

Since I had defined this accidentally as an attribute of the FullSimulation class in 'main.py' instead of the PopulationModel class in 'population.py' it could not access this attribute. The way to fix this was to make the infectionRate an attribute of the PopulationModel class instead of the FullSimulation class. Firstly, I assigned the infectionRate attribute to the PopulationModel class in 'population.py' with an initial value of 0.1:

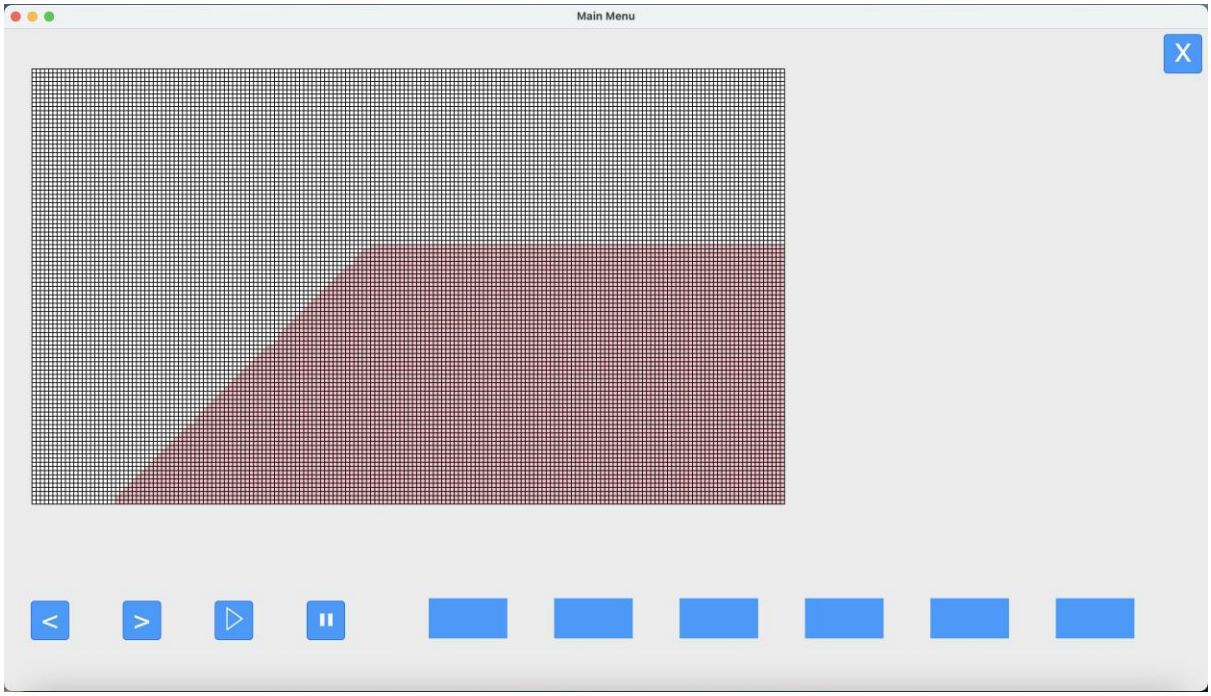
```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.infectionRate = 0.1
        self.width = 180
        self.height = 104
```

I then called the step method five times within the FullSimulation class to see how the infection algorithm would change the population when run five times. I added the cateogrise_agents function so that the colour changes were updated and I also added the tkinter method window.update() which would update the GUI once every loop, so I could see the colour changes in real-time:

```
#creates reference square grid
self.squareGrid = draw_canvas(self.squareCanvas)
#draws grid
self.squareCanvas.pack()

for i in range(5) :
    self.model.step()
    categorise_agents(self.model,self.squareCanvas,self.squareGrid)
    window.update()
```

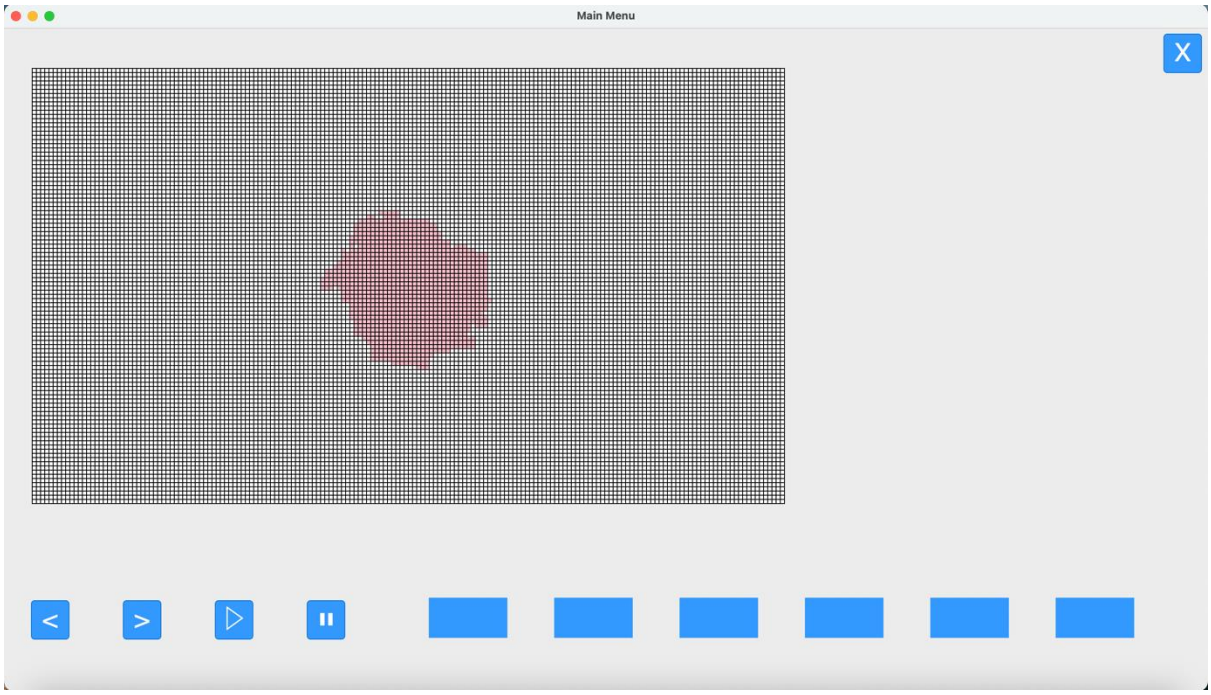
This shows that it worked and the simulation was now running with the infection algorithm being run simultaneously for each agent when the step method of the model was called. However, I ran into another issue.

When the infection rate was set to a high value such as 0.9, the result looked like this:



This was obviously not what a random disease spread looks like at so I investigated to see what the problem was. Although I couldn't pinpoint the exact cause of the problem I decided to switch the scheduler mode to activate each agent randomly which lead to a much more reasonable result:

```
self.width = 180
self.height = 104

#agents can't be within the same cell in the grid
self.grid = mesa.space.SingleGrid(self.width, self.height, True)
#defines scheduler that will activate all agents randomly at each time step
self.schedule = mesa.time.RandomActivation(self)
```



The implementation of the infection algorithm was now complete. It was time to work on the next step which was implementing the recovery algorithm.

The first thing I did was to add another attribute to the model named recoveryRate:

```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.infectionRate = 0.9
        self.recoveryRate = 0.1
        self.width = 180
        self.height = 104
```

This algorithm would be implemented in the model's step method since it only needs to be run once per time-step instead of each agent running it in their step method, like in the infection algorithm. To start with I created it like the infection algorithm, except it iterated through the population and changed the agent's state to 'R' :

```
def step(self):
    #iterates through population
    for a in self.grid:
        #recovery algorithm
        if a.state == 'I' :
            #chance of recovery
            randnum = (random.randint(1,100)/100)
            if randnum <= self.recoveryRate :
                a.state = 'R'
    self.schedule.step()
```

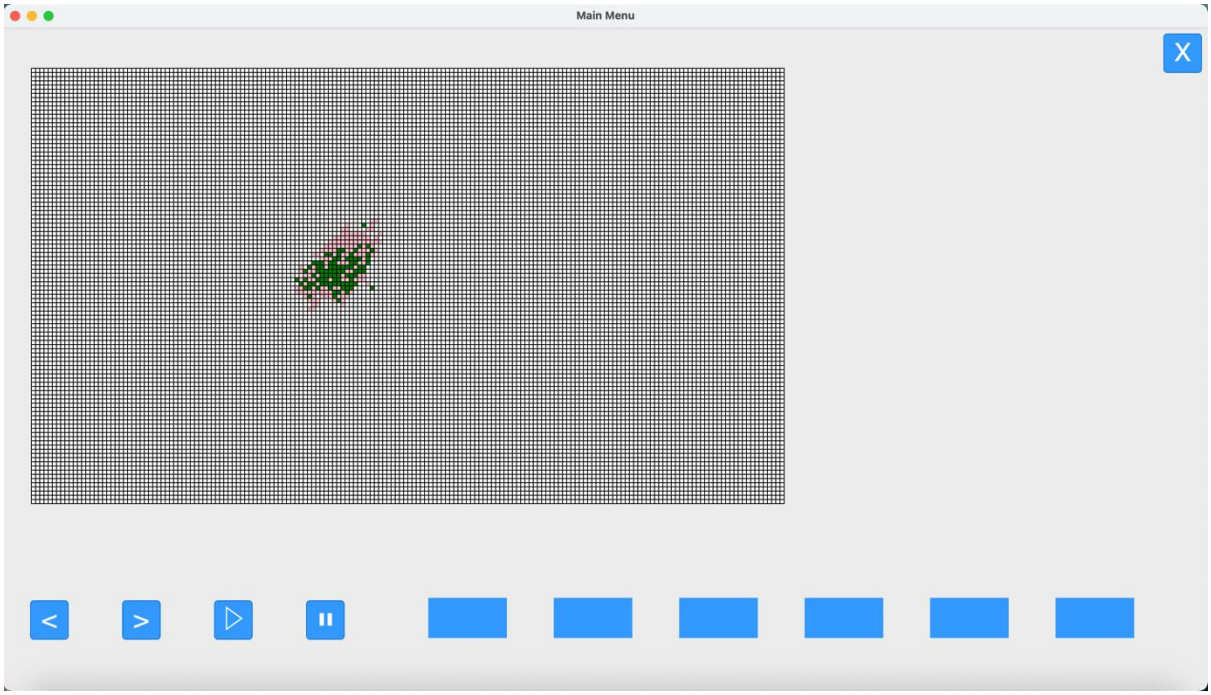
Since both the recovery and infection algorithms had both been moved to the model's and agent's step methods respectively this meant I could call both of them in a single time-step by calling the model's step() method which automatically calls whatever is in it as well as the agent's step() method at the end. The whole execution process therefore was only made up of calling the step method of the model, and then calling the categorise_agents function to change the colours of each agent on the grod. Therefore I decided to consolidate this into one function called time_step :

```
def time_step(model,canvas,squares) :
    #calls the model and agent steps methods
    model.step()
    #changes grid colour
    categorise_agents(model,canvas,squares)
    #updates GUI
    window.update()
```

And called it at the end of the FullSimulation class :

```
#draws grid
self.squareCanvas.pack()
for i in range(10) :
    #does one time-step of the simulation
    time_step(self.model,self.squareCanvas,self.squareGrid)
```

With the recovery algorithm implemented as well, this was the result:



This shows that the algorithm worked and is implemented properly, however in this example the recovery-rate was set to 0.1, yet the result was much higher than I feel warrants for a ten-percent chance of recovery so the way this algorithm generates the chance of a agent becoming infected may change in the future.

The next step was to implement the fatality algorithm, which was just adding a fatalityRate parameter, modifying the recovery rate algorithm to accept this parameter in place of the recoveryRate, and then placing it in the step() method of the model in 'population.py'. I then altered the recovery rate algorithm to accommodate checking for the fatality rate as well, and if so changing the agent's state to a fatality (I also altered the random integer to a higher degree of accuracy):

```
def step(self):
    #iterates through population
    for a in self.grid:
        #recovery algorithm
        if a.state == 'I' :
            #chance of recovery
            randnum = (random.randint(1,10000)/10000)
            #avoids bias towards fatality or recovery by 50/50 chance
            #chance of recovery
            if randnum <= self.recoveryRate :
                a.state = 'R'
            else:
                #chanve of fatality
                if a.state != 'R' and randnum <= self.fatalityRate :
                    a.state = 'F'
```

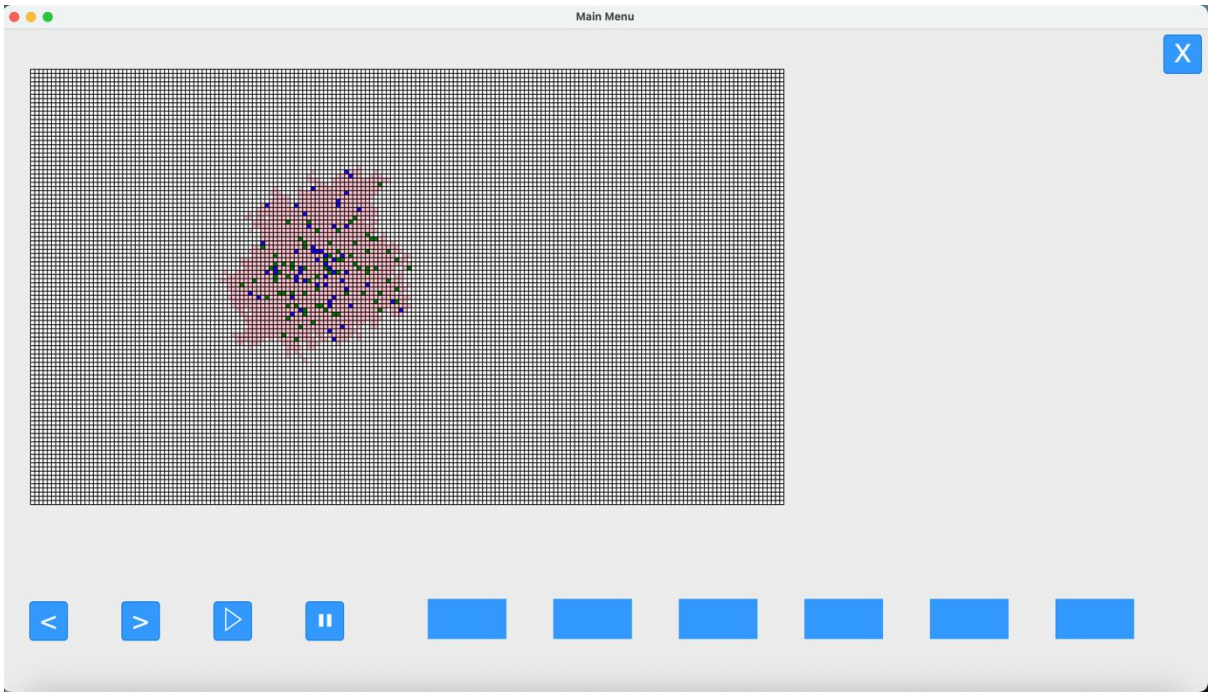
However there was a problem with this, since the recovery rate was checked first there is a bias towards the fatality rate, with more members of the population becoming fatalities than recovered. To fix this I added another if statement that would add an element of randomness, so there's a fifty-percent chance that the fatality rate will be checked first then the recovery rate and then a fifty-percent chance that they get checked the other way round:

```
for a in self.grid:
    #recovery algorithm
    if a.state == 'I' :
        #chance of recovery
        randnum = (random.randint(1,10000)/10000)
        #avoids bias towards fatality or recovery by 50/50 chance
        if random.randint(0,9) < 5 :
            #chance of recovery
            if randnum <= self.recoveryRate :
                a.state = 'R'
            else:
                #chanve of fatality
                if a.state != 'R' and randnum <= self.fatalityRate :
                    a.state = 'F'
        #flipped so in this case fatality gets cehcked first
        else :
            if a.state != 'R' and randnum <= self.fatalityRate :
                    a.state = 'F'
            else :
                if randnum <= self.recoveryRate :
                    a.state = 'R'
```

I then called the model's step method at the end of the FullSimulation class with a infectionRate of 0.1 and fatalityRate of 0.01, recoveryRate of 0.01:

```
#creates reference square grid
self.squareGrid = draw_canvas(self.squareCanvas)
#draws grid
self.squareCanvas.pack()
for i in range(40) :
    #does one time-step of the simulation
    time_step(self.model,self.squareCanvas,self.squareGrid)
```

Leading to this result:



This shows that all algorithms are now working together in synchronisation with the squares changing colour accordingly to show the random spread. This now means this iteration is complete. Since I have now completed the simulation functionality it meant that I could traverse back to iteration 1 and implement the button functionalities.

Changes in Design Made in this Iteration

There are no changes to the fundamental structure of these algorithms outlined in the design section with the only difference being that some of the variable names and references of these algorithms have been changed to fit within the program code.

How has this filled Success Criteria and User Requirements?

This iteration has fulfilled the following success criteria:

- Disease Model

It has created a disease model that moves through the population by implementing a set of algorithms that model each characteristic of a disease and allow it to move through the population.

Iteration 1: Menus, Boxes and Buttons pt. 2

Now that the simulation was in place, I could add the final functionalities outlined in this iteration, namely adding the functionality for the arrow buttons to move between time-steps, and adding the functionality for the play button and pause button to start and stop the simulation accordingly.

The first function I decided to implement was the ability for when the user pressed the play button, that the simulation will start playing time-steps one after another until either the end of the simulation is reached, or the user presses the pause button. Until I implement the pause button functionality, when the play button is pressed, the simulation will run until it reaches its end, e.g. all the squares are filled.

The first part of the function that I decided to implement was the detection that the simulation has ended. To do this I decided that the user would have to enter a time for how many time-steps they wanted the simulation to execute. For the moment I decided to assign this input to the first input box by the simulation (temporarily). Firstly I created the new attribute timeStepsNum and decided to set it at a default one-hundred :

```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.infectionRate = 0.9
        self.recoveryRate = 0.5
        self.fatalityRate = 0.5
        self.timeStepsNum = 100
        self.width = 180
        self.height = 104
```

I then assigned the user input for this attribute for the first input box :

```
def retrieve_input(model,inputText,inputBox):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    paramaterValue = inputText.get("1.0",tk.END)
    #deletes text from input box so isnt added on next time function is run
    inputText.delete("1.0",tk.END)
    #validates whether the value is an integer or float
    try:
        paramaterValue = int(paramaterValue)
    except ValueError:
        paramaterValue = float(paramaterValue)

    #input validation as outlined via test plan
    match inputBox :
        case 1 :
            if paramaterValue > 0 and paramaterValue < 5001 and type(paramaterValue) == int:
                model.timeStepsNum = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
```

A handy attribute that the model has by default, since it was implemented using the python mesa module, is model.schedule.time. This attribute counts the current number of time-steps as the simulation executes. This was very useful since I could compare it against the user-inputted time-steps value and if it exceeded it, then the simulation could be stopped. I then implemented this within the FullSimulation class:

```
#creates reference square grid
self.squareGrid = draw_canvas(self.squareCanvas)
#draws grid
self.squareCanvas.pack()

#while time-steps do not exceed user set limit
while model.schedule.time < model.timeStepsNum :
    #does one time-step of the simulation
    time_step(self.model,self.squareCanvas,self.squareGrid)
```

I then set the timeStepsNum attribute to ten and implemented it so that the simulation would print out the current time-step it was on to see if it would adhere to the user limit:

```
#while time-steps do not exceed user set limit
while model.schedule.time < model.timeStepsNum :
    #does one time-step of the simulation
    time_step(self.model,self.squareCanvas,self.squareGrid)
    print(f'The current time_step is {model.schedule.time} while the user limit is {model.timeStepsNum}')
```

Leading to this result when the simulation was run:

```
af@macbookpro program % python3 main_window.py
The current time_step is 1 while the user limit is 10
The current time_step is 2 while the user limit is 10
The current time_step is 3 while the user limit is 10
The current time_step is 4 while the user limit is 10
The current time_step is 5 while the user limit is 10
The current time_step is 6 while the user limit is 10
The current time_step is 7 while the user limit is 10
The current time_step is 8 while the user limit is 10
The current time_step is 9 while the user limit is 10
The current time_step is 10 while the user limit is 10
```

This shows that the simulation only ran the amount that the user inputted so I had implemented the ability to tell when the simulation had finished. The next step was to implement the play button functionality so that when pressed the simulation had run until it reached it's end (user-defined limit).

To do this I moved the code to while loop into the time-step function:

```
def time_step(model,canvas,squares) :
    #while time-steps do not exceed user set limit
    while model.schedule.time < model.timeStepsNum :
        #calls the model and agent steps methods
        model.step()
        #changes grid colour
        categorise_agents(model,canvas,squares)
        #updates GUI
        window.update()
```

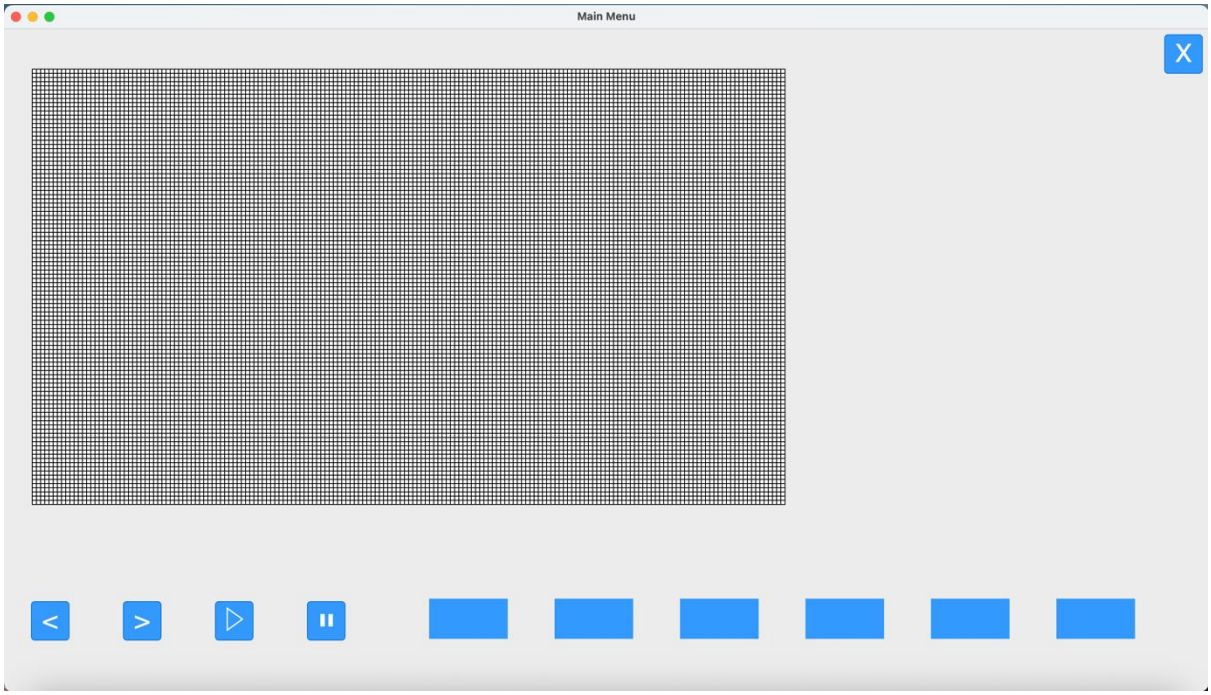
I then assigned this function to the play button:

```
self.p1.pack(anchor='s',side='left',pady=60,padx=30)
self.p2.pack(anchor='s',side='left',pady=60,padx=30)

#assigns time-step function to play button
self.p1.configure(command=lambda: time_step(model,self.squareCanvas,self.squareGrid))
```

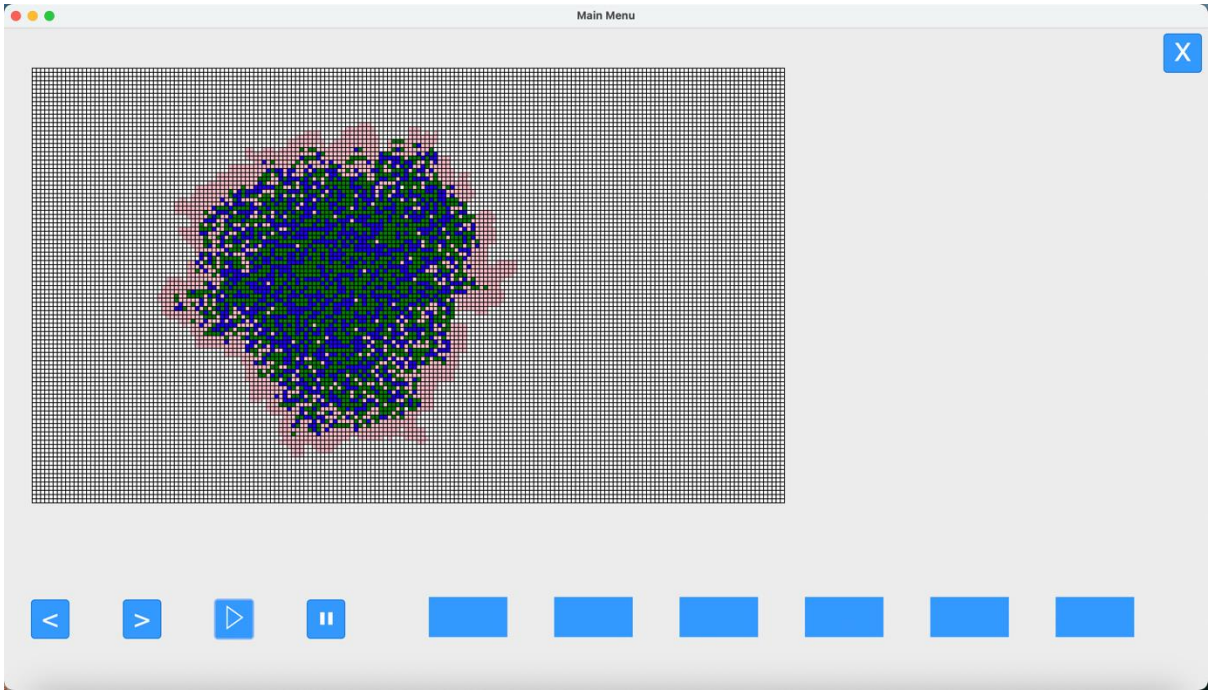
This meant that when the play button the simulation should run with the following attributes:

```
self.infectionRate = 0.9
self.recoveryRate = 0.5
self.fatalityRate = 0.5
self.timeStepsNum = 10
```

This was the simulation before the play button was pressed:



And this was the result afterwards:



This shows that the play button functionality had been added and the simulation ran when it was pressed. It also only ran for 10 time-steps, so the user time-limit has also been implemented properly.

The next step was to implement the pause button functionality. This was where if the pause button was pressed, then the simulation would stop at its current time step. The way I did this was by defining a new attribute in the FullSimulation class titled 'pause':

```
class FullSimulation :
    def __init__(self,model) :
        #assigns model
        self.model = model
        #pause initially set to false
        self.pause = False
```

I then added the self-parameter to the time-step function so that it could reference this attribute and altered the while loop so the function would only run while this attribute was set to False:

```
def time_step(self,model,canvas,squares) :
    #when time-step called pause set to false so simulation starts
    self.pause = False
    #while time-steps do not exceed user set limit and pause is false
    while model.schedule.time < model.timeStepsNum and self.pause == False:
        #calls the model and agent steps methods
        model.step()
        #changes grid colour
        categorise_agents(model,canvas,squares)
        #updates GUI
        window.update()
```

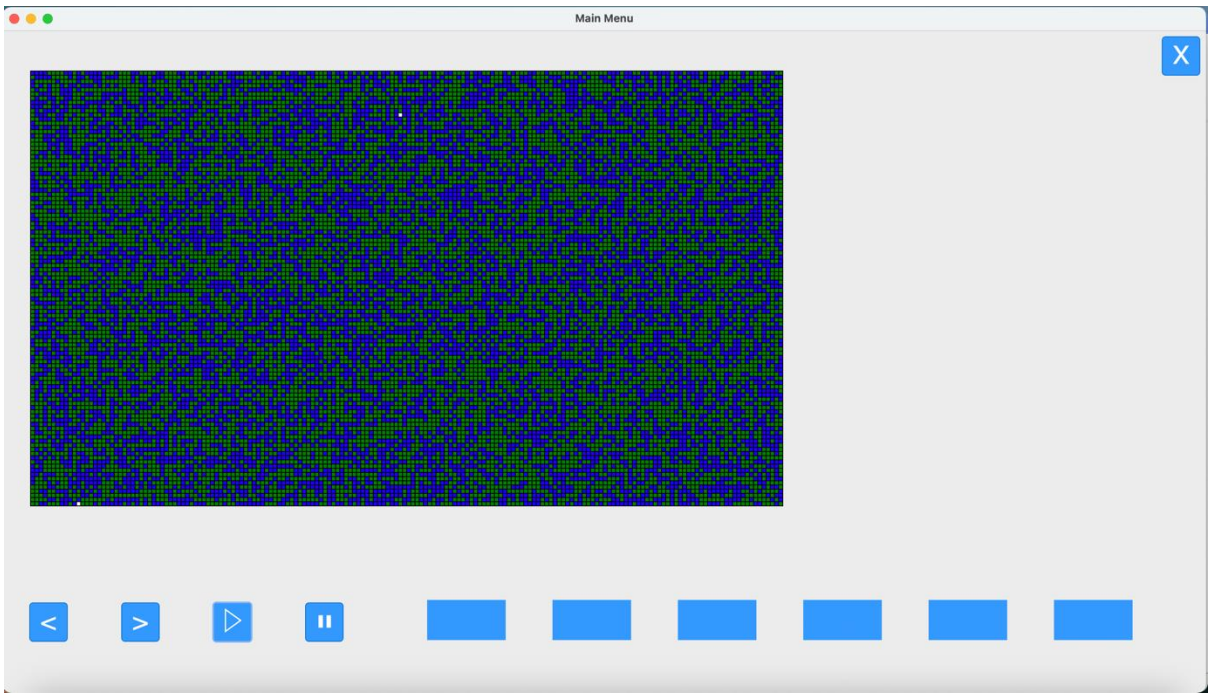
I then creaed another function titled stop, with self as a parameter, that when called would reference the pause attribute and set it to True, thus stopping time-steps while-loop:

```
def stop(self) :
    self.pause = True
```

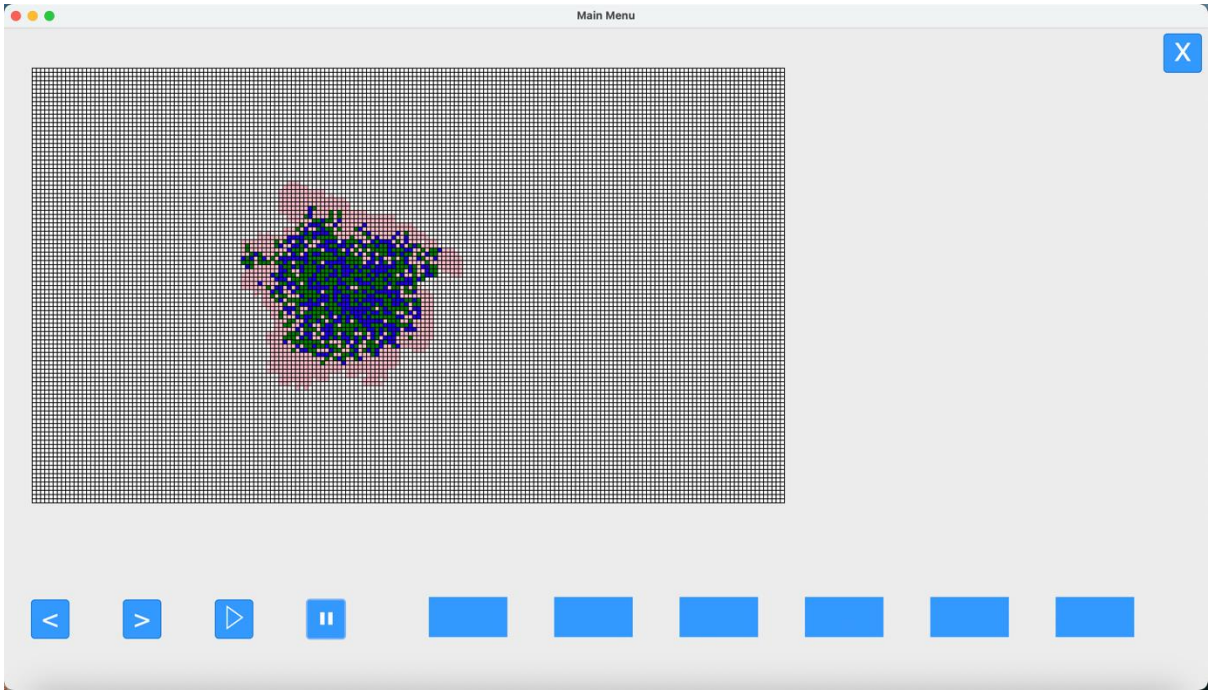
I then assigned this function to the pause button :

```
#assigns time-step function to play button
self.p1.configure(command=lambda: time_step(self,model,self.squareCanvas,self.squareGrid))
#assigns pause function to pause button
self.p2.configure(command=lambda: stop(self))
```

So this was the result of the simulation running normally without interference and user value of 100 time-steps with the previous attributes :



And this was the result after the pause button had been pressed mid-way through:



This showed that the pause button worked at pausing the simulation, and the simulation could be continued if need be, by pressing the play button, since that calls the time-step function where the pause button is once again set to False, allowing the loop in time-step to run.

The next step was to implement the functionality of the arrow buttons, so that each one would move the simulation back one time-step and forward one time-step respectively. The way that this would work is that whenever either is pressed, the simulation is paused and will move either backwards or forwards a time-step depending on the button pressed, with each button push corresponding to one time-step in either direction. However, the simulation will not continue unless the user presses the play button again.

The first thing I decided to implement was the ability to press on the backwards arrow and move the simulation backwards a single time-step. The way I decided that I would do this is to create a new attribute for each agent called 'memory' in the 'population.py' file that would store each state of the agent at each time-step of the simulation as it runs:

```
class PopulationAgent(mesa.Agent) :
    def __init__(self,unique_id,model):
        super().__init__(unique_id,model)
        self.state = 'S'
        self.neighbour_agents = []
        self.memory = {}
```

Then, since each agent's step method was executed once per time step, I made it so that each time-step the agent's state would be recorded in the dictionary at that current time-step, with the time-step number being the key in the dictionary using the model.schedule.time attribute mentioned earlier. This meant every agent would have a list of their own states at each respective time-step as an attribute:

```
def step(self):
    #iterates through population
    #counts number of suceptible agents
    for a in self.grid:
        #assigns state to time-step number in dictionary
        a.memory[self.schedule.time] = a .state
        if a.state == 'I' :
```

I then created a new function called backwards_time_step that when called would move the simulation back by one time-step and using the categorise_agents function to update the colours within the simulation display. The way that it did this was by subtracting one from the model.schedule.time attribute.

It then referenced the previous state of each agent in the grid using the newly subtracted model.scheudle.time attribute as the key

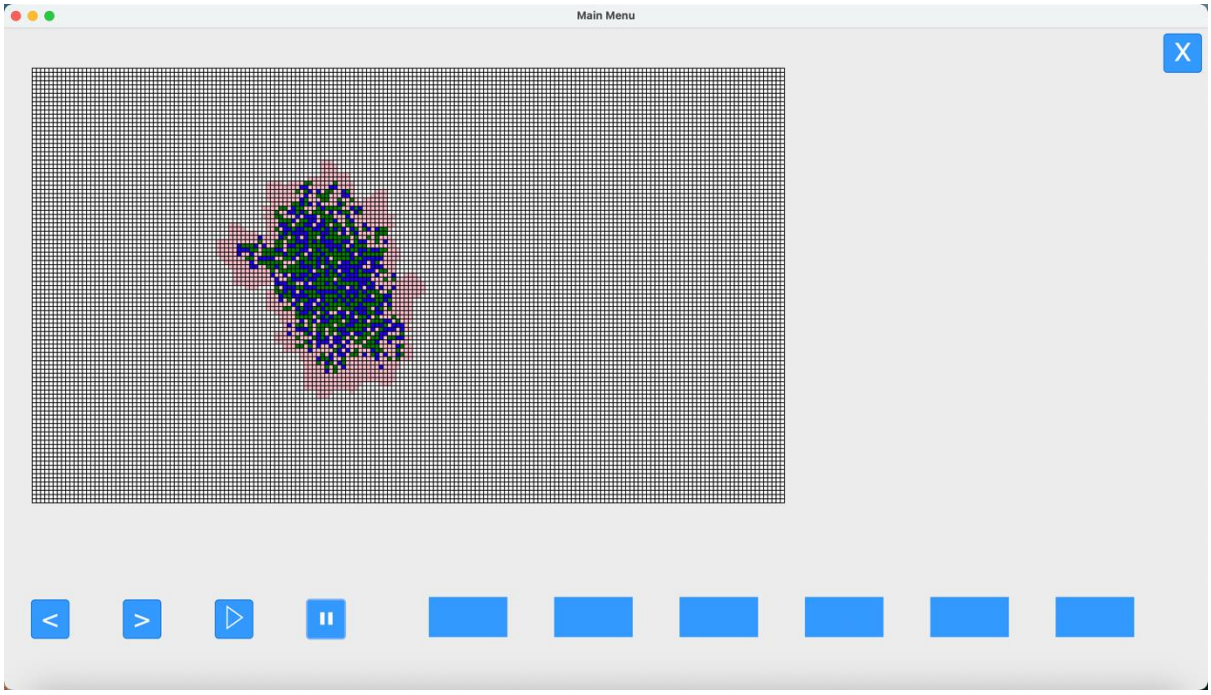
The simulation needed to be paused using the stop() function, otherwise it would continue running once the backwards time-step had been executed, not allowing the user to look at the previous stage:

```
def time_step_backwards(self,model,canvas,squares) :
    #means that the time-step time will be set to the previous
    self.model.schedule.time -= 1
    #pauses the model
    stop(model)
    for i in model.grid:
        #agent state will be set to previous time-step state
        i.state = i.memory[self.model.schedule.time]
    #updates colours
    categorise_agents(model,canvas,squares)
    window.update()
```

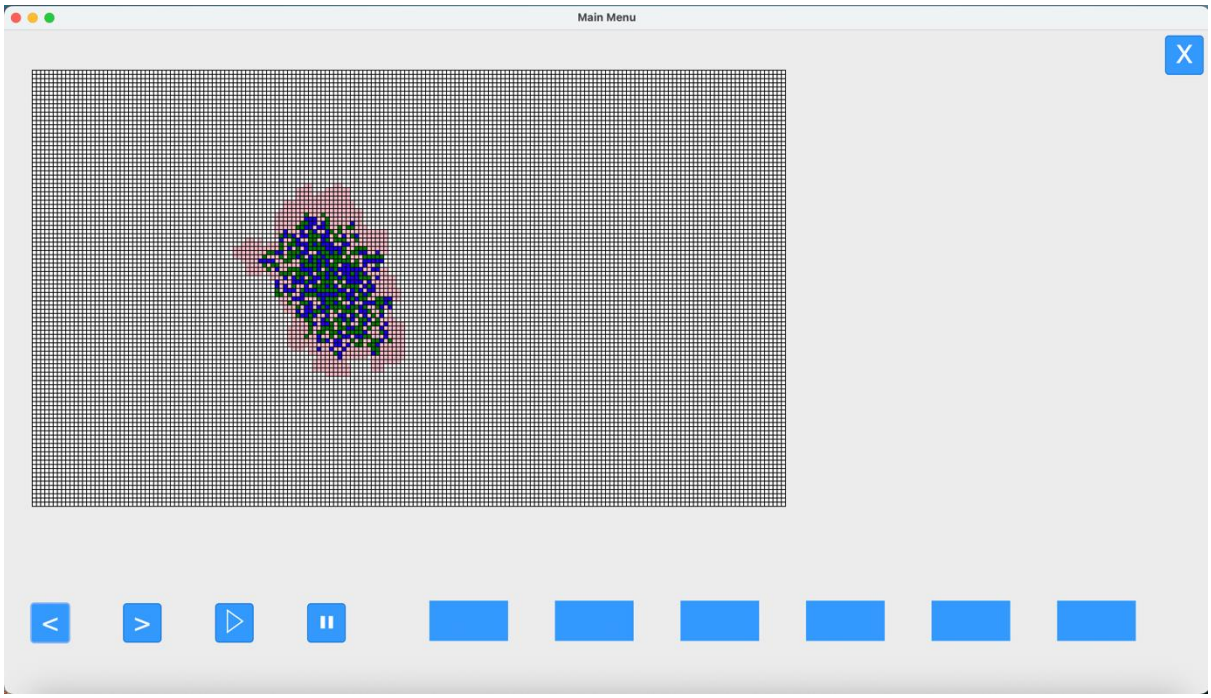
I then assigned this function to the backwards arrow button:

```
#assigns backwards time-step function to previous arrow button
self.t1.configure(command=lambda: time_step_backwards(self,model,self.squareCanvas,self.squareGrid))
```

This was the simulation before the backwards arrow was pressed:



And this was the simulation after the backwards arrow button had been pressed:



This shows that the simulation had been moved back by one time-step, so that meant the function was working properly and was also assigned to the backwards arrow button so I had finished implementing the backwards time-step feature and the next step was to work on the forward arrow button to move the simulation forwards one time-step.

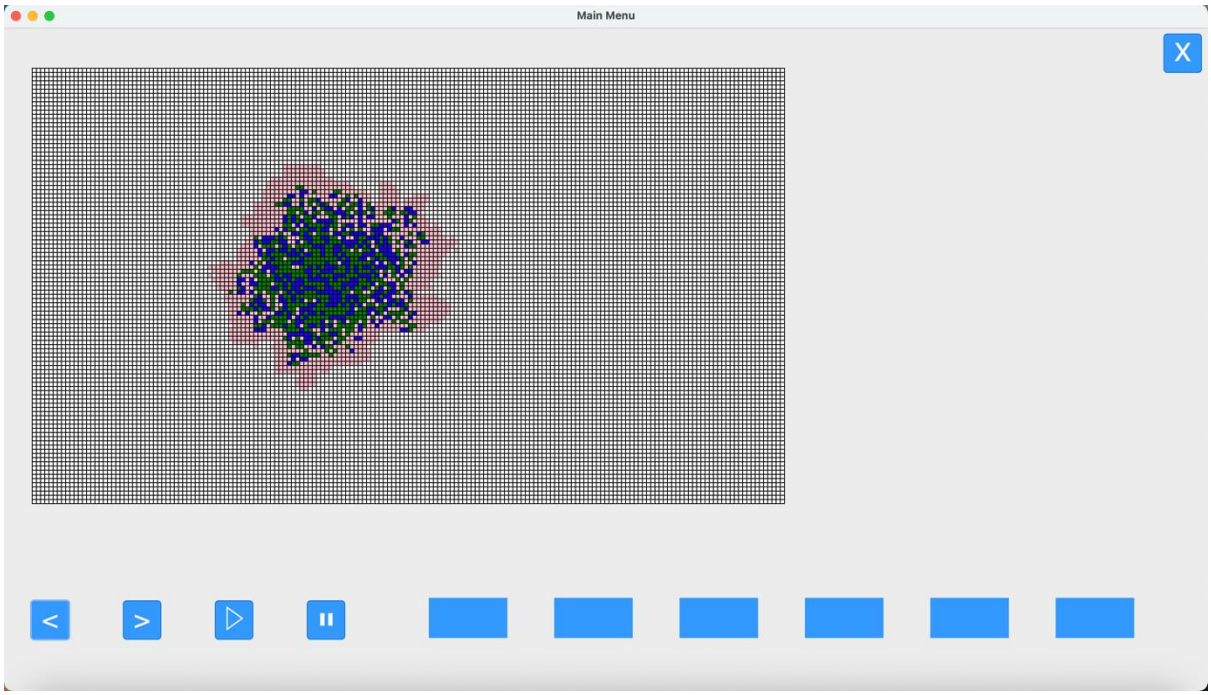
The way that I did this was by simply copied over the backward_time_step function but reversed it so it would add one to the model.schedule.time attribute, and look for the agent states ahead instead in the memory dictionary instead of behind:

```
def forward_time_step(self,model,canvas,squares) :
    #pauses model
    stop(model)
    self.model.schedule.time += 1
    for i in model.grid :
        #gets state of agents one ahead
        i.state = i.memory[self.model.schedule.time]
    #changes colours
    categorise_agents(model,canvas,squares)
    window.update()
```

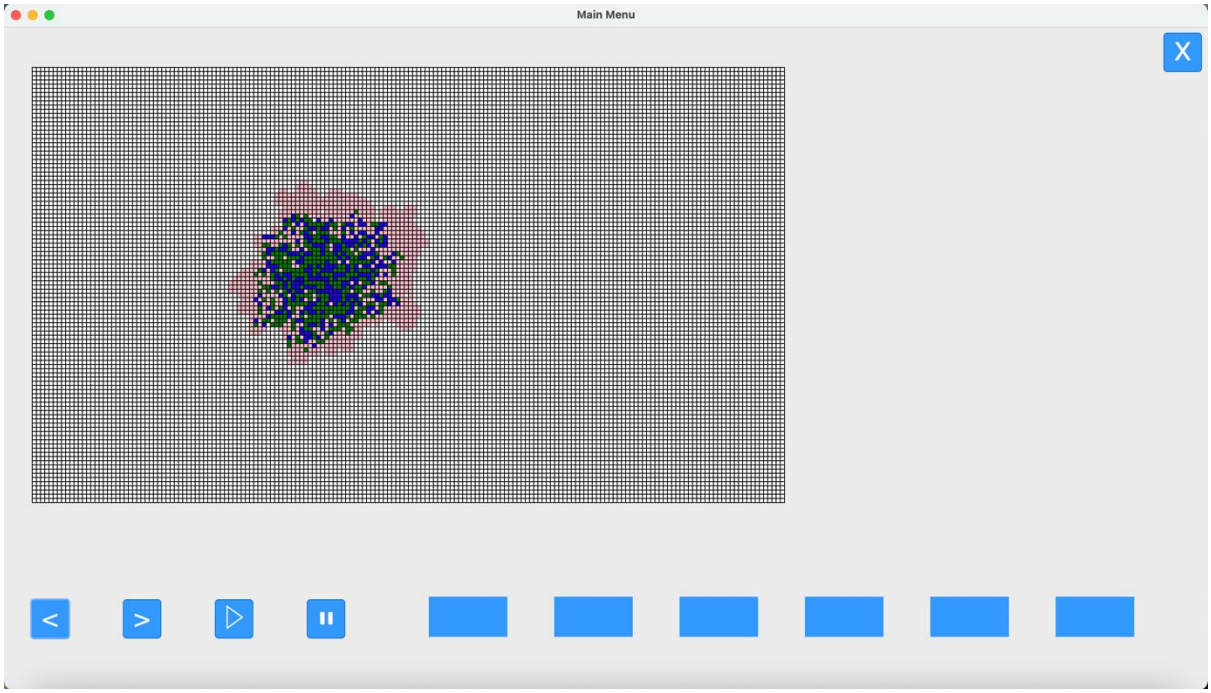
I then assigned this function to the forwards arrow button:

```
#assigns backwards time-step function to previous arrow button
self.t1.configure(command=lambda: backward_time_step(self,model,self.squareCanvas,self.squareGrid))
#assings forward time-step function to forward arrow button
self.t2.configure(command=lambda: forward_time_step(self,model,self.squareCanvas,self.squareGrid))
```

This was the simulation moved backwards one time-step from its most recent point:



And this was the simulation after the forward arrow button was pressed:



This showed that the function was working properly and could move the simulation forwardss by one time-step. Howver I ran into an issue, when I wanted to move one time-step forwards from where the most recent time-step in the simulation was, I encountered an error:

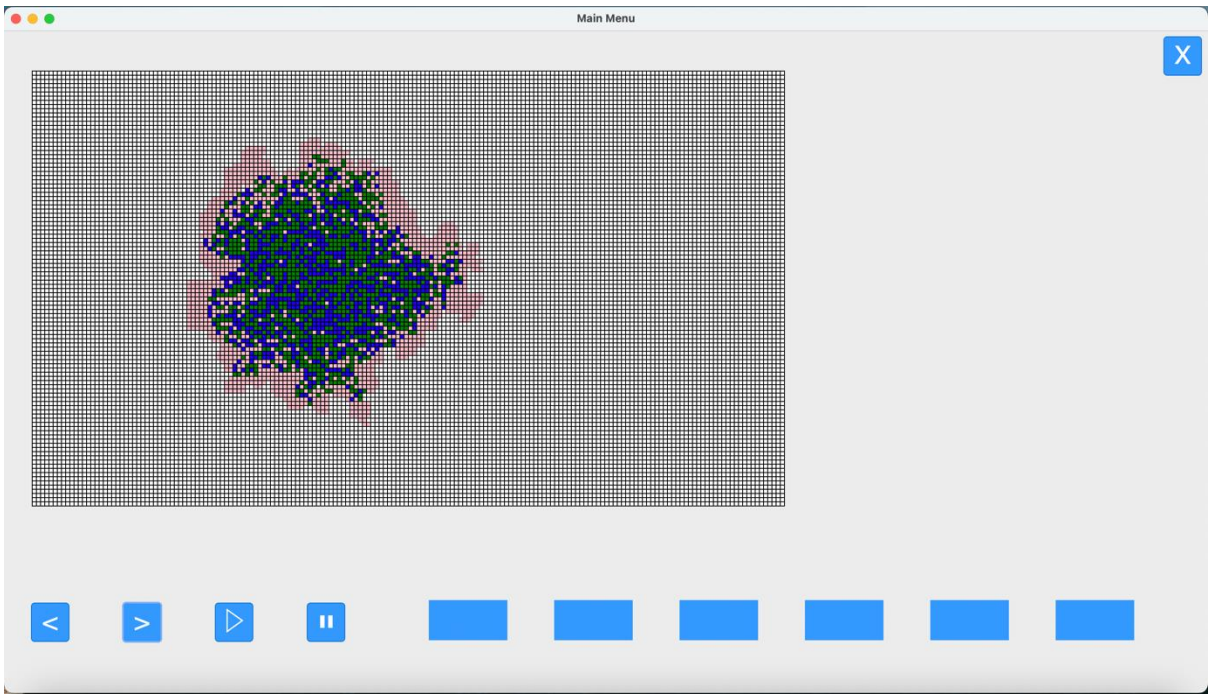
```
  File "/Users/af/Library/CloudStorage/OneDrive-MidhurstRotherCollege/NEA/progra
m/main_window.py", line 141, in forward_time_step
    i.state = i.memory[self.model.schedule.time]
              ~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyError: 5
```

The reason for this error was that there wasn't an entry in the memory database for the state of the agent after the current time-step since every new time-step was generated randomly using the time_step() function. The solution to this problem was to add a conditional statement that would check whether the model.schedule.time was smaller than the length of the memory dictionary. This meant if the model.scheudle.time attribute was up to date then it would be equal to the number of entries in the memory array and instead of going back to retrieve the previous position of each agent, it would generate a new random time-step, as it usually did when the play button was

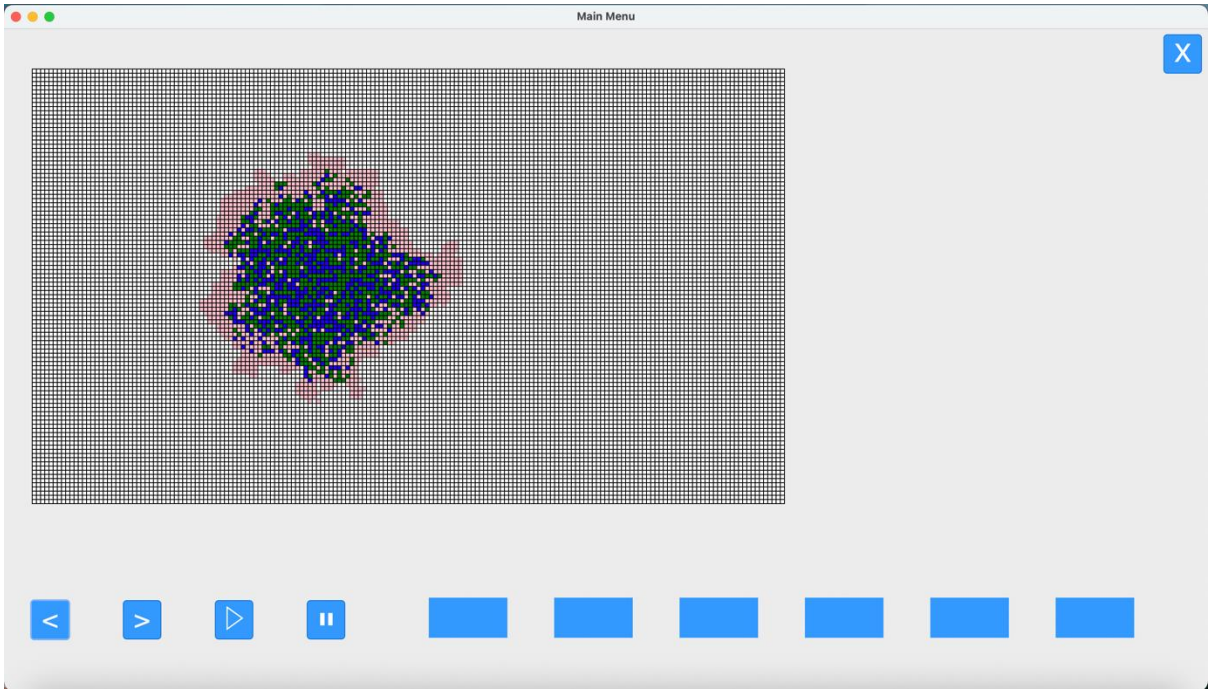
pressed:

```
def forward_time_step(self,model,canvas,squares) :
    #if model is up to date
    if self.model.schedule.time == len(model.grid[0][0].memory) :
        model.step()
        #changes colours
        categorise_agents(model,canvas,squares)
        window.update()
    #if model is outdated
    else :
        #pauses model
        stop(model)
        self.model.schedule.time += 1
        for i in model.grid :
            #gets state of agents one ahead
            i.state = i.memory[self.model.schedule.time]
        #changes colours
        categorise_agents(model,canvas,squares)
        window.update()
```

This then fixed the issue. This was the simulation caught up to it's most recent time-step:



This was after the forward arrow button was pressed:



As you can see it now works and generates a completely new time-step, independent of the memory dictionary. This meant that the forward-arrow function had now been implemented properly.

However there was another issue, whenever I used either the forwards or backwards arrow buttons and then subsequently pressed the play button, the simulation wouldn't continuously run when in a normal scenario when the user presses the play button the simulation should run continuously. Since the time_step function was the function assigned to the play button, I decided to alter it a bit to take account for the different states that the simulation may be in, meaning that no matter what if the simulation is up to date, or out of date due to the user using the arrow buttons, the play button can account for this and run the simulation continuously:

```
def time_loop(model,canvas,squares) :
    #when time-step called pause set to false so simulation starts
    model.pause = False
    #while time-steps do not exceed user set limit (while model is out of date) and pause is false
    while model.schedule.time != len(model.grid[0][0].memory) :
        for i in model.grid :
            i.state = i.memory[model.schedule.time]
        categorise_agents(model,canvas,squares)
        window.update()
        model.schedule.time += 1
    #while the model is up to date and not paused
    while model.schedule.time < model.timeStepsNum and model.pause == False:
        #calls the model and agent steps methods
        model.step()
        #changes grid colour
        categorise_agents(model,canvas,squares)
        #updates GUI
        window.update()
```

I renamed the function to time-loop and instead of using conditional statements like in the arrow button functions, I used while loops which meant that if the simulation was out of date it would iterate through each state in the memory dictionary (changing state colour accordingly) and then once it was up to date, then it would move on to the second loop which is generating new time-steps until the pause button is pressed or the the simulation exceeds the user time limit. This meant that the simulation as well as button functionality was complete.

The next step was to add functionality to each of the input boxes so that when the user entered a value, the corresponding simulation attribute e.g 'Infection Rate' will change to this user inputted value. To do this I had to change the retrieve_input function. Since the validation was already in place, the only thing I had to change was that for each case, when the paramater value had passed the validation checks, it would be assigned to it's corresponding attribute. However before I did this I made the travel radius an attribute of the model instead of a paramater of the model so the model in the main_menu.py file went from being defined as this:

```
class MainMenu :
    #initialises the objects on the page
    def __init__(self) :
        #assigns the model
        self.model = PopulationModel(1)
```

To this:

```
class MainMenu :
    #initialises the objects on the page
    def __init__(self) :
        #assigns the model
        self.model = PopulationModel()
```

While the travelRadius attribute in the population.py file wentn from being defined as this:

```
class PopulationModel(mesa.Model):
    def __init__(self, travelRadius):
        self.travelRadius = travelRadius
        self.infectionRate = 0.2
        self.recoveryRate = 0.5
        self.fatalityRate = 0.5
        self.timeStepsNum = 100
        self.width = 180
        self.height = 104
        self.pause = False

        #agents can't be within the same cell in the grid
```

To this:

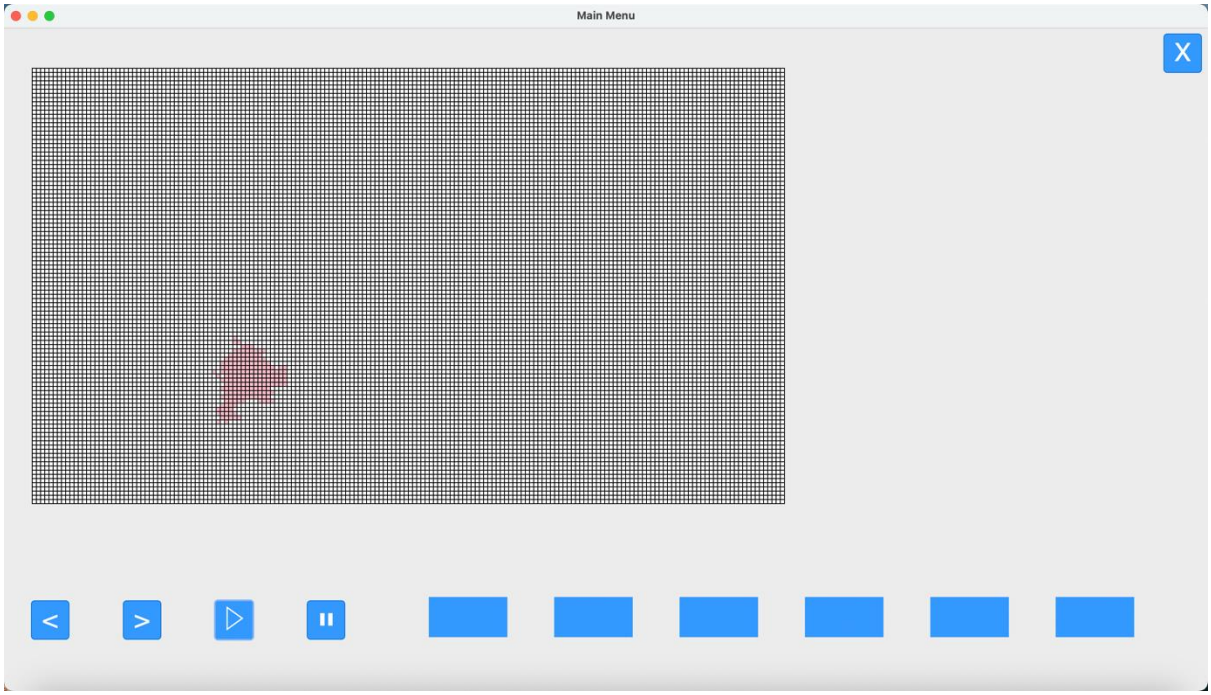
```
class PopulationModel(mesa.Model):
    def __init__(self):
        self.travelRadius = 1
        self.infectionRate = 0.2
        self.recoveryRate = 0.5
        self.fatalityRate = 0.5
        self.timeStepsNum = 100
        self.width = 180
        self.height = 104
        self.pause = False
```

Once I had altered the travelRadius attribute so that it could be called from the model, I altered the retireve_input function in the main_menu.py file:

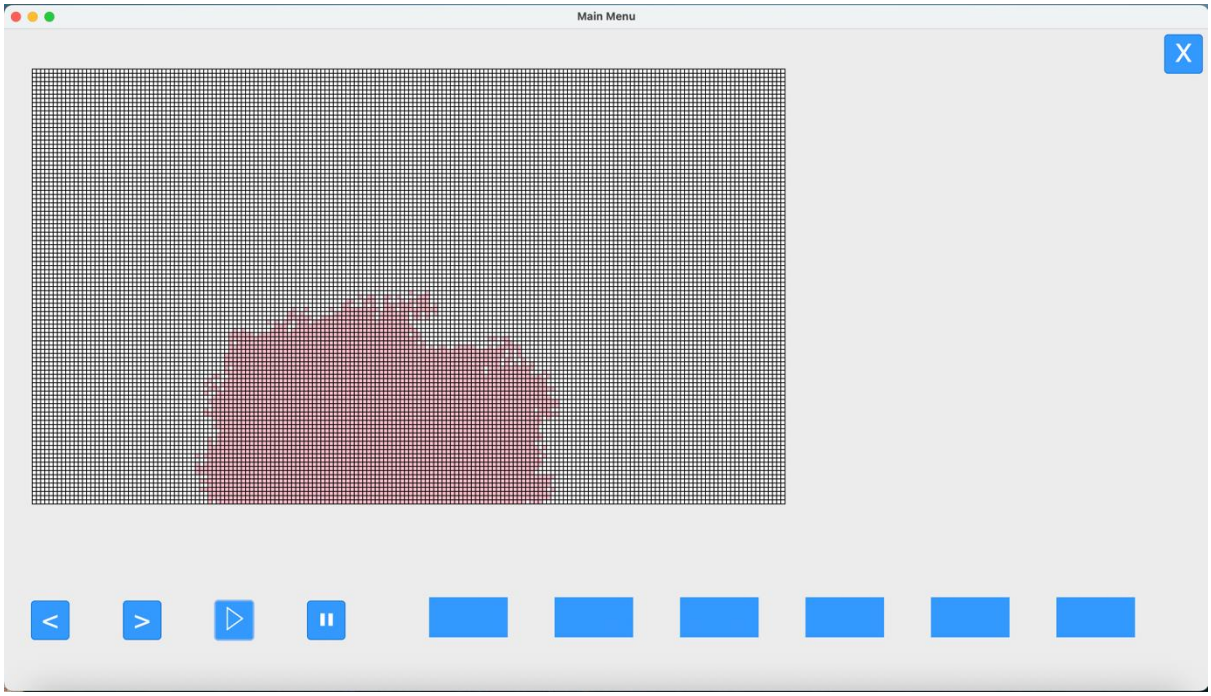
```
def retrieve_input(model,inputText,inputBox):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    paramaterValue = inputText.get("1.0",tk.END)
    #deletes text from input box so isnt added on next time function is run
    inputText.delete("1.0",tk.END)
    #validates whether the value is an integer or float
    try:
        paramaterValue = int(paramaterValue)
    except ValueError:
        paramaterValue = float(paramaterValue)

    #input validation as outlined via test plan
    match inputBox :
        #num of time-steps box
        case 1 :
            if paramaterValue > 0 and paramaterValue < 10000 and type(paramaterValue) == int:
                model.timeStepsNum = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #number of initial infections
        case 2 :
            if paramaterValue > 0 and paramaterValue < 18720 and type(paramaterValue) == int:
                #assign to initial infections paremeter
            else :
                print(f'{paramaterValue} is not valid')
        #travel radius
        case 3 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == int:
                model.travelRadius = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #infection rate box
        case 4 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.infectionRate = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #recovery rate box
        case 5 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.recoveryRate = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #fatality rate box
        case 6 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.fatalityRate = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
```

This meant that every input box now had a related attribute that when changed would alter how the similation behaves, as outlined in the design plan. So this was the simulation result with a travel

radius of 1 and an infectionr rate of 0.4 with no fatality or recovery algorithms running running for four time-steps:



And then this is the result when the travel-radius is 2 with the rest of the paramaters being indentical to the previous test:



As shown the travel radius has doubled with the spread of the disease pretty much doubling, showing that the altering of the travel radius attribute has been implemented successfully, so when the user changes that attribute it directly affects the result of the simulation.

However I noticed there was one paramater box that hadn't been assigned an attribute and that was the number of initial infections. I had forgot to implement the functionality of a user-set number of

agents starting as infected into the program. This meant that I had to add this functionality. The first thing I did was move to the population.py file and create a new attribute in the model titled initialInfections with an initial value of 1:

```
class PopulationModel(mesa.Model):
    def __init__(self):
        self.travelRadius = 1
        self.initialInfections = 1
        self.infectionRate = 0.2
        self.recoveryRate = 0.5
        self.fatalityRate = 0.5
        self.timeStepsNum = 100
        self.width = 180
        self.height = 104
        self.pause = False
```

I then went to the initialisation method for the agents to make it so that a user-defined amount of agents would start off as infected. The first thing I did was to create a list filled with random agent ids, with the amount of agent ids in that list being defined by the initialInfections paramater set by the user:

```
#generates user-set amount of agent ids to be infected
initialInfectedAgents = [random.randint(0,18720) for i in range(self.initialInfections)]
```

I then coded it so that while the agents were being initialised if it's agent id was within this list, it's state would be set to infected thus creating a set number of agents that would start off in the simulation as infected:

```
#loop to place agents in grid and scheduler
for d,i in enumerate(refCoordinates):
    a = PopulationAgent(d, self)

    #if agent id in initial infected list
    if d in initialInfectedAgents :
        a.state = 'I'
```

This meant that when the initialInfections paramater was set to 4, this was the result:

You might have to look closely but from this screenshot of the first time-step in the simulation there are four inefcted agents, in completely random positions thus showing that this functionality has been implemented.

The next step was to alter the retrieve_input function in main_menu.py to accommodate for the initialInfections paramater:

```
def retrieve_input(model,inputText,inputBox):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    paramaterValue = inputText.get("1.0",tk.END)
    #deletes text from input box so isnt added on next time function is run
    inputText.delete("1.0",tk.END)
    #validates whether the value is an integer or float
    try:
        paramaterValue = int(paramaterValue)
    except ValueError:
        paramaterValue = float(paramaterValue)

    #input validation as outlined via test plan
    match inputBox :
        #num of time-steps box
        case 1 :
            if paramaterValue > 0 and paramaterValue < 10000 and type(paramaterValue) == int:
                model.timeStepsNum = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #number of initial infections
        case 2 :
            if paramaterValue > 0 and paramaterValue < 18720 and type(paramaterValue) == int:
                model.initialInfections = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #travel radius
        case 3 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == int:
                model.travelRadius = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #infection rate box
        case 4 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.infectionRate = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #recovery rate box
        case 5 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.recoveryRate = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #fatality rate box
        case 6 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.fatalityRate = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
```

This now meant that every input box in the GUI had an associated attribute that the user could change by entering a value and thus would change the way the simulation behaves.

Finally, the final function that I needed to implement was something extra that I found would be useful when running the program. At it's current state, in order to test whether new functionalities work, I needed to reset the simulation in order to test the new changes. However in order to do this I needed to close and re-open the whole program. Therefore I decided to implement a function that would reset the simulation to it's initial state and that it would run whenever the user entered a new paramater in one of the input boxes.

To implement this function I would pretty much have to call the whole initialisation function for the model again. Since I wanted to avoid repeated code, I decided to separate the different initalisation components in the model into separate methods that could be called from the class. This meant that

the clear function that I wanted to implement could merely call a sequence of these methods to completely re-initialise the model and create a clean slate.

The first thing I did was to separate the population intialisation within the population model into it's own method titled 'init_population':

```
        #agents can't be within the same cell in the grid
        self.grid = mesa.space.SingleGrid(self.width, self.height, True)
        #defines scheduler that will activate all agents randomly at each time step
        self.schedule = mesa.time.RandomActivation(self)

    #defines a method to create population
    def init_population(self):
        #creates reference array of coordinates for agents to be added to
        refCoordinates = [[x,y] for y in range(0,self.height) for x in range(0,self.width)]

        #generates user-set amount of agent ids to be infected
        initialInfectedAgents = [random.randint(0,18720) for i in range(self.initialInfections)]

        #loop to place agents in grid and scheduler
        for d,i in enumerate(refCoordinates):
            a = PopulationAgent(d, self)

            #if agent id in initial infected list
            if d in initialInfectedAgents :
                a.state = 'I'

            #adds the agent to the scheduler
            self.schedule.add(a)
            #places agent within the grid
            self.grid.place_agent(a, (i[0],i[1]))
```

I then seperated the algorithm to generate neighbouring agent lists for each agent into it's own method as well:

```
            self.grid.place_agent(a, (i[0],i[1]))

    #defines a method to generate neighbouring agents list for each agent
    def init_neighbour_agents(self) :
        for a in self.grid:
            #max and min x,y neighbour positions
            minxPos, maxxPos = a.pos[0]-self.travelRadius, a.pos[0]+self.travelRadius
            minyPos, maxyPos = a.pos[1]-self.travelRadius, a.pos[1]+self.travelRadius
            #coordinate validation
            if minxPos < 0 : minxPos = 0
            if maxxPos > self. width -1:
                maxxPos = self.width - 1
            if minyPos < 0 : minyPos = 0
            if maxyPos > self.height -1:
                maxyPos = self.height - 1

            #creates a list of all possible positions with range of the max and min x,y neighbour positions
            total_radius_positions = [[x,y] for x in range(minxPos,maxxPos+1)
                                            for y in range(minyPos,maxyPos+1)]

            #removes the position of the agent it's finding the neighbours of
            total_radius_positions.remove([a.pos[0],a.pos[1]])

            #assigns the list as an attribute
            a.neighbour_agents = total_radius_positions
```

I then referenced bot of these methods when the model is defined in the 'FullSimulation' class in the 'main_window.py' file, so that the whole initalisation is carried out as it was:

```
class FullSimulation :
    def __init__(self,model) :
        #assigns model
        self.model = model
        #initiates neighbouring agents list for each agent
        model.init_population()
        model.init_neighbour_agents()
```

This now meant that I could create a function that could reference these two methods, to effectively run the whole model initialisation process again, resetting the simulation. I therefore implemented these two functions into a function called ‘clear’ :

```
def clear(self,model) :
    #resets population and neighbouring agents i.e if travel radius changes
    model.init_population()
    model.init_neighbour_agents()
```

I then added a few algorithms to the clear method to reset the memory dictionary of each agent, so it had no history of states to be referred to. I also reset the time scheduler so the time-steps would start from zero as it would in a new simulation. Finally, I then iterated through the grid to randomly create initial infected agents based on the initial-infections parameter that the user entered. :

```
def clear(self,model,canvas,squares) :
    #resets population and neighbouring agents i.e if travel radius changes
    model.init_population()
    model.init_neighbour_agents()

    #resets time of simulation to zero
    self.model.schedule.time = 0
    #stops simulation in progress
    stop(model)
    #randomly selects agents to be selected based on initial infections paramater
    initialInfectedAgents = [random.randint(0,18720) for i in range(model.initialInfections)]
    for i in model.grid :
        #resets memory dictionary so each agent has no memor of previous states
        i.memory = {}
        #if agent is in initial infections list set state to infected, else set to susceptible
        if i.unique_id in initialInfectedAgents :
            i.state = 'I'
        else:
            i.state = 'S'

    #colours agents based on state
    categorise_agents(model,canvas,squares)
    #updates GUI
    window.update()
```

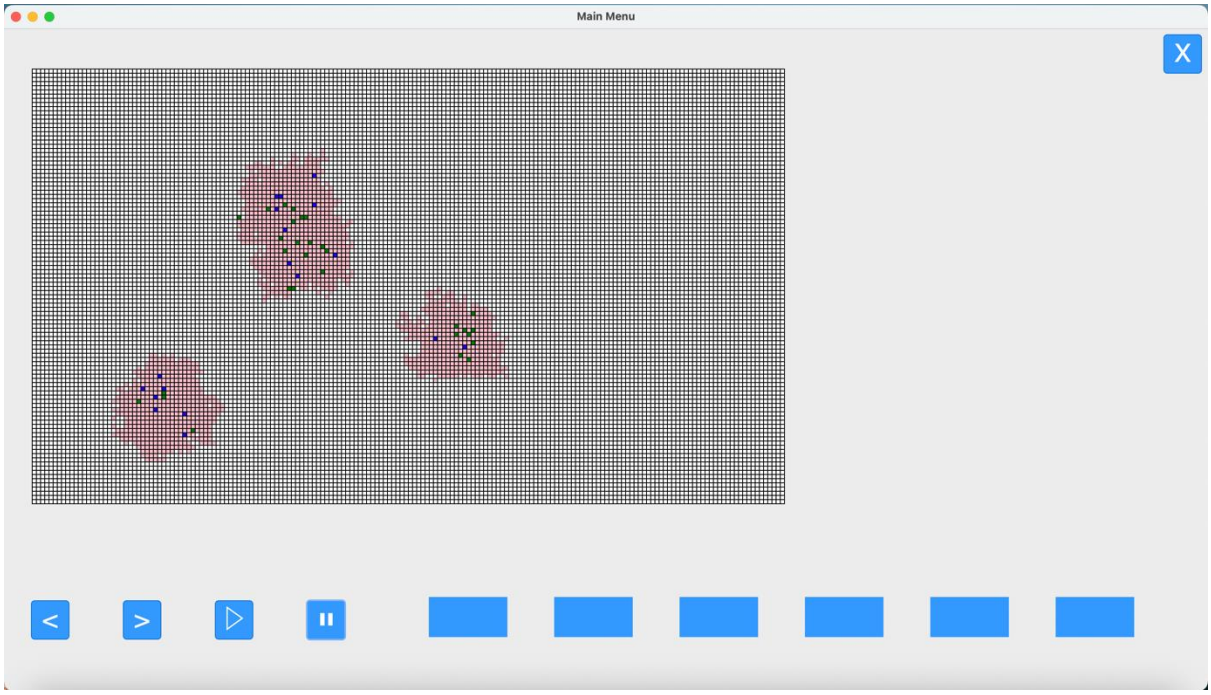
I then temporarily assigned this function to the exit button to see if it worked. However, when I tried to click on the button to clear the simulation this error message appeared:

```
  File "/usr/local/lib/python3.11/site-packages/mesa/time.py", line 76, in add
    raise Exception(
Exception: Agent with unique id 0 already added to scheduler
```

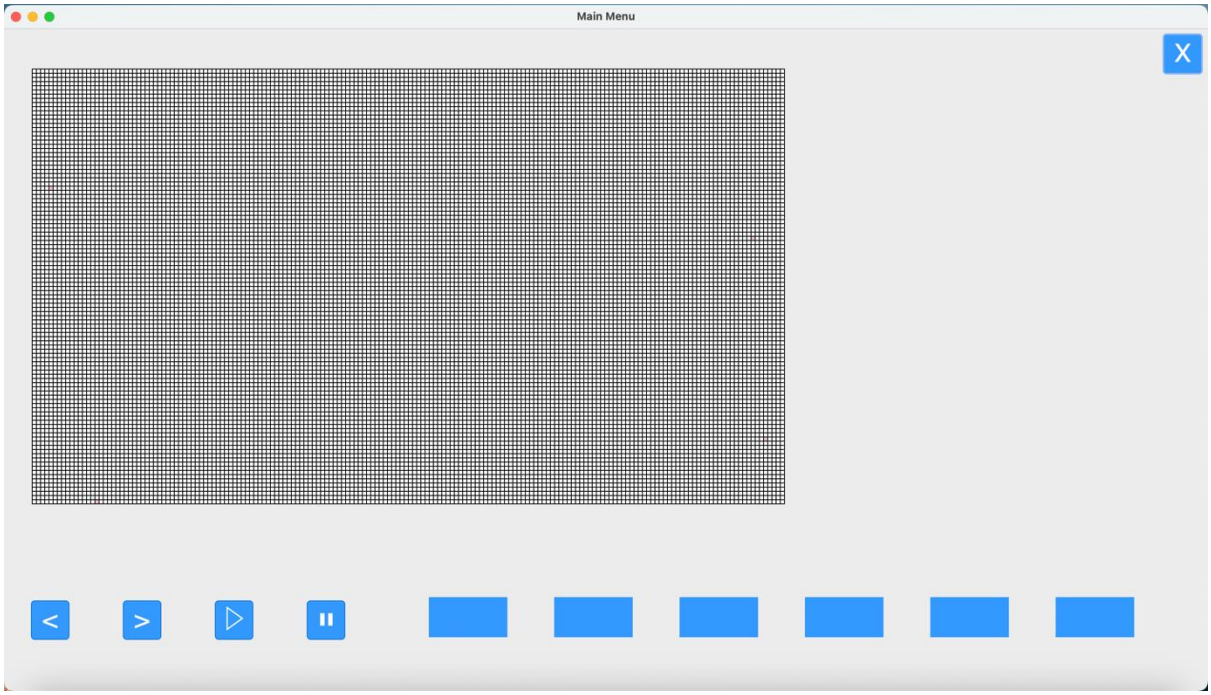
This was because the population initialisation algorithm was running again when the population had already been initialised, so it was running twice needlessly. To fix this, I removed the population initialisation method from the clear function so that it would only run the neighbour initialisation method alongside the other algorithms:

```
def clear(self,model,canvas,squares) :
    #resets population and neighbouring agents i.e if travel radius changes
    model.init_neighbour_agents()

    #resets time of simulation to zero
    self.model.schedule.time = 0
    #stops simulation in progress
    stop(model)
    #randomly selects agents to be selected based on initial infections paramater
    initialInfectedAgents = [random.randint(0,18720) for i in range(model.initialInfections)]
    for i in model.grid :
        #resets memory dictionary so each agent has no memor of previous states
        i.memory = {}
        #if agent is in initial infections list set state to infected, else set to susceptible
        if i.unique_id in initialInfectedAgents :
            i.state = 'I'
        else:
            i.state = 'S'

    #colours agents based on state
    categorise_agents(model,canvas,squares)
    #updates GUI
    window.update()
```

So, this was the simulation before:



And this was the simulation when the exit button was pressed, (where the function was assigned):



This showed that the function was now working as intended with the simulation being reset back to it's starting state once the function was called. The next step was to assign this function to the input boxes so that whenever the user changed a value within the simulation, the clear() function would be called and the simulation would be reset, ready to start from the new parameters that have been changed by the user.

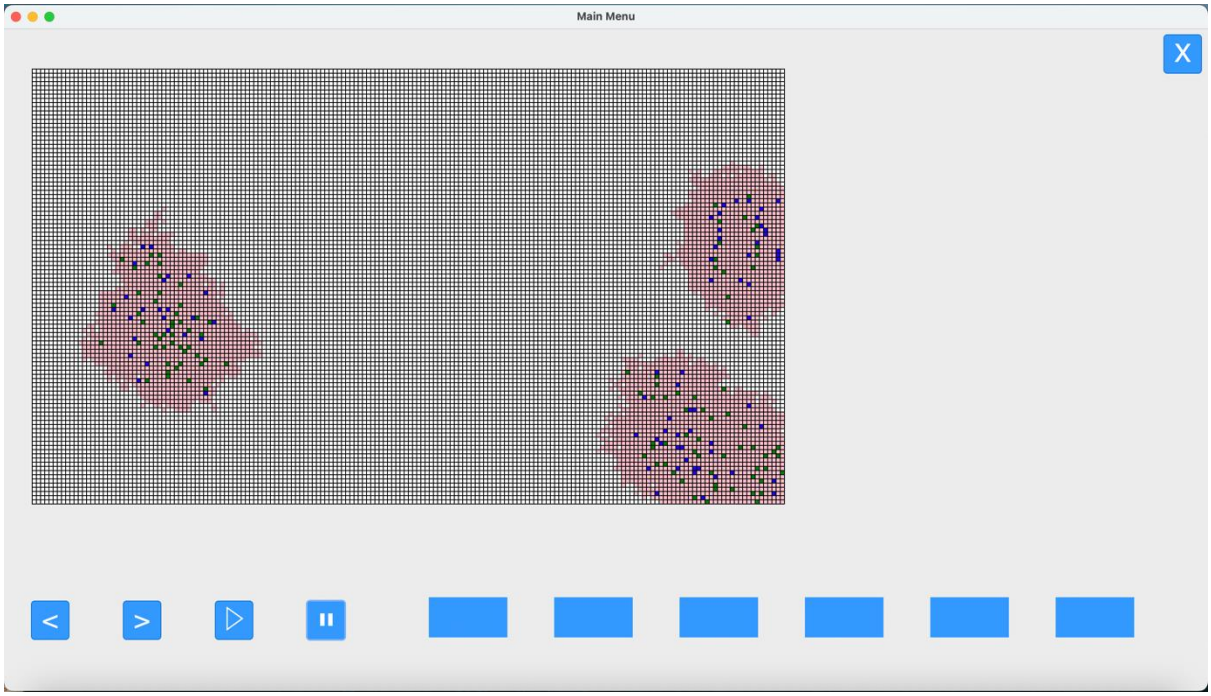
To do this all I needed to alter the retireve_input function since that controlled what happened every time a user entered a value into an input box, so that every time a value was entered the clear function with the needed parameter values would be called. Therefore, I altered the function to accept two new parameters that would be needed when the clear() function was called inside of it:

```
def retrieve_input(self,model,inputText,inputBox):
```

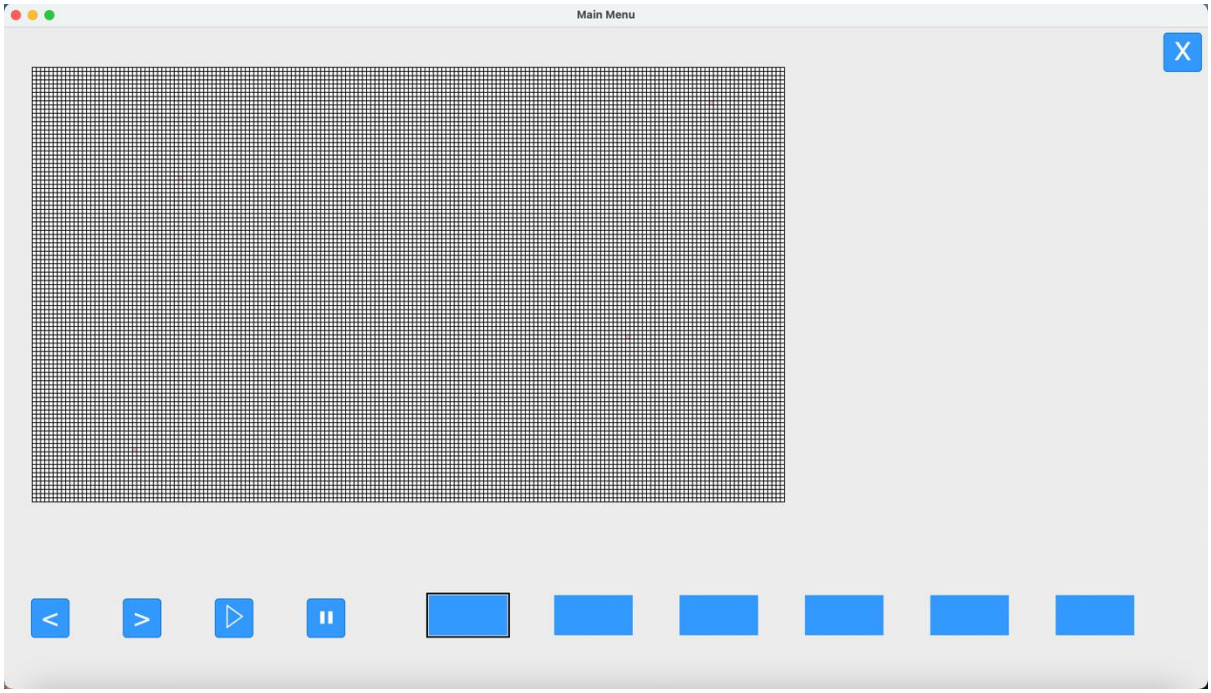
I then referenced the clear function inside every successful input case in the retrieve_input function:

```
def retrieve_input(self,model,inputText,inputBox):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    paramaterValue = inputText.get("1.0",tk.END)
    #deletes text from input box so isnt added on next time function is run
    inputText.delete("1.0",tk.END)
    #validates whether the value is an integer or float
    try:
        paramaterValue = int(paramaterValue)
    except ValueError:
        paramaterValue = float(paramaterValue)

    #input validation as outlined via test plan
    match inputBox :
        #num of time-steps box
        case 1 :
            if paramaterValue > 0 and paramaterValue < 10000 and type(paramaterValue) == int:
                model.timeStepsNum = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0 or paramaterValue > 10000:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #number of initial infections
        case 2 :
            if paramaterValue > 0 and paramaterValue < 18720 and type(paramaterValue) == int:
                model.initialInfections = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0 or paramaterValue > 18720:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #travel radius
        case 3 :
            if paramaterValue > 0 and paramaterValue < 5 and type(paramaterValue) == int:
                model.travelRadius = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
                print(f'The travel radius is now {model.travelRadius}')
            elif paramaterValue < 0 or paramaterValue > 5:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #infection rate box
        case 4 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.infectionRate = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #recovery rate box
        case 5 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.recoveryRate = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #fatality rate box
        case 6 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.fatalityRate = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
```

This now meant that every time the user entered a new value in one of the parameter input boxes, the simulation should automatically reset itself to a blank state. Here was the simulation before the parameter value in the input box is entered:



And here was the simulation afterwards once the user had entered a value into an input box and pressed enter:

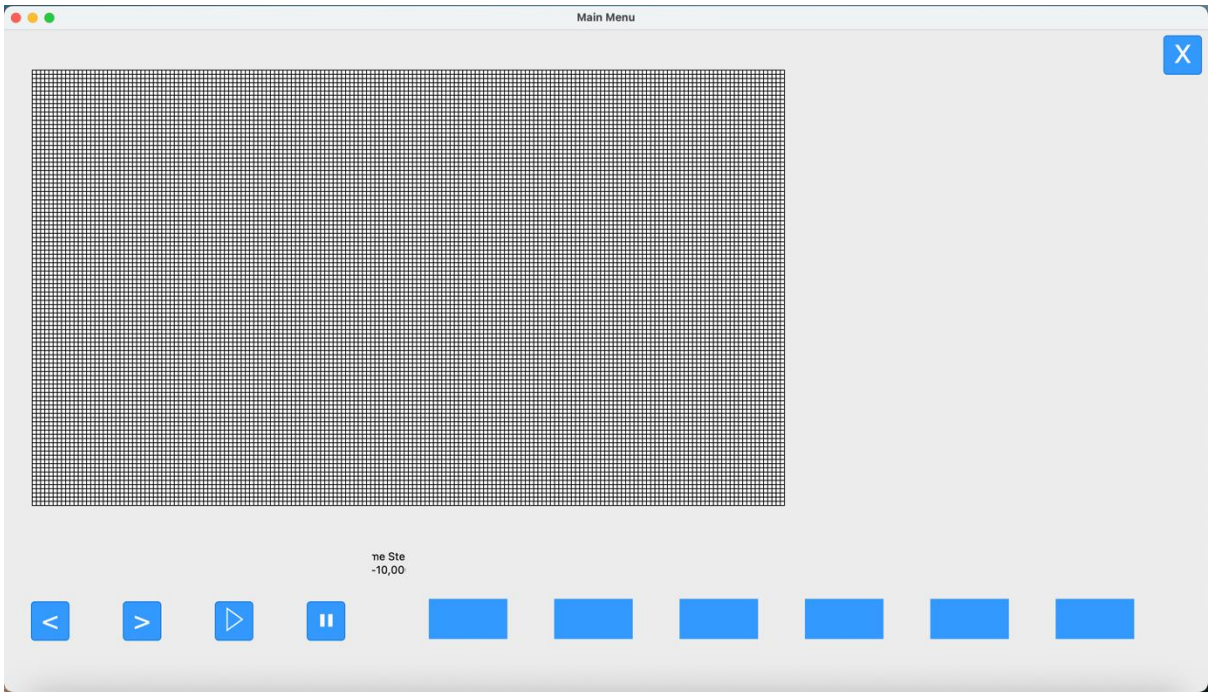


This meant that the function had been created and implemented successfully so now whenever a user changed a parameter by entering a new one in the box, the simulation would reset itself successfully

The last thing to implement, after looking through the success criteria for this section, were parameter box labels. These are an instrumental of the program since they allow the user to know how each input box affects the simulation, as well as specifying the range of values that can be entered into the box. I had tried to implement this earlier in the development cycle but to no avail. This is because tkinter's alignment method pack() that I had used to place all my elements didn't

allow to place text elements underneath the text boxes. This was the outcome when I tried to pack a text element underneath the first box:

```
#time steps num
self.i1.pack(side='right',pady=60,padx=25)

#text labels for user accessibility
self.i1text = tk.Label(window, text="Time Steps\n(1-10,000)")
self.i1text.pack()
```

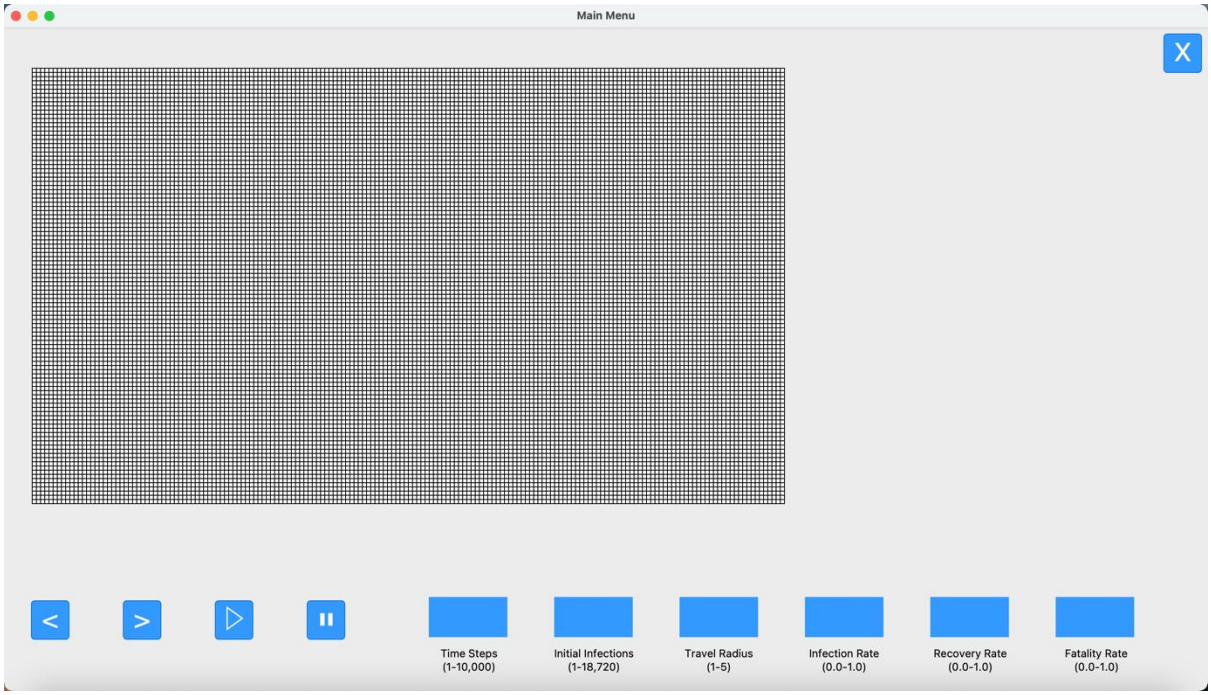


However, after going searching for alternative methods for placing elements in tkinter, I came across the place() method. This uses the geometry() method of alignment instead of pack(). This meant that any element placed using that method would be independent from all the elements placed using pack() and could therefore be placed anywhere in the window regardless of elements that were already there.

So I created six text-labels, one for each parameter box and used the place() method to place each one as well as x and y coordinates, using trial and error to place them properly within the window. Here's the code for the implementation of the six text-labels:

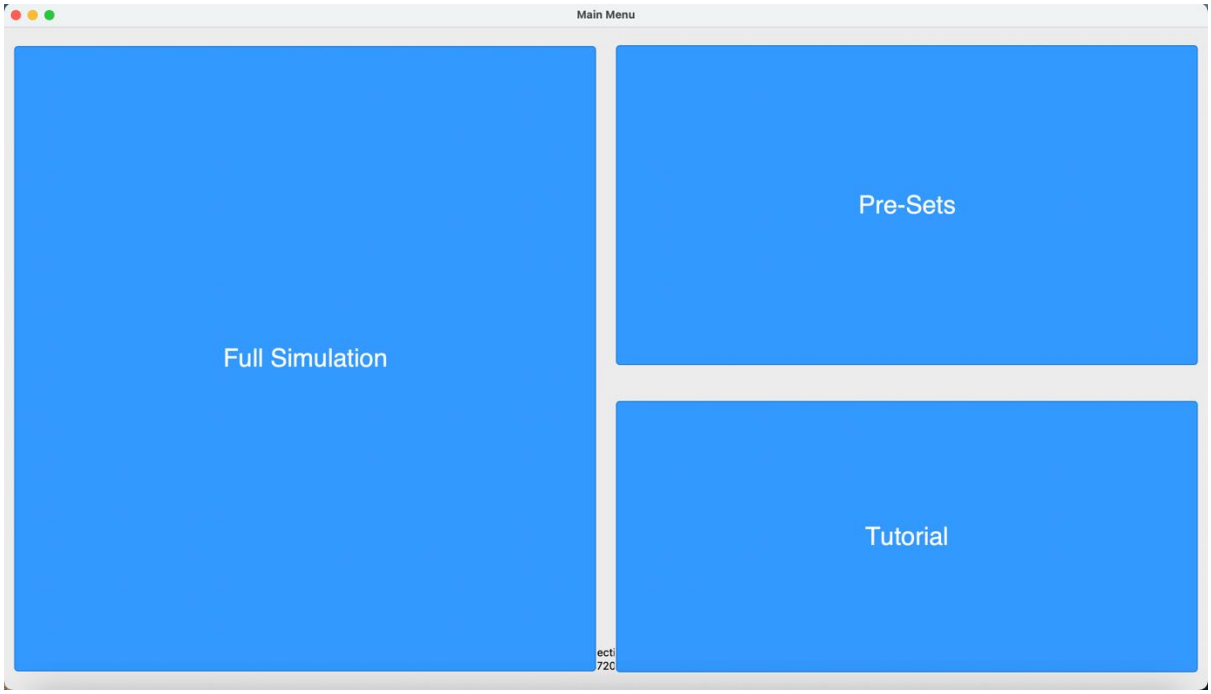
```
#text labels for user accessibility
self.i1text = tk.Label(window, text="Time Steps\n(1-10,000)")
self.i1text.place(x=519, y=735)

self.i2text = tk.Label(window, text="Initial Infections\n(1-18,720)")
self.i2text.place(x=655, y=735)

self.i3text = tk.Label(window, text="Travel Radius\n(1-5)")
self.i3text.place(x=811, y=735)

self.i4text = tk.Label(window, text="Infection Rate\n(0.0-1.0)")
self.i4text.place(x=960, y=735)

self.i5text = tk.Label(window, text="Recovery Rate\n(0.0-1.0)")
self.i5text.place(x=1109, y=735)

self.i6text = tk.Label(window, text="Fatality Rate\n(0.0-1.0)")
self.i6text.place(x=1266, y=735)
```

And here's what the result looked like:



This meant that labels for each parameter box were now implemented, allowing the user to see what parameter they are inputting as well as the valid value range for each parameter box. With that the final piece of success criteria for this section had been carried out.

However while running the program, I encountered an error. Whenever I went back to the main menu screen from the simulation the elements didn't disappear:



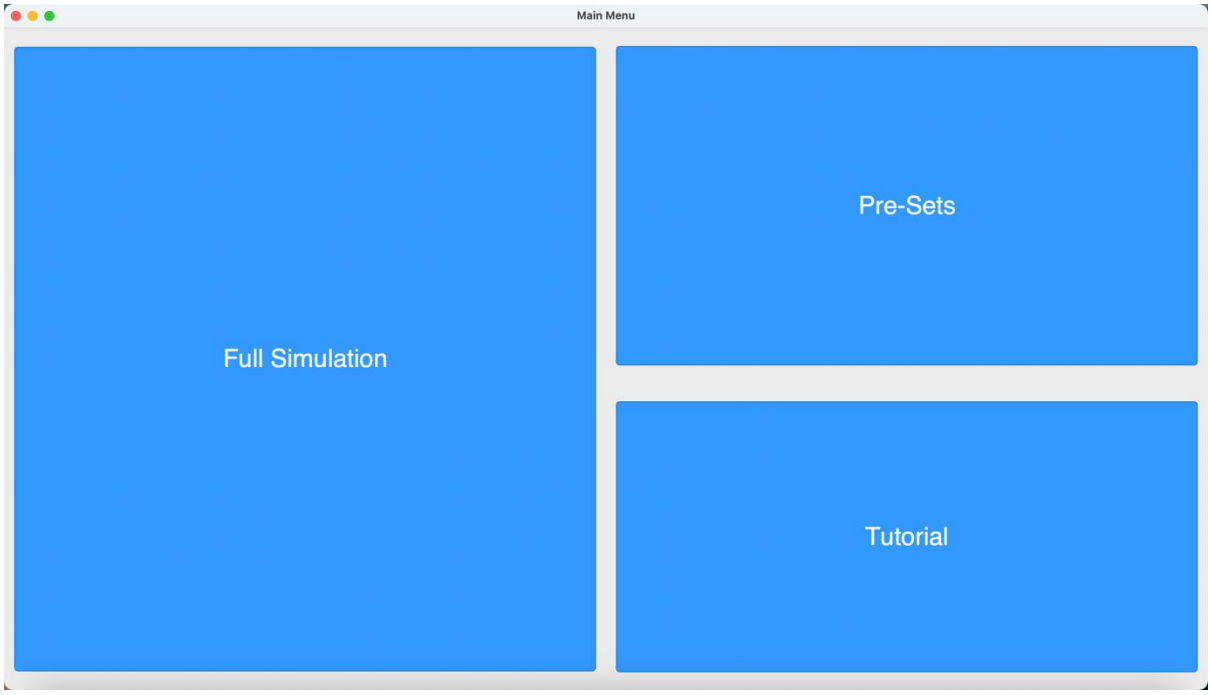
This was because the method used to remove elements from the screen once a new page is initialised was pack_forget() and since I had implemented these elements using the place() method it was not compatible with these new elements. After looking up a method to get rid of elements palced using the place() method I came across the destroy() method which was the equivalent for removing elements. I therefore added these elements to the same list as the others:

```
self.i6text = tk.Label(window, text="Fatality Rate\n(0.0-1.0)")
self.i6text.place(x=1266, y=735)

self.objects = [self.b1,self.t1,self.t2,self.p1,self.p2,self.i1,self.i2,self.i3,self.i4,self.i5,self.i6,self.squareCanvas,
                self.i1text,self.i2text,self.i3text,self.i4text,self.i5text,self.i6text]
```

Then I added a special case in the get_page() function so that if it was an element placed using place() it would get removed using destroy() instead of pack_forget(). I did this by specifying that the last six elements would be removed using destroy() since those are the special-case elements:

```
def get_page(items,pageNum,model):
    #stops model if currently running
    #iterates through items list and gets rid of each item from page
    [items[i].pack_forget() for i in range(len(items)) if i < 13 ]
    [items[i].destroy() for i in range(len(items)) if i > 12 ]
    #loads the specified page based on number paramater
    if pageNum == 0:
        MainMenu()
    elif pageNum == 1 :
        FullSimulation(model)
    elif pageNum == 2:
        PreSets(model)
    elif pageNum == 3:
        Tutorial(model)
```

This was the result now that this new special-case had been implemented:



This meant that the error was now fixed and that the text labels had been properly added to the program.

Changes in Design Made in this Iteration

This iteration implements the simulation loop algorithm outlined in the design plan. It creates two versions of this algorithm, one to move forwards and one to move backwards, each being able to be called based on what the user wants to do. Just like the algorithm in the design plan, it calls a series of functions that will move the simulation one time-step forward in total but these algorithms are different from the ones outlined in the design plan due to the actual implementation of these being different in program code than they were in an abstracted flowchart form, with these changes from design being outlined in previous iterations.

I haven't stated the algorithm to move back through the simulation in the design section of the program since I only realises the need for such an algorithm part-way through the development of the program to meet the success criteria outlined. However, if more time was given, I would outline this algorithm in the design section before implementing it in the program.

How has this filled Success Criteria and User Requirements?

This iteration has fulfilled the following success criteria:

- Play Button
- Pause Button
- Forward Arrow Button
- Backward Arrow Button

Although these were implemented in the earlier part of this iteration, the actual functionality in relation to the simulation has been added meaning each one is now practically implemented and useable.

Testing

Iteration 1: Menu, Boxes and Buttons

Buttons

Button Name	Intended Functionality	Functionality when User Clicks
Exit	Will close program when clicked	WORKS AS INTENDED
Full Simulation	Will take the user to the 'Full Simulation' page when clicked	WORKS AS INTENDED
Tutorial	Will take the user to the 'Tutorial' page when clicked	NOT YET IMPLEMENTED
Pre-sets	Will take the user to the 'pre-sets' page when clicked	NOT YET IMPLEMENTED
Play Button	Will start the simulation when clicked	REFEER TO VIDEO
Pause Button	Will pause the simulation when clicked	REFEER TO VIDEO
Add Button	Will add pre-set values to database when clicked	NOT YET IMPLEMENTED
Remove Button	Will remove selected pre-set values when clicked	NOT YET IMPLEMENTED
Next Time-step Button	Will move to the next time-step when clicked	WORKS AS INTENDED
Previous Time-step Button	Will move to the previous time-step when clicked	WORKS AS INTENDED

Input Boxes

Maximum Number of Time-Steps

Input	Validation Type	Intended Output	Actual Output
3,200	Normal	Simulation will run for 3,200 time-steps	WORKS AS INTENDED
0	Invalid	User will be asked to enter a valid value within specified range of 1 – 10,000	WRONG MESSAGE OUTPUTTED
10,001	Invalid	User will be asked to enter a valid value within specified range of 1 – 5,000	WRONG MESSAGE OUTPUTTED
1	Boundary	Simulation will run for one time-step	WORKS AS INTENDED
10,000	Boundary	Simulation will run for 10,000 time-steps	WORKS AS INTENDED
'1'	Erroneous	Program will not allow user to input quotation marks	WORKS AS INTENDED
1.5	Erroneous	Program will not allow the user to input a decimal point	WORKS AS INTENDED
''	Erroneous	User will be asked to enter a value	WRONG MESSAGE OUTPUTTED

Number of Initial Infections

Input	Validation Type	Intended Output	Actual Output
3,200	Normal	Simulation will start with 3,200 infected agents	WORKS AS INTENDED
0	Invalid	User will be asked to enter a valid value within specified range of 1 – 9,999	WRONG MESSAGE OUTPUTTED
18,721	Invalid	User will be asked to enter a valid value within specified range of 1 – 4,999	WRONG MESSAGE OUTPUTTED
1	Boundary	Simulation will start with one infected agent	WORKS AS INTENDED
18,720	Boundary	Simulation will start with 18,27- infected agents	WORKS AS INTENDED
‘1’	Erroneous	Program will not allow user to input quotation marks	WORKS AS INTENDED
1.5	Erroneous	Program will not allow the user to input a decimal point	WORKS AS INTENDED
‘’	Erroneous	User will be asked to enter a value	WRONG MESSAGE OUTPUTTED

Measure of Infection Rate

Input	Validation Type	Intended Output	Actual Output
0.4	Normal	Simulation will start with an infection rate of 0.4	WORKS AS INTENDED
0.0	Invalid	User will be asked to enter a valid value within specified range of 0.01 - 0.99.	WRONG MESSAGE OUTPUTTED
1.0	Invalid	User will be asked to enter a valid value within specified range of 0.01 - 0.99.	WRONG MESSAGE OUTPUTTED
0.01	Boundary	Simulation will start with an infection rate of 0.01	WORKS AS INTENDED
0.99	Boundary	Simulation will start with an infection rate of 0.99	WORKS AS INTENDED
‘0.4’	Erroneous	Program will not allow user to input quotation marks	WORKS AS INTENDED

‘’	Erroneous	User will be asked to enter a value	WRONG MESSAGE OUTPUTTED

Travel Radius

Input	Validation Type	Intended Output	Actual Output
2	Normal	Adds travel radius amount to the current x and y coordinates to find the	WORKS AS INTENDED
-1	Invalid	User will be asked to enter a valid value within the specified range of 1 - 10	WRONG MESSAGE OUTPUTTED
11	Invalid	User will be asked to enter a valid value within the specified range	WRONG MESSAGE OUTPUTTED
1	Boundary	Simulation will start with a travel radius of 1	WORKS AS INTENDED
10	Boundary	Simulation will start with a travel radius of 10	WORKS AS INTENDED
‘1’	Erroneous	Program will not allow user to input quotation marks	WORKS AS INTENDED
1.5	Erroneous	Program will not allow the user to input a decimal point	WORKS AS INTENDED
‘’	Erroneous	User will be asked to enter a value	WRONG MESSAGE OUTPUTTED

Recovery Rate

Input	Validation Type	Intended Output	Actual Output
0.4	Normal	Simulation will start with an infection rate of 0.4	WORKS AS INTENDED
0.0	Invalid	User will be asked to enter a value in valid range	WRONG MESSAGE OUTPUTTED
1.0	Invalid	User will be asked to enter a value in valid range	WRONG MESSAGE OUTPUTTED
0.01	Boundary	Simulation will start with an infection rate of 0.01	WORKS AS INTENDED
0.99	Boundary	Simulation will start with an infection rate of 0.99	WORKS AS INTENDED

‘0.4’	Erroneous	Program will not allow user to input quotation marks	WORKS AS INTENDED
‘’	Erroneous	User will be asked to enter a value	WRONG MESSAGE OUTPUTTED

Fatality Rate

Input	Validation Type	Intended Output	Actual Output
0.4	Normal	Simulation will start with an infection rate of 0.4	WORKS AS INTENDED
0.0	Invalid	User will be asked to enter a value in valid range	WRONG MESSAGE OUTPUTTED
1.0	Invalid	User will be asked to enter a value in valid range	WRONG MESSAGE OUTPUTTED
0.01	Boundary	Simulation will start with an infection rate of 0.01	WORKS AS INTENDED
0.99	Boundary	Simulation will start with an infection rate of 0.99	WORKS AS INTENDED
‘0.4’	Erroneous	Program will not allow user to input quotation marks	WORKS AS INTENDED
‘’	Erroneous	User will be asked to enter a value	WRONG MESSAGE OUTPUTTED

Changes Made Post-Testing:

The only thing that I needed to alter to satisfy the test plan was to alter the output message, tailoring the output message to the specific error case.

I did this by changing the retrieve_input function in main_window.py from this:

```
def retrieve_input(model,inputText,inputBox):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    paramaterValue = inputText.get("1.0",tk.END)
    #deletes text from input box so isnt added on next time function is run
    inputText.delete("1.0",tk.END)
    #validates whether the value is an integer or float
    try:
        paramaterValue = int(paramaterValue)
    except ValueError:
        paramaterValue = float(paramaterValue)

    #input validation as outlined via test plan
    match inputBox :
        #num of time-steps box
        case 1 :
            if paramaterValue > 0 and paramaterValue < 10000 and type(paramaterValue) == int:
                model.timeStepsNum = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #number of initial infections
        case 2 :
            if paramaterValue > 0 and paramaterValue < 18720 and type(paramaterValue) == int:
                model.initialInfections = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #travel radius
        case 3 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == int:
                model.travelRadius = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #infection rate box
        case 4 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.infectionRate = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #recovery rate box
        case 5 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.recoveryRate = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
        #fatality rate box
        case 6 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.fatalityRate = paramaterValue
            else :
                print(f'{paramaterValue} is not valid')
```

To this:

```
def retrieve_input(model,inputText,inputBox):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    paramaterValue = inputText.get("1.0",tk.END)
    #deletes text from input box so isnt added on next time function is run
    inputText.delete("1.0",tk.END)
    #validates whether the value is an integer or float
    try:
        paramaterValue = int(paramaterValue)
    except ValueError:
        paramaterValue = float(paramaterValue)

    #input validation as outlined via test plan
    match inputBox :
        #num of time-steps box
        case 1 :
            if paramaterValue > 0 and paramaterValue < 10000 and type(paramaterValue) == int:
                model.timeStepsNum = paramaterValue
            elif paramaterValue < 0 or paramaterValue > 10000:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #number of initial infections
        case 2 :
            if paramaterValue > 0 and paramaterValue < 18720 and type(paramaterValue) == int:
                model.initialInfections = paramaterValue
            elif paramaterValue < 0 or paramaterValue > 18720:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #travel radius
        case 3 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == int:
                model.travelRadius = paramaterValue
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #infection rate box
        case 4 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.infectionRate = paramaterValue
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #recovery rate box
        case 5 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.recoveryRate = paramaterValue
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
        #fatality rate box
        case 6 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.fatalityRate = paramaterValue
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                print(f'Value is out of range')
            else :
                print('Please enter a value')
```

This now meant that every error case had a specialised output message, satisfying the test plan.

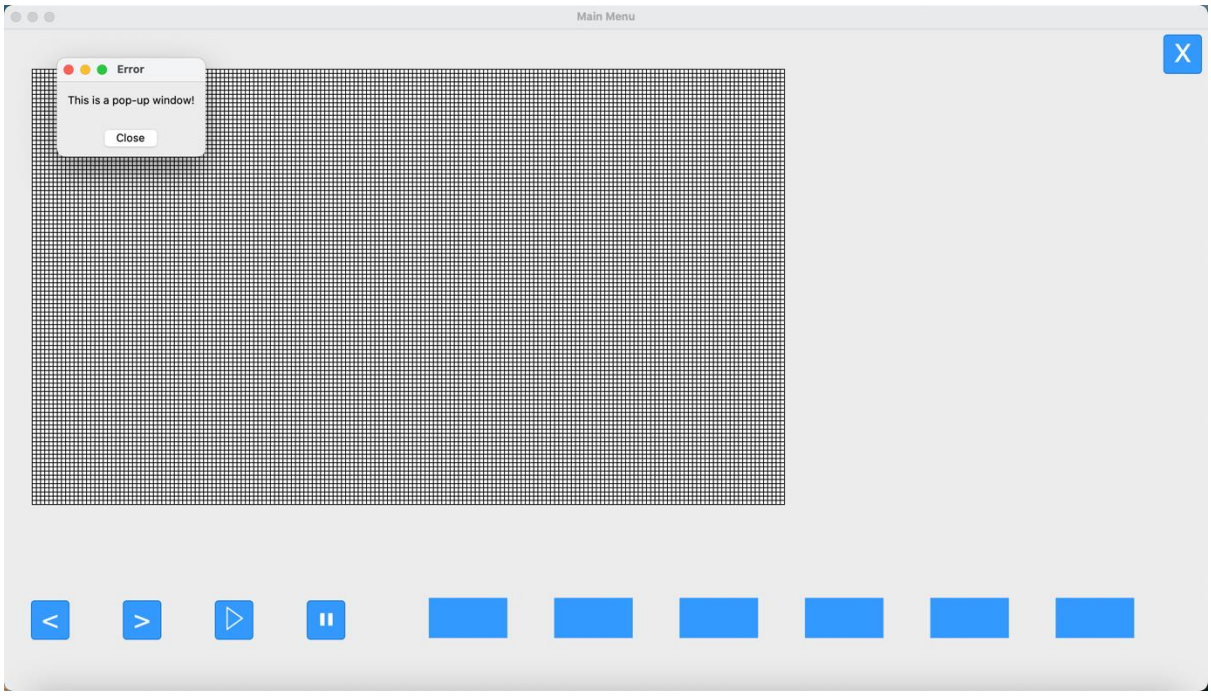
However what I also realised while changing these error messages in tkinter was that while the program is running, the user will not be able to see the terminal while the program is runing,

meaning that for each error message a pop-up window needs to be initialised so that the user can cactually see the error message displayed within the program.

To do this I created a new function called create_popup(). The purpose of this function is to create a window that would contain an error message depending on the paramater box and the associated validation for that box. After doing some research into the methods needed to create this using tkinter here was the function in 'main_window.py':

```
def create_popup():
    #cretes new window
    popup = tk.Toplevel(window)
    popup.title("Error")
    #sets text in window
    label = tk.Label(popup, text="This is a pop-up window!")
    #aligns in center of screen
    label.pack(padx=10, pady=10)
    #adds a close button for the user
    close_button = tk.Button(popup, text="Close", command=popup.destroy)
    close_button.pack(pady=100)
```

And here was the result when it was run:

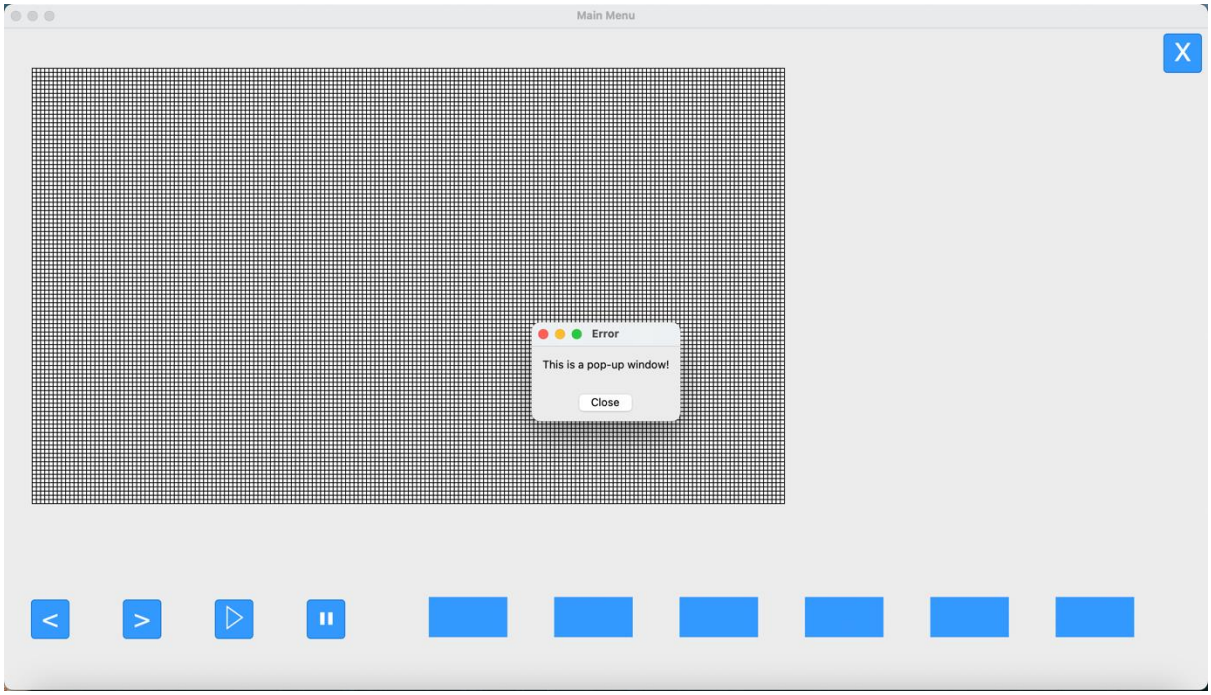


There were two things I needed to alter, I needed to centre the pop-up message so that it would pop-up in the centre of the program so that the user won't miss it and I also needed to customise the error message outputted in a case-by-case basis, so that depending on the input boundary violated by the user when entering a value into a parameter box, a customised error message will output. The first problem I decided to tackle was the centering of the pop-up box which turned out to be relatively simple.

By using tinker's geometry method, which is an alternative way of aligning elements, instead of tkinter's pack() method that I've been using to align elements so far throughout the program, I was able to set a series of coordinates that through much trial and error, placed the pop-up window in roughly the centre of the screen. Here's the modified function:

```
def create_popup():
    #cretes new window
    popup = tk.Toplevel(window)
    popup.title("Error")
    #sets text in window
    label = tk.Label(popup, text="This is a pop-up window!")
    #aligns in center of screen
    label.pack(pady=10,padx=10)
    #adds a close button for the user
    close_button = tk.Button(popup, text="Close", command=popup.destroy)
    close_button.pack(pady=10)
    #specifies width,heigh of the popup window itself, then the coords to place the window within the larger simulation page
    popup.geometry('178x90+631+405')
```

And here's the result:



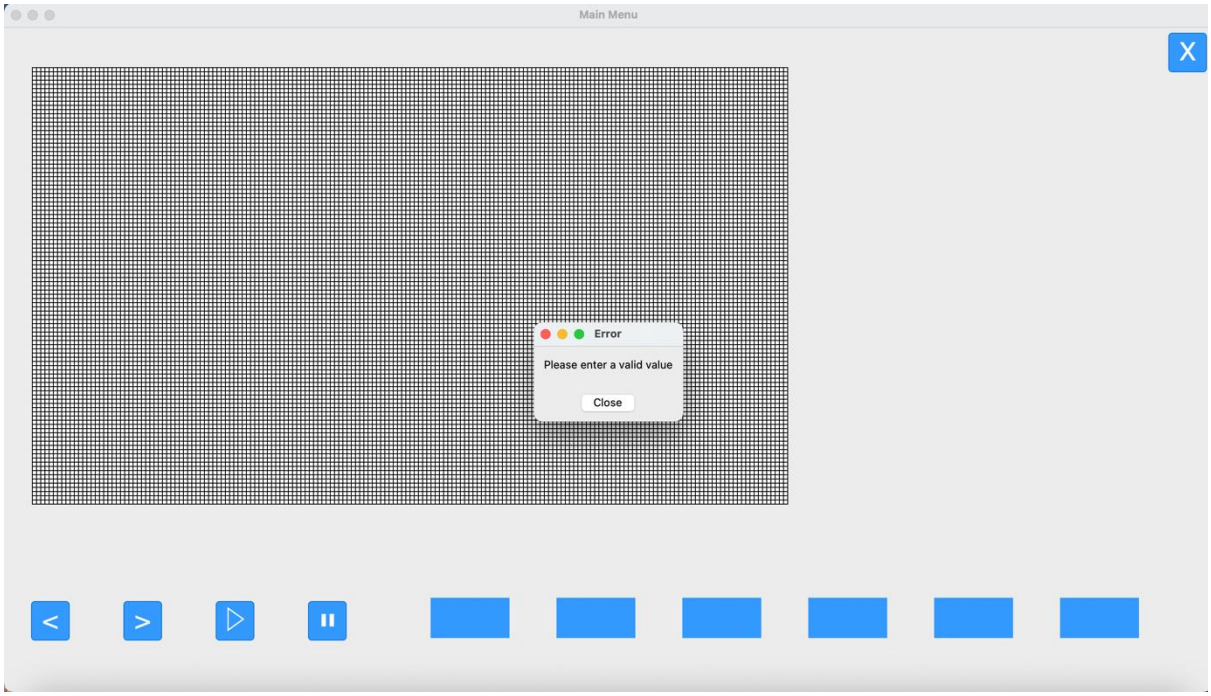
Now that it was centred the next step was to make it so that it outputted a specific error cased, based on the invalid value that the user enters a parameter box. The first thing I did was to add a new parameter to the create_popup function, titled message, which would be output on the popup window itself:

```
def create_popup(message):
    #cretes new window
    popup = tk.Toplevel(window)
    popup.title("Error")
    #sets paramater text in window
    label = tk.Label(popup, text=message)
```

I then referenced this function in each case of the retrieve_input function and instead of each message being outputted in terminal, I placed each of them as a parameter in the create_popup() function:

```
def retrieve_input(self,model,inputText,inputBox):
    #retrieves the text from the text box, 1.0 is beginning, tk.END is end
    paramaterValue = inputText.get("1.0",tk.END)
    #deletes text from input box so isnt added on next time function is run
    inputText.delete("1.0",tk.END)
    #validates whether the value is an integer or float
    try:
        paramaterValue = int(paramaterValue)
    except ValueError:
        paramaterValue = float(paramaterValue)

    #input validation as outlined via test plan
    match inputBox :
        #num of time-steps box
        case 1 :
            if paramaterValue > 0 and paramaterValue < 10000 and type(paramaterValue) == int:
                model.timeStepsNum = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0 or paramaterValue > 10000:
                create_popup('Value is out of range')
            else :
                create_popup('Please enter a valid value')
        #number of initial infections
        case 2 :
            if paramaterValue > 0 and paramaterValue < 18720 and type(paramaterValue) == int:
                model.initialInfections = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0 or paramaterValue > 18720:
                create_popup('Value is out of range')
            else :
                create_popup('Please enter a valid value')
        #travel radius
        case 3 :
            if paramaterValue > 0 and paramaterValue < 5 and type(paramaterValue) == int:
                model.travelRadius = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0 or paramaterValue > 5:
                create_popup('Value is out of range')
            else :
                create_popup('Please enter a valid value')
        #infection rate box
        case 4 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.infectionRate = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                create_popup('Value is out of range')
            else :
                create_popup('Please enter a valid value')
        #recovery rate box
        case 5 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.recoveryRate = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                create_popup('Value is out of range')
            else :
                create_popup('Please enter a valid value')
        #fatality rate box
        case 6 :
            if paramaterValue > 0.0 and paramaterValue < 1.0 and type(paramaterValue) == float:
                model.fatalityRate = paramaterValue
                clear(self,model,self.squareCanvas,self.squareGrid)
            elif paramaterValue < 0.0 or paramaterValue > 1.0:
                create_popup('Value is out of range')
            else :
                create_popup('Please enter a valid value')
```

This now meant that whenever an incorrect value was entered into a parameter box an error message would pop-up, customised to the parameter box and error case as the example shown below:



Therefore, I had successfully implemented this function and completed the test case for a suitable error message appearing for when the user enters an incorrect value.

Iteration 2: Simulation Model

Neighbouring Agents within Travel Radius Algorithm

Conditional Statement	Intended Output	Actual Output
IF neighbour within maximum x and y coordinates	Will append agents within the maximum x and y coordinate radius to the neighbour_agents list	WORKS AS INTENDED

Iteration 3: Simulation Display

SIR Category	Intended Colour Output	Actual Colour Output
Susceptible	White	WHITE
Infected	Pink	PINK
Recovered	Green	GREEN
Fatality	Red	RED

Iteration 4: Infection Algorithm, Recovery Algorithm, Fatality Algorithm and Travel Radius

Infection Algorithm

Conditional Statement	Intended Output	Actual Output
IF randominteger <= infectionRate	Will change the agent's state to 'infected'	AGENT'S STATE IS CHANGED TO INFECTED
IF agent NOT IN neighbouring_agents	Will not consider agent in the algorithm	AGENT'S STATE STAYS AS SUSCEPTIBLE

Recovery Algorithm

Conditional Statement	Intended Output	Actual Output
IF randominteger <= recoveryRate	Will change the agent's state to 'recovered'	AGENT'S STATE IS CHANGED TO RECOVERED
IF agent.state NOT 'infected'	Will not consider agent in the algorithm	AGENT'S STATE STAYS AS SUSCEPTIBLE

Fatality Algorithm

Conditional Statement	Intended Output	Actual Output
IF randominteger <= recoveryRate	Will change the agent's state to 'fatality'	AGENT'S STATE IS CHANGED TO FATALITY
IF agent.state NOT 'infected'	Will not consider agent in the algorithm	AGENT'S STATE REMAINS AS SUSCEPTIBLE

Iteration 5: Pre-sets

Full Simulation Page Pre-set List

Input	Intended Output	Actual Output
User Clicks on Pre-set Name	Associated pre-set values are loaded from the database and used in the current simulation	NOT YET IMPLEMENTED

List of Pre-set Page

Input	Intended Output	Actual Output
User clicks on Cut button then clicks on a pre-set	Corresponding named pre-set entity will be removed from the SQL database	NOT YET IMPLEMENTED

Editable Pre-set Page

Input	Intended Output	Actual Output
User clicks on Save Button	The pre-set values will be added to the data base as a named entity	NOT YET IMPLEMENTED

Inputs and Outputs

Inputs	Process	Intended Output	Actual Output
User clicks on Full Simulation Button	Initialises the 'menu' class which creates the 'Full Simulation' page	'Full Simulation' page is displayed with text boxes and icons	'Full Simulation' PAGE IS DISPLAYED
User clicks on 'Pre-sets' button	Initialises the 'menu' class which creates the 'Pre-sets' page	'Pre-sets' page will be displayed with list of loadable pre-sets and icons	'Pre-sets' PAGE IS DISPLAYED
User clicks on 'Tutorial' button	Initialises the 'menu' class which creates the 'Tutorial' page	'Tutorial' page will be displayed with a central text box	'Tutorial' PAGE IS DISPLAYED

Parameters are inputted by user	Parameters class will be called, with attributes becoming values that the user has inputted	Parameters class will be used by the simulation loop to define simulation characteristics	WORKS AS INTENDED
User clicks on a Loadable Pre-Set	User will click a pre-set title in a list of them to load a set of saved parameters into the simulation	Pre-set values will be loaded from the database. Attributes of the class 'Parameters' will be changed to the pre-set values	NOT YET IMPLEMENTED
User clicks on Play / Pause Button	Will pause at the current time-step of the simulation, will resume from the current time-step of the simulation	Simulation will resume from its current time-step, or it will pause at its current time-step	WORKS AS INTENDED
User clicks on Forward or Back Buttons	Will increment the simulation forward by one time-step, or will move the simulation back one time-step	Simulation will move forward by one time-step or move backwards by one time-step	WORKS AS INTENDED
User clicks on Cut Pre-set Button, then clicks on a Pre-set	Will remove the selected pre-set from the database	The selected pre-set will be removed from the 'Editable pre-sets list' and database	NOT YET IMPLEMENTED
User clicks on Add Pre-set button	Initialises the 'menu' class which will creates the 'Editable Pre-set' page	'Editable Pre-set Page' will be displayed with parameter input boxes	NOT YET IMPLEMENTED
User clicks on Exit Button in Editable Pre-set Page	Will initialise the 'menu' class to return to the 'Pre-sets' page	'Editable Pre-set Page' will close, and 'Pre-sets' page will be displayed	NOT YET IMPLEMENTED
User clicks on Save Button in Editable Pre-set Page	Will initialise the 'menu' class to return to the 'Pre-sets' page, adds new pre-set to database	'Editable Pre-set Page' will close, and 'Pre-sets' page will be displayed, entered pre-set values will be added to the database	NOT YET IMPLEMENTED

Evaluation

Test Scenarios

In this section I will be using the test scenarios outlined in the design section to make sure that all components of the program are working.

Since a few sections have not been implemented due to the time constraints of the project, I will be only using the test scenarios that are directly relevant to the project in its current state:

1 – the user wants to run a simulation, moving backwards and forwards

- selects the full simulation page from the main menu
- enters the parameters they want for their simulation and hits the play button
- presses the backwards arrow button a number of times
- presses the forwards arrow button a number of times
- pauses the simulation and repeats the two earlier steps
- continues the simulation using the play button
- exits simulation

Test Scenario Video 1 Attached

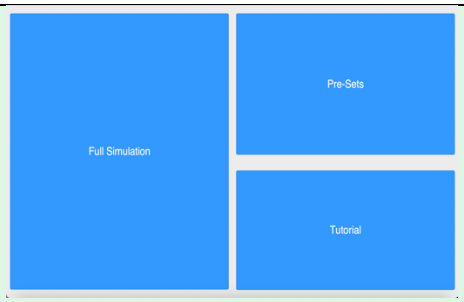
2 – the user wants to run a simulation, changing the parameters while the simulation is in progress

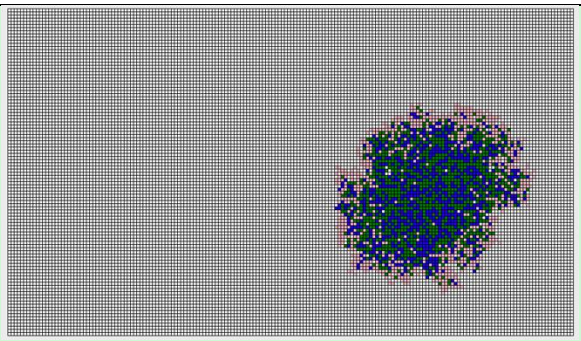
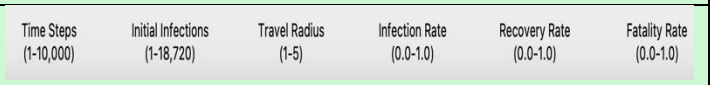
- selects the full simulation page from the main menu
- enters the parameters they want for their simulation and hits the play button
- changes a parameter in one of the parameter boxes while the simulation is in progress
- exits simulation

Test Scenario Video 2 Attached

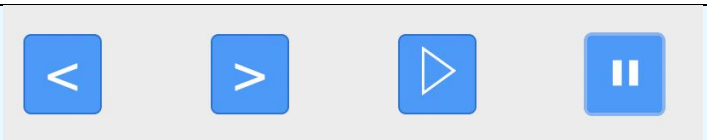
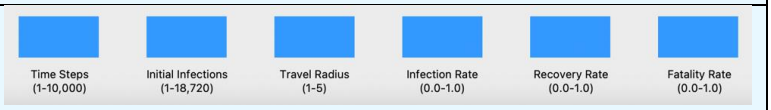
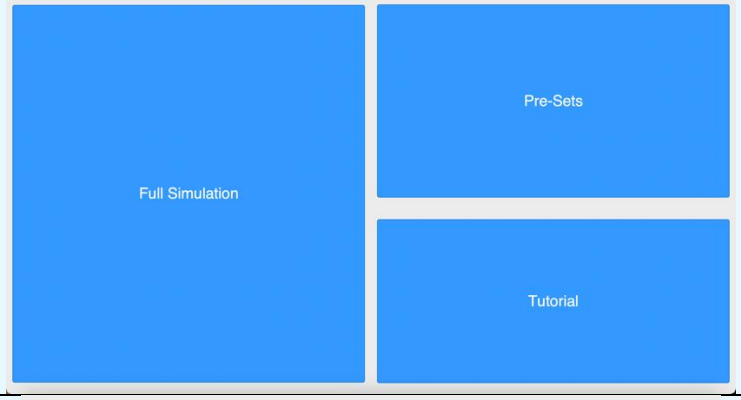
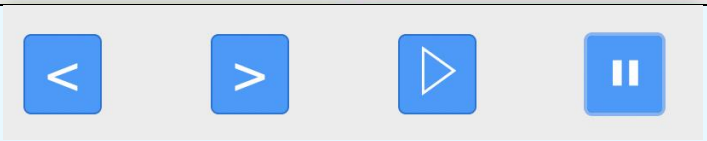
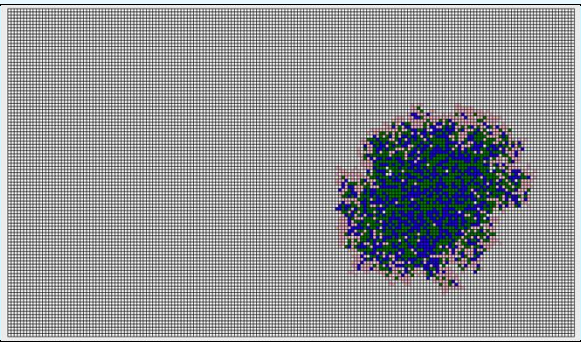
Success Criteria Evaluation

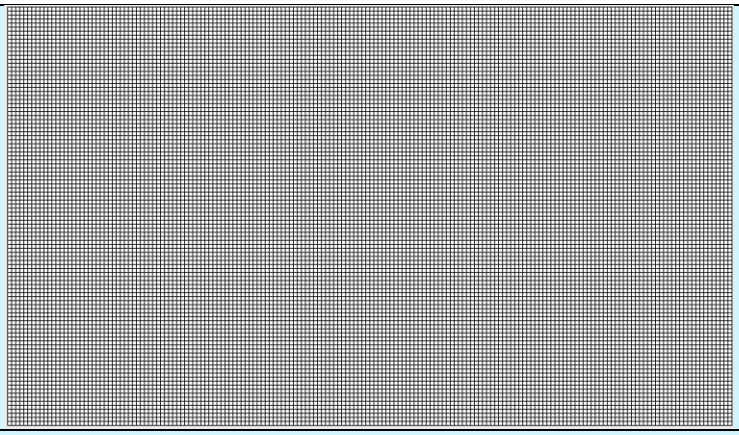
Since there are some parts of the program I haven't yet implemented, for the evaluation I have only included the success criteria for the parts of my program that have been properly implemented.

Success Criteria	Reason	Evidence
Main Menu	To show the different sections to the user	
Buttons: Full Simulation Tutorial Pre-sets	To allow the user to navigate and use the different functionalities of the program	**PICTURED ABOVE**
Quit Button	To allows the user to exit the program	
Full Simulation Section	Allow the user to run a simulation of how a disease moves through a population	**DEMONSTRATED BY TEST SCENARIOS ABOVE**

2D Grid Graphic	Presents the data through a grid which visually demonstrates how the disease moves through the population using squares to represent the population	
Titled List of Pre-sets Templates	Allows the user to select and load pre-set values into the simulation	**NOT YET IMPLEMENTED**
Play Button	Allows the user to start the simulation in real-time	
Pause Button	Allows the user to pause the simulation	
Forward Arrow Button	Allows the user to iterate the simulation by one time-step forward	
Backward Arrow Button	Allows the user to iterate the simulation by one time-step backward	
Parameter Titles	Allows the user to see all of the parameters that can be changed for the simulation	
Parameter Input Boxes	Allows the user to change the values of different parameters in the simulation	
Exit Button	Allows the user to exit the section and return to the main menu	

User Requirements Evaluation

Feature	Justification	Evidence
Labelled Buttons	Allows user to easily navigate the program / pages	
Labelled Boxes	Allows user to easily know how each input box affects the outcome of the program	
Main Menu	Allows user to navigate the whole program from a single page, simplifies layout	
Large Buttons	Makes the GUI more intuitive, user can easily see different functions / pages of program	
Tutorial Page	Explains different functions of program to the user, increases understanding of the program	**NOT YET IMPLEMENTED**
SIR Colours (within grid)	Allows the user to see the state of a square at a glance with only three different colours	
SIR Colour Key	Allows the user to know which colour corresponds to which state an agent is in	

Squares (within grid)	Simplifies the population of the simulation to graphical shapes which can be understood by the user	
Pre-sets	Allows the user to load parameters into the simulation without having to manually input them every time the simulation is run	**NOT YET IMPLEMENTED**

Changes made post-evaluation:

Although the pre-sets and tutorial page where outside of my current project scope with the time I had left, I decided that implementing an SIR colour key would barely take any time and would add a lot of accessibility to my program, since the user would be able to interpret the grid-display properly and know what the colour of each square meant.

To do this all I did was add another label using the place() method as I had done earlier when adding labels to each parameter box. This label would sit just under the grid, using specific coordinates, and specify what each colour in the SIR model meant:

```
self.i7text = tk.Label(window, text="Key: White: Susceptible, Pink: Infected, Green: Recovered: Blue: Fatality")
self.i7text.place(x=31, y=575)
```

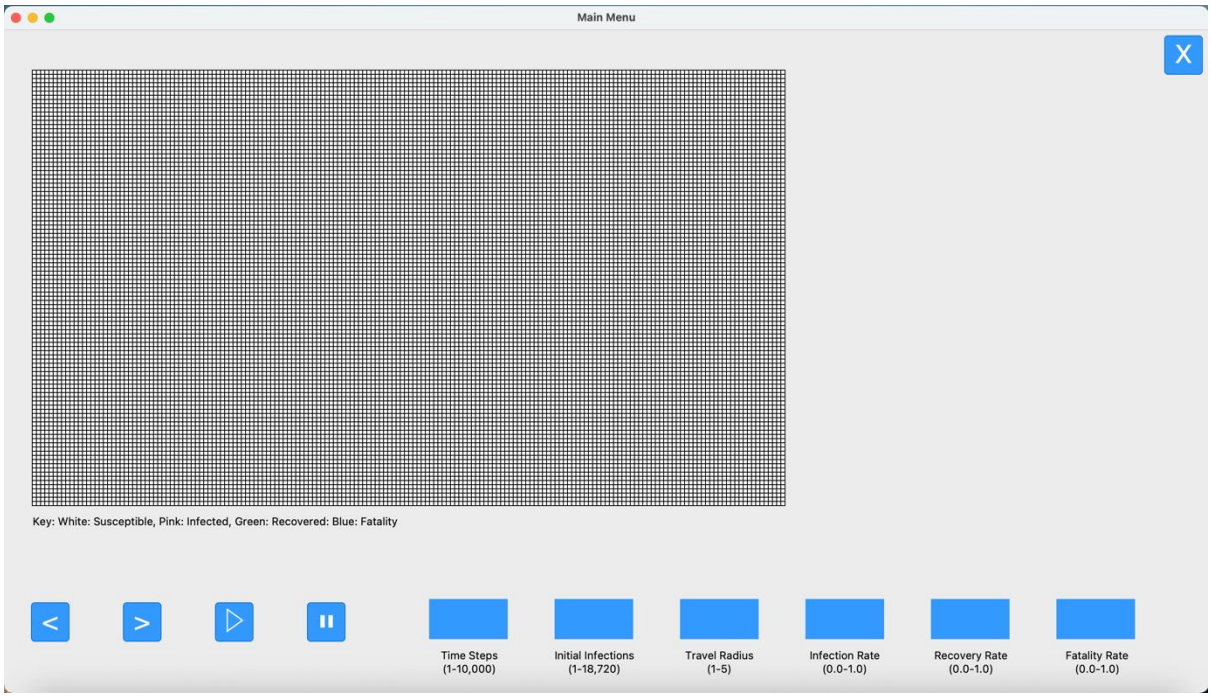
I then also added it to the list of object for that page:

```
self.objects = [self.b1,self.t1,self.t2,self.p1,self.p2,self.i1,self.i2,self.i3,self.i4,self.i5,self.i6,self.squareCanvas,
                self.i1text,self.i2text,self.i3text,self.i4text,self.i5text,self.i6text,self.i7text]
```

Finally, I modified the function that removes all objects from the page, making it so it included this new object I had added to the objects list by increasing the range of objects that it would use the destroy() method for by one:

```
def get_page(items,pageNum,model):
    #stops model if currently running
    #iterates through items list and gets rid of each item from page
    [items[i].pack_forget() for i in range(len(items)) if i < 13 ]
    [items[i].destroy() for i in range(len(items)) if i > 13 ]
    #loads the specified page based on number paramater
    if pageNum == 0:
```

This was the result:



This now meant that a colour key was included under the grid, specifying to the user what each colour meant thus meaning that the user requirement was fulfilled, and the program was made more accessible.

Stakeholder Testing

For this section I got some stakeholders to complete a series of tasks outlined below and recorded their experience in a feedback sheet. This will help evaluate whether I have met the user requirements set out in the design section of this program as well as component of the program I can improve in the future.

Tasks for the Stakeholder to Complete:

- open the simulation page
- enter the parameters they want to run
- start the simulation
- move the simulation either forwards or backwards
- pause the simulation
- anything else they find interesting

Group 1 Testing:

Stakeholder 1: Taylor

Question	**Answer**
Do you find the UI simple and intuitive?	Yes
Could you understand what was going on during the simulation?	Yes
Was the program easy to navigate?	Yes
Did you find that the responsiveness of the program was to your satisfaction?	A little slow but that doesn't detract from the experience
Are there any features you would want to see added to the program?	Tutorial and the pre-sets

Is there any part of the program that you would change?	No
Do you think this is effective in understanding the spread of a disease through a population?	Yes
Could you rate the usability of the program on a scale from 1-10?	8
Could you rate your general satisfaction with the program on a scale from 1-10?	8

Stakeholder 2: Josh

Question	**Answer**
Do you find the UI simple and intuitive?	Yes
Could you understand what was going on during the simulation?	Yes/Maybe
Was the program easy to navigate?	Easy
Did you find that the responsiveness of the program was to your satisfaction?	Yes
Are there any features you would want to see added to the program?	Be able to set timer for when recovery starts as well as the tutorial
Is there any part of the program that you would change?	No
Do you think this is effective in understanding the spread of a disease through a population?	Yes
Could you rate the usability of the program on a scale from 1-10?	6
Could you rate your general satisfaction with the program on a scale from 1-10?	9

Stakeholder 3: Katie

Question	**Answer**
Do you find the UI simple and intuitive?	I found that the main menu was easy to navigate around, I do wish that it looked more aesthetically pleasing but it does its job well
Could you understand what was going on during the simulation?	Yes
Was the program easy to navigate?	It was not clear that you had to click enter when putting in the values
Did you find that the responsiveness of the program was to your satisfaction?	Yes
Are there any features you would want to see added to the program?	A clear button, an explanation as to how you enter in the values and a counter at the side of the screen with the current statistics of number of infected, number dead and number recovered etc
Is there any part of the program that you would change?	Just the existing visual aspects
Do you think this is effective in understanding the spread of a disease through a population?	Yes

Could you rate the usability of the program on a scale from 1-10?	7
Could you rate your general satisfaction with the program on a scale from 1-10?	9

Group 2 Testing:

Stakeholder 1: Mr Ovett

Question	**Answer**
Do you find the UI simple and intuitive?	Yes I find the program easy to navigate
Could you understand what was going on during the simulation?	I got the idea of it but I think it would be better if there was some more clarification on how the population is affected by the disease
Was the program easy to navigate?	Yes it was
Did you find that the responsiveness of the program was to your satisfaction?	Yes the program responsiveness was good
Are there any features you would want to see added to the program?	Perhaps some stats at the end of the simulation that can summarise the facts about the population, such as how many people died and the rate of mortality
Is there any part of the program that you would change?	No I can't think of anything
Do you think this is effective in understanding the spread of a disease through a population?	Although it can be improved I think it does get thel point across
Could you rate the usability of the program on a scale from 1-10?	7
Could you rate your general satisfaction with the program on a scale from 1-10?	7

The main feedback from this program was that the simulation part of the program had been executed well with the UI especially being intuitive and easy to use however the absence of the other components in the program were noticed. When asked, their most-desired feature was the tutorial. Funnily enough the pre-sets feature's absence wasn't noticed by the majority of the stakeholders with the tutorial being the most noticed absence. This is because there are some features of the program that aren't easily displayed to the user without the inclusion of a tutorial, e.g the user has to press enter on the keyboard to submit a parameter value however in each stakeholder test this wasn't realised by the stakeholder. Another source of confusion was the time-steps when a few stakeholders didn't realise the meaning of the word and were confused the parameter box's purpose.

All of these stakeholder issues would be resolved with the inclusion of a tutorial section since it would specify all of the specific cases that the user might need to know about. Overall I'm satisfied with the state that the program is in since with the parts that have been implemented being in a great state, with the stakeholders being satisfied with them too. I have taken the feedback from the stakeholders and filled out the user requirements table with those the requirements that have been met and those that have not based on stakeholder feedback:

Feature	**Has it been met?**
Labelled Buttons	YES
Labelled Boxes	YES

Main Menu	YES
Large Buttons	YES
Tutorial Page	NO
SIR Colours (within grid)	YES
SIR Colour Key	YES
Squares (within grid)	YES
Pre-sets	NO

Incomplete Components

There are two main components to my program that haven't been implemented and that is the database pre-set loading functionality as well as the tutorial section, however the simulation section has been fleshed out fully.

I decided to leave out the pre-set storage component of my program early on in development since I knew it would be a complex component to implement especially with the implementation of an SQL database which would take a lot of time and extra research to implement. It also wouldn't add a significant amount to the program since users could find out the parameter values for different diseases and write them in themselves.

When I asked a stakeholder about this, they stated that although they could see the user for storing different sets of values for the simulation, it wasn't essential but would be a good feature to implement for more useability.

The second component that was left out was the tutorial and this was purely due to time constraints. It would be an easy component to implement however the time available to implement this isn't there. Due to the program meeting all the user requirements, meaning that the program is accessible, in my opinion all of the controls for the simulation are fairly intuitive due to the symbols and parameter box labels, however I may be biased due to my involvement in developing the program.

Finally, the UI is incomplete due to the missing components stated above. This is noticeable when traversing the program since both the 'tutorial' page button and 'pre-sets' page button both lead to blank pages with nothing on them. The reason this is not complete is that although I could remove both buttons and have the program consist of only the simulation page, since I want to add these features to the program at a later date outside of this project log I've decided to keep them there so that when I revisit the program there's no need to re-write the UI merely add to it

Limitations of my Program

I think the largest limitation is that I used tkinter as the framework for all components of this program. Although it did allow me to achieve everything, I wanted to do within the program it has many disadvantages with the main ones being the slow response time and convoluted coding process for aligning elements within windows. It also means that any user that wants to run this program will need to have tkinter installed as a perquisite which can be confusing to install for someone not well-versed in technology and since the main breadth of users of my program are biologists and not computer-scientists, it presents quite a limitation in accessibility. Another limitation is that an interpreter is required to run the program. This is unideal since it means the interpreter that will need to be installed alongside the program.

Another limitation of my program was the computing power my program needed to run. The computer that I used to develop this project and run it is a laptop with only eight gigabytes of ram.

Due to this when running the program, the responses to parameter changes were sometimes slow with the program taking a time to update. This is a limitation because if users install this program on even older computers, e.g ones that may be present in a school, it may not be able to run at the speed desired by the user.

How to Overcome these Limitations

The best way to overcome the module limitations would be to use a different module for the GUI, one that is much more intuitive and efficient as well as using a different language to write the program such as C. This would mean the program is easily distributable without the user having to install modules or know how to run a program from an interpreter.

Another way to overcome the hardware limitation of the program would be to review the code implement more efficient solutions to blocks of code as well as maybe using a more processor efficient language as stated above. The good thing is that this limitation can definitely be overcome during further development since the program isn't fundamentally inefficient, there are just small changes in the code that can be made to make the algorithms used more efficient and thus make the program run at the standard expected.

Maintenances

In this section of the evaluation, I will write about how maintainable the program is for developers who may want to change or add on features using the pre-written source code.

If this program were to be distributed on windows, it's maintainability is relatively assured. This is because it only uses one truly external module, mesa, while the other modules are main python modules that are packaged with python when it is installed. However, as the program currently stands, it is OS-dependent for mac OS. This is because the tkinter module implementation has an error which means buttons can't be implemented properly on mac devices, therefore I had to use an alternative module named tkintermacosx. This means that if the program was distributed, there would have to be a special case where if the use's system is mac, it will use that module specifically for some parts of the program, while if it was windows there would be no need for that module to be installed. Therefore, it's maintenance is much more assured since the main tkinter module is unlikely to undergo any significant changes that would affect the program whereas since the mac-version of the program would have to have tkintermacosx installed which is a third-party module and could undergo significant changes or become incompatible with the latest version of tkinter meaning that significant efforts will need to be taken to maintain this part of the program.

The other third-party modules that I have used in this program is called mesa. This may also need some maintenance since it is constantly updating with new, more efficient features being implemented with the last change to its's source code being weeks ago. Therefore, some efforts may be required to see if the program source-code is properly making use of mesa's updated features for increased efficiency.

Since the program is modular and split into classes and functions, all distinctly labelled, if stakeholders wanted to add more features these can easily be incorporated into the program code without changing the structure of the program due to its modularity which is a huge advantage and gives the program much more longevity than if modular programming was not used.

Further Development

The obvious choice for further development will be the implementation of the tutorial and pre-set components which were both planned to be implemented during the design phase but ultimately weren't able to be implemented due to time constraints.

Apart from those, the SIR model itself could've also been developed further since in real-life epidemiology practices differential equations are used to calculate the spread of the disease whereas my model simply used random chance. Another feature I would've liked to implement in this model how the population itself would respond to a disease. Herd immunity is already accounted for in the simulation however disease prevention measures such as vaccination or a cure have not been implemented into the simulation and, if more development time were give, would make for a more dynamic and realistic simulation of a disease spread.

A feature that one of my stakeholders suggested was changing the visual aspects of the program but although I agree with the fact that the GUI may look a bit plain, I think the most important thing that further development time should go towards is furthering the usability and features of the simulation itself so the program can provide much more functionality. Another reason that this GUI wasn't developed further to look more aesthetically pleasing was the module being used which was tkinter. This is because it was initially designed for simple GUIs and allows for almost zero styling options.

Another feature that I would've liked to implement would've been a priority feature. For example, if a population member was part of an older demographic, they will have a higher chance of being infected than a younger member and therefore a higher priority in the infection algorithm. This would've also allowed for some more complex algorithms such as for a multi-level feedback queue algorithm to be implemented which would allow for a more complex program as well as tying in nicely to the skills learned in the A-level computer science specification.

Overall, I am happy with the state that the program is and while I do recognise that there are components missing, I feel as all the essential features are implemented, fulfilling the core concept stated at the start of this NEA which was to represent the spread of a disease graphically and I look forward to developing this program further independently.

Final Code

"main_window..py"

```
from population import PopulationModel
import tkmacosx as tkmacosx
import tkinter as tk
import random

window = tk.Tk()
window.title("Main Menu")
window.geometry("1440x900")

def draw_canvas(canvas) :
      #reference dictionary to link agent and square coords
      grid = {}
      #nested for loop for coordinates
      ix = -1
      iy = -1
      for y in range(3,900,5) :
            ix += 1
```

```
            iy = -1
            for x in range(3,520,5) :
                  iy += 1
                  #creates and packs squares
                  rectangle = canvas.create_rectangle(y, x, y+5, x+5,
fill="white")
                  grid[ix,iy] = rectangle
      return grid

def categorise_agents(model,canvas,squares) :
      #iterates through agents in the model
      for a in model.grid :
            #stops model if currenlty running
            if a.state == 'S' :
                  canvas.itemconfig(squares[a.pos], fill='white')
            elif a.state == 'I' :
                  canvas.itemconfig(squares[a.pos], fill='pink')
            elif a.state == 'R' :
                  canvas.itemconfig(squares[a.pos], fill='green')
            elif a.state == 'F' :
                  canvas.itemconfig(squares[a.pos], fill='blue')

def retrieve_input(self,model,inputText,inputBox):
      #retrieves the text from the text box, 1.0 is beginning, tk.END is
end
      paramaterValue = inputText.get("1.0",tk.END)
      #deletes text from input box so isnt added on next time function is
run
      inputText.delete("1.0",tk.END)
      #validates whether the value is an integer or float
      try:
            paramaterValue = int(paramaterValue)
      except ValueError:
            paramaterValue = float(paramaterValue)

      #input validation as outlined via test plan
      match inputBox :
            #num of time-steps box
            case 1 :
                  if paramaterValue > 0 and paramaterValue < 10000 and
type(paramaterValue) == int:
                        model.timeStepsNum = paramaterValue
                        clear(self,model,self.squareCanvas,self.squareGrid)
                  elif paramaterValue < 0 or paramaterValue > 10000:
                        create_popup('Value is out of range')
                  else :
                        create_popup('Please enter a valid value')
            #number of initial infections
            case 2 :
                  if paramaterValue > 0 and paramaterValue <= 18720 and
type(paramaterValue) == int:
                        model.initialInfections = paramaterValue
                        clear(self,model,self.squareCanvas,self.squareGrid)
                  elif paramaterValue < 0 or paramaterValue > 18720:
                        create_popup('Value is out of range')
                  else :
                        create_popup('Please enter a valid value')
```

```
            #travel radius
            case 3 :
                  if paramaterValue > 0 and paramaterValue <= 5 and
type(paramaterValue) == int:
                        model.travelRadius = paramaterValue
                        clear(self,model,self.squareCanvas,self.squareGrid)
                  elif paramaterValue < 0 or paramaterValue > 5:
                        create_popup('Value is out of range')
                  else :
                        create_popup('Please enter a valid value')
            #infection rate box
            case 4 :
                  if paramaterValue >= 0.0 and paramaterValue <= 1.0 and
type(paramaterValue) == float:
                        model.infectionRate = paramaterValue
                        clear(self,model,self.squareCanvas,self.squareGrid)
                  elif paramaterValue < 0.0 or paramaterValue > 1.0:
                        create_popup('Value is out of range')
                  else :
                        create_popup('Please enter a valid value')
            #recovery rate box
            case 5 :
                  if paramaterValue >= 0.0 and paramaterValue <= 1.0 and
type(paramaterValue) == float:
                        model.recoveryRate = paramaterValue
                        clear(self,model,self.squareCanvas,self.squareGrid)
                  elif paramaterValue < 0.0 or paramaterValue > 1.0:
                        create_popup('Value is out of range')
                  else :
                        create_popup('Please enter a valid value')
            #fatality rate box
            case 6 :
                  if paramaterValue >= 0.0 and paramaterValue <= 1.0 and
type(paramaterValue) == float:
                        model.fatalityRate = paramaterValue
                        clear(self,model,self.squareCanvas,self.squareGrid)
                  elif paramaterValue < 0.0 or paramaterValue > 1.0:
                        create_popup('Value is out of range')
                  else :
                        create_popup('Please enter a valid value')

def create_button(ppage,ptext,pheight, pwidth):
      #assigns constant and paramater values to a new button
      return tkmacosx.Button(ppage,borderless=1,font=("Helvetica",
30),text=ptext,height=pheight,width=pwidth,bg='#3399ff',fg='#FFFFFF')

def create_popup(message):
      #cretes new window
    popup = tk.Toplevel(window)
    popup.title("Error")
    #sets paramater text in window
    label = tk.Label(popup, text=message)
    #aligns in center of screen
    label.pack(pady=10,padx=10)
    #adds a close button for the user
    close_button = tk.Button(popup, text="Close", command=popup.destroy)
    close_button.pack(pady=10)
```

```
    #specifies width,heigh of the popup window itself, then the coords to
place the window within the larger simulation page
    popup.geometry('178x90+631+405')

def get_page(items,pageNum,model):
      #stops model if currently running
      #iterates through items list and gets rid of each item from page
      [items[i].pack_forget() for i in range(len(items)) if i < 13 ]
      [items[i].destroy() for i in range(len(items)) if i > 13 ]
      #loads the specified page based on number paramater
      if pageNum == 0:
            MainMenu()
      elif pageNum == 1 :
            FullSimulation(model)
      elif pageNum == 2:
            PreSets(model)
      elif pageNum == 3:
            Tutorial(model)

def time_loop(model,canvas,squares) :
      #when time-step called pause set to false so simulation starts
      model.pause = False
      #while time-steps do not exceed user set limit (while model is out of
date) and pause is false
      while model.schedule.time != len(model.grid[0][0].memory) :
            for i in model.grid :
                  i.state = i.memory[model.schedule.time]
            categorise_agents(model,canvas,squares)
            window.update()
            model.schedule.time += 1
      #while the model is up to date and not paused
      while model.schedule.time < model.timeStepsNum and model.pause ==
False:
            #calls the model and agent steps methods
            model.step()
            #changes grid colour
            categorise_agents(model,canvas,squares)
            #updates GUI
            window.update()

def stop(model) :
      model.pause = True

def backward_time_step(self,model,canvas,squares) :
      #means that the time-step time will be set to the previous
      self.model.schedule.time -= 1
      #pauses the model
      stop(model)
      for i in model.grid:
            #agent state will be set to previous time-step state
            i.state = i.memory[self.model.schedule.time]
      #updates colours
      categorise_agents(model,canvas,squares)
      window.update()

def forward_time_step(self,model,canvas,squares) :
      #if model is up to date
```

```
		if self.model.schedule.time == len(model.grid[0][0].memory) :
			model.step()
			#changes colours
			categorise_agents(model,canvas,squares)
			window.update()
		#if model is outdated
		else :
			#pauses model
			stop(model)
			self.model.schedule.time += 1
			for i in model.grid :
				#gets state of agents one ahead
				i.state = i.memory[self.model.schedule.time]
			#changes colours
			categorise_agents(model,canvas,squares)
			window.update()

def clear(self,model,canvas,squares) :
		#resets population and neighbouring agents i.e if travel radius
changes
		model.init_neighbour_agents()
		#resets time of simulation to zero
		self.model.schedule.time = 0
		#stops simulation in progress
		stop(model)
		#randomly selects agents to be selected based on initial infections
paramater
		initialInfectedAgents = [random.randint(0,18720) for i in
range(model.initialInfections)]
		for i in model.grid :
			#resets memory dictionary so each agent has no memor of
previous states
			i.memory = {}
			#if agent is in initial infections list set state to infected,
else set to susceptible
			if i.unique_id in initialInfectedAgents :
				i.state = 'I'
			else:
				i.state = 'S'

		#colours agents based on state
		categorise_agents(model,canvas,squares)
		#updates GUI
		window.update()

class MainMenu :
		#initialises the objects on the page
		def __init__(self) :
			#assigns the model
			self.model = PopulationModel()

			#creates each button for the main menu page
			self.b1 = create_button(window,"Full Simulation",750,700)
			self.b2 = create_button(window,"Pre-Sets",385,700)
			self.b3 = create_button(window,"Tutorial",385,700)
			#adds each button to a list
			self.objects = [self.b1,self.b2,self.b3]
```

```
            #packs each button into page
            self.b1.pack(anchor='center',side='left',pady=20,padx=10)
            self.b2.pack(anchor='n',pady=20,padx=10)
            self.b3.pack(anchor='s',pady=20,padx=10)

            #runs get_page() method to run with objects list when button is
clicked
            self.b1.configure(command=lambda:
get_page(self.objects,1,self.model))
            self.b2.configure(command=lambda:
get_page(self.objects,2,self.model))
            self.b3.configure(command=lambda:
get_page(self.objects,3,self.model))

class FullSimulation :
      def __init__(self,model) :
            #assigns model
            self.model = model
            #initiates neighbouring agents list for each agent
            model.init_population()
            model.init_neighbour_agents()

            #back button
            self.b1 = create_button(window,"X",50,50)
            #temporary assignment to clear function
            self.b1.configure(command=lambda:
get_page(self.objects,0,model))

            #canvas
            self.squareCanvas = tk.Canvas(window,bg="#FFFFFF", height=521,
width=901)

            #time-step buttons
            self.t1 = create_button(window,"<",50,50)
            self.t2 = create_button(window,">",50,50)

            #play/pause buttons buttons
            self.p1 = create_button(window,"▷",50,50)
            self.p2 = create_button(window,"□",50,50)

            #defines input boxes
            self.i1 =
tk.Text(window,width=4,height=1.25,bg='#3399ff',fg='#FFFFFF',font=("Helveti
ca",40))
            self.i2 =
tk.Text(window,width=4,height=1.25,bg='#3399ff',fg='#FFFFFF',font=("Helveti
ca",40))
            self.i3 =
tk.Text(window,width=4,height=1.25,bg='#3399ff',fg='#FFFFFF',font=("Helveti
ca",40))
            self.i4 =
tk.Text(window,width=4,height=1.25,bg='#3399ff',fg='#FFFFFF',font=("Helveti
ca",40))
            self.i5 =
tk.Text(window,width=4,height=1.25,bg='#3399ff',fg='#FFFFFF',font=("Helveti
ca",40))
```

```
            self.i6 =
tk.Text(window,width=4,height=1.25,bg='#3399ff',fg='#FFFFFF',font=("Helveti
ca",40))

            self.b1.pack(anchor='n',side='right',pady=5,padx=5)

            self.squareCanvas.pack(side='top',anchor='w',pady=45,padx=30)

            self.t1.pack(anchor='s',side='left',pady=60,padx=30)
            self.t2.pack(anchor='s',side='left',pady=60,padx=30)

            self.p1.pack(anchor='s',side='left',pady=60,padx=30)
            self.p2.pack(anchor='s',side='left',pady=60,padx=30)

            #assigns backwards time-step function to previous arrow button
            self.t1.configure(command=lambda:
backward_time_step(self,model,self.squareCanvas,self.squareGrid))
            #assings forward time-step function to forward arrow button
            self.t2.configure(command=lambda:
forward_time_step(self,model,self.squareCanvas,self.squareGrid))

            #assigns time-step function to play button
            self.p1.configure(command=lambda:
time_loop(model,self.squareCanvas,self.squareGrid))
            #assigns pause function to pause button
            self.p2.configure(command=lambda: stop(model))

            #packs input boxes
            #have to be in reverse to be assigned correctly due to packing
from the right
            #fatality rate
            self.i6.pack(side='right',pady=60,padx=25)
            #recovery rate
            self.i5.pack(side='right',pady=60,padx=25)
            #infection rate
            self.i4.pack(side='right',pady=60,padx=25)
            #travel radius
            self.i3.pack(side='right',pady=60,padx=25)
            #initial infections
            self.i2.pack(side='right',pady=60,padx=25)
            #time steps num
            self.i1.pack(side='right',pady=60,padx=25)

            #text labels for user accessibility
            self.i1text = tk.Label(window, text="Time Steps\n(1-10,000)")
            self.i1text.place(x=519, y=735)

            self.i2text = tk.Label(window, text="Initial Infections\n(1-
18,720)")
            self.i2text.place(x=655, y=735)

            self.i3text = tk.Label(window, text="Travel Radius\n(1-5)")
            self.i3text.place(x=811, y=735)

            self.i4text = tk.Label(window, text="Infection Rate\n(0.0-
1.0)")
            self.i4text.place(x=960, y=735)
```

```
            self.i5text = tk.Label(window, text="Recovery Rate\n(0.0-1.0)")
            self.i5text.place(x=1109, y=735)

            self.i6text = tk.Label(window, text="Fatality Rate\n(0.0-1.0)")
            self.i6text.place(x=1266, y=735)

            self.i7text = tk.Label(window, text="Key: White: Susceptible,
Pink: Infected, Green: Recovered: Blue: Fatality")
            self.i7text.place(x=31, y=575)

            self.objects =
[self.b1,self.t1,self.t2,self.p1,self.p2,self.i1,self.i2,self.i3,self.i4,se
lf.i5,self.i6,self.squareCanvas,

      self.i1text,self.i2text,self.i3text,self.i4text,self.i5text,self.i6te
xt,self.i7text]

            # #calls main menu class
            # self.b1.configure(command=lambda:
get_page(self.objects,0,self.model))

            #binds retrieve input function to input box whenever enter key
is pressed
            self.i1.bind("<Return>",lambda event:
retrieve_input(self,model,self.objects[5],1))
            self.i2.bind("<Return>",lambda event:
retrieve_input(self,model,self.objects[6],2))
            self.i3.bind("<Return>",lambda event:
retrieve_input(self,model,self.objects[7],3))
            self.i4.bind("<Return>",lambda event:
retrieve_input(self,model,self.objects[8],4))
            self.i5.bind("<Return>",lambda event:
retrieve_input(self,model,self.objects[9],5))
            self.i6.bind("<Return>",lambda event:
retrieve_input(self,model,self.objects[10],6))

            #creates reference square grid
            self.squareGrid = draw_canvas(self.squareCanvas)
            #draws grid
            self.squareCanvas.pack()

class PreSets :
      def __init__(self,model) :
            self.model = model
            self.b1 = create_button(window,"X",50,50)
            self.b1.pack(anchor='n',side='right',pady=5,padx=5)
            self.objects = [self.b1]

            self.b1.configure(command=lambda:
get_page(self.objects,0,model))

class Tutorial:
      def __init__(self,model) :
            self.b1 = create_button(window,"X",50,50)
            self.b1.pack(anchor='n',side='right',pady=5,padx=5)
            self.objects = [self.b1]
```

```
            self.b1.configure(command=lambda:
get_page(self.objects,0,model))

#loops the page so it's ineractable
main = MainMenu()


window.mainloop()
```

"population.py"

```
from numpy import random
import mesa

class PopulationAgent(mesa.Agent) :
      def __init__(self,unique_id,model):
            super().__init__(unique_id,model)
            self.state = 'S'
            self.neighbour_agents = []
            self.memory = {}

      def step(self):
            #infection algorithm
            #if the agent is infected
            if self.state == 'I' :
                  #iterates through agent list
                  for b in self.neighbour_agents :
                        #gets the agent at that coordinate position
                        neighbour =
self.model.grid.get_cell_list_contents([(b[0], b[1])])[0]
                        if neighbour.state == 'S' :
                              #chance of infection
                              randnum = (random.randint(1,10000)/10000)
                              if randnum <= self.model.infectionRate :
                                    neighbour.state = 'I'

class PopulationModel(mesa.Model):
      def __init__(self):
            self.travelRadius = 1
            self.initialInfections = 4
            self.infectionRate = 0.2
            self.recoveryRate = 0.01
            self.fatalityRate = 0.01
            self.timeStepsNum = 100
            self.width = 180
            self.height = 104
            self.pause = False

            #agents can't be within the same cell in the grid
            self.grid = mesa.space.SingleGrid(self.width, self.height,
True)
            #defines scheduler that will activate all agents randomly at
each time step
            self.schedule = mesa.time.RandomActivation(self)

      #defines a method to create population
```

```
      def init_population(self):
            #creates reference array of coordinates for agents to be added
to
            refCoordinates = [[x,y] for y in range(0,self.height) for x in
range(0,self.width)]

            #generates user-set amount of agent ids to be infected
            initialInfectedAgents = [random.randint(0,18720) for i in
range(self.initialInfections)]

            #loop to place agents in grid and scheduler
            for d,i in enumerate(refCoordinates):
                  a = PopulationAgent(d, self)

                  #if agent id in initial infected list
                  if d in initialInfectedAgents :
                        a.state = 'I'

                  #adds the agent to the scheduler
                  self.schedule.add(a)
                  #places agent within the grid
                  self.grid.place_agent(a, (i[0],i[1]))

      #defines a method to generate neighbouring agents list for each agent
      def init_neighbour_agents(self) :
            for a in self.grid:
                  #max and min x,y neighbour positions
                  minxPos, maxxPos = a.pos[0]-self.travelRadius,
a.pos[0]+self.travelRadius
                  minyPos, maxyPos = a.pos[1]-self.travelRadius,
a.pos[1]+self.travelRadius
                  #coordinate validation
                  if minxPos < 0 : minxPos = 0
                  if maxxPos > self. width -1:
                        maxxPos = self.width - 1
                  if minyPos < 0 : minyPos = 0
                  if maxyPos > self.height -1:
                        maxyPos = self.height - 1

                  #creates a list of all possible positions with range of
the max and min x,y neighbour positions
                  total_radius_positions = [[x,y] for x in
range(minxPos,maxxPos+1)
                                                                  for y in
range(minyPos,maxyPos+1)]

                  #removes the position of the agent it's finding the
neighbours of
                  total_radius_positions.remove([a.pos[0],a.pos[1]])

                  #assigns the list as an attribute
                  a.neighbour_agents = total_radius_positions

      def step(self):
            #iterates through population
            #counts number of suceptible agents
            for a in self.grid:
```

```
                  #assigns state to time-step number in dictionary
                  a.memory[self.schedule.time] = a .state
                  if a.state == 'I' :
                        #random chance
                        randnum = (random.randint(1,10000)/10000)
                        #avoids bias towards fatality or recovery by 50/50
chance
                        if random.randint(0,9) < 5 :
                              #chance of recovery
                              if randnum <= self.recoveryRate :
                                    a.state = 'R'
                              else:
                                    #chanve of fatality
                                    if a.state != 'R' and randnum <=
self.fatalityRate :
                                          a.state = 'F'
                        #flipped so in this case fatality gets cehcked
first
                        else :
                              if a.state != 'R' and randnum <=
self.fatalityRate :
                                          a.state = 'F'
                              else :
                                    if randnum <= self.recoveryRate :
                                          a.state = 'R'
            self.schedule.step()
```